

Asymptote Fault Tolerance (AFT): Maximal Deterministic Byzantine Safety — All-But-One ($n - 1$ of n), Relay-Free, Fully Attributable — in the AFT Model Delta

IOI

March 17, 2026

Conditionality and exact-figures note. Every result in this paper is proven within the AFT model delta M_{AFT} — an assumed common publication boundary whose non-circular construction is the subject of, and is tracked by, the AFT-CB program. The public claim surface is governed by the whitepaper’s §5.3 assumption ledger (A1–A9) and prints only the interim conditional claim until that program’s gates pass. Tolerance figures appear in this paper only in exact all-but-one form: “all-but-one ($n - 1$ of n) Byzantine safety.” A rounded percentage is never printed as a tolerance figure, because with $n = 16$ – 64 validators the true all-but-one maximum is 93.75%–98.4375%; the all-but-one statement is both stronger and exact.

Abstract

Asymptote Fault Tolerance (AFT) is a family of consensus and settlement constructions for high-throughput ordering, guardian-backed base finality, proof-carrying canonical ordering, and deterministic release of irreversible effects. Its central move is to stop treating dense positive voting as the only possible source of ordering truth. Instead, AFT separates throughput-critical publication and chain progression from authority-critical proof verification, omission detection, and effect collapse.

The protocol family has four interacting surfaces. *GuardianMajority* supplies the production base-finality path: a validator quorum certificate is necessary but not sufficient, because each proposal must also carry a registered guardian committee certificate anchored into shared evidence. *CanonicalOrdering* supplies the proof-carrying ordering path: a slot order is accepted because a canonical certificate is uniquely valid and recoverable from public inputs, not because a dense validator quorum voted the order into existence. *Asymptote* supplies the sealing path for irreversible effects: **BaseFinal** chain progress is upgraded to **SealedFinal** through deterministic witness or observer-backed collapse, with canonical observer challenges dominating unsafe releases. *NestedGuardian* supplies a witness-augmented mode for layered threshold constructions.

This paper is self-contained and normative for the protocol family. Its ordering subtheorem remains all-but-one ($n - 1$ of n) equal-authority ordering consensus, conditional on the AFT model delta’s assumed common publication boundary and on explicit bulletin-publication, cutoff, omission, protocol-native retrievability, and proof-soundness assumptions. The runtime also internalizes compact signed publication-frontier metadata on the hot path: positive committed blocks carry a predecessor-linked frontier summary and compact availability-receipt binding, while same-slot conflicts or stale parent links admit short objective contradiction objects. Those frontier objects bind the claimed bulletin / ordering surface, while the protocol-native retrievability plane internalizes deterministic extraction-or-abort on the cold path. For each closed slot boundary there exists at most one admissible public execution object, every conflicting fully specified

candidate admits a short public contradiction witness, durable state advances only through canonical collapse, and sealed-effect release is bound to the same collapse surface.

The lower-bound claim is explicit about model membership. The classical $n > 3f / f < n/3$ lower bound remains true in the classical partially synchronous message-passing model whose adversary may keep honest groups mutually ignorant of each other’s published views for the duration needed by the indistinguishability argument. AFT does not refute that theorem from inside its quantified model. It operates in the model where an admissible published artifact cannot be indefinitely suppressed from correct parties after stabilization, and it realizes that delta without a trusted relay: honest publishers send artifacts or erasure-coded shards directly over authenticated point-to-point links to validators or deterministic custodians, while validators publish only compact commitments, receipts, frontiers, and challenges as the echo layer. Payload delivery is relay-free; equivocation and omission detection still require compressed honest-party information flow.

In that model delta, and conditional on it, AFT is *relay-free*, *coordinator-free*, *pure-software deterministic all-but-one* ($n - 1$ of n) *Byzantine safety*: the classical threshold stops being load-bearing because the DLS adversary’s indefinite-suppression move is no longer legal, not because an impossibility theorem is violated inside its own model. Compact bindings stay on the hot path while cold-path recovery / reveal / reconstruction re-materialize the same close, abort, collapse, replay-prefix, execution-restart, and restart-ancestry surfaces through the AFT-native recovery-family contract. Accountable publication and validator removal remain operational reinforcement, not the source of correctness.

Stated at full strength: within the model delta, AFT occupies the two ceilings of the safety-accountability plane simultaneously, and both ceilings are theorems, not comparisons. All-but-one is not merely a large threshold but the *terminal* one — with zero honest signers, equivocation is unpreventable by any protocol in any model (the zero-honest bound), so no consensus method, in any model, can ever claim a stronger safety threshold than $n - 1$ of n ; AFT claims exactly that threshold, weight-independently, with a mechanized proof. Symmetrically, no protocol can attribute a violation beyond the signers of the conflicting certificates (the attribution cap), and AFT attributes exactly that set — every member of the violating configuration, individually, from the two certificates alone. There is no stronger safety claim and no stronger accountability claim available to any consensus protocol; AFT prints both, paired with the impossibility results that make them ceilings. The price is stated beside the claim: unanimity means one withholder freezes seal cadence (the forced trade of the unanimity bound — never chain progress, which continues on the live tier), live-tier liveness remains synchrony-conditional, and the classical lower bound is refused rather than refuted: its carrier — dense positive voting relayed through the same parties the adversary may control — is replaced by direct publication with compressed public echo, and in the replaced carrier the safety threshold ascends to its information-theoretic maximum.

1 Scope and Status

This paper contains the full architectural and theorem-level description of the instantiated AFT family. It does not require any companion prose specification to be understood.

The instantiated protocol surfaces are:

- **GuardianMajority** as the production guardian-backed base-finality mode.
- **Asymptote** as the two-tier sealing and irreversible-effects mode.
- **CanonicalOrdering** through the live **CommittedSurfaceV1** certificate path plus compact **PublicationFrontier** summaries attached to committed blocks.

- **NestedGuardian** as an experimental witness-augmented operational mode with safety proofs, bounded operational checks, and a finished composed-liveness / classical-agreement bridge rather than a remaining composed-liveness kernel.

Implementation correspondence is included below as exact symbol references. That section is non-normative. The protocol and theorem content of the paper stands on its own.

The strongest theorem surface of the runtime is singular, and it is conditional on the AFT model delta. AFT claims not only all-but-one ($n - 1$ of n) equal-authority ordering for the ordering layer, but relay-free, coordinator-free, pure-software deterministic all-but-one ($n - 1$ of n) Byzantine safety for durable ordering and sealed-effect safety in the AFT model delta, with public-state continuity as its protocol-internal safety carrier rather than dense positive quorum intersection. Compact hot-path bindings plus cold-path recovery / reveal / reconstruction materialize the same recovered close-or-abort, collapse, replay-prefix, execution-restart, and streamed restart-ancestry surfaces through the AFT-native recovery-family contract. Compact publication/frontier metadata is internalized on the live path throughout, and deeper recovered history is now ordinary endogenous AFT history through the canonical-collapse / replay retrievability anchor and the matching recovered-state historical retrievability surface.

Model discipline: same ambition, explicit delta. No protocol refutes a lower bound from inside the model over which that lower bound universally quantifies. AFT’s claim is therefore a membership claim: the classical DLS / PBFT lower bound applies to a model whose adversary can keep honest groups mutually ignorant of published admissible information for the duration needed by the indistinguishability argument. AFT operates in a model delta where that indefinite-suppression move is revoked for content-addressed admissible protocol artifacts after stabilization. The delta is not a hidden helper regime: it is realized by authenticated direct publisher-to-validator or publisher-to-custodian dissemination plus a compact echo layer of commitments, receipts, frontiers, and challenges. The substantive objection would be a hidden relay, privileged coordinator, TEE, bootstrap oracle, external recovery service, or circular realization of the visibility resource by the same Byzantine-majority suppression power. AFT has none in the stated model.

Scope precision is not a concession. A criticism that AFT is “permissioned only” is non-responsive because the promoted claim is a permissioned theorem over fixed authenticated authorities and epoch-scoped identity state. Replacing that domain with open-membership consensus is a target switch, not a rebuttal. The lower-bound question is instead mechanical: execute the classical adversary against AFT and identify the first illegal move. That move is indefinite suppression of a published admissible artifact from correct parties after stabilization. Everything before that point is a standard adversarial schedule; everything after that point is outside the classical model and inside AFT’s explicit model delta.

Singular theorem sentence. The theorem surface of this repository is now singular all the way up: AFT claims relay-free, coordinator-free, pure-software deterministic all-but-one ($n - 1$ of n) Byzantine safety in the AFT model delta. It breaks the classical threshold in the precise model-relative sense that the theorem is proved in the AFT model delta, where published admissible artifacts cannot be indefinitely suppressed from correct parties and where that delta is realized without a trusted relay. AFT does not ask credit for defeating DLS / PBFT while remaining inside the bare DLS / PBFT model. It identifies the exact adversary power removed, implements the removal through standard authenticated channels and compressed echo, and then proves the

close-or-abort safety surface in that model. Proof-carrying public evidence, endogenous historical retrievability, restart continuity, and collapse-gated durability are the realizing architecture of that claim, not a residual semantic gap.

Claim ladder. The full promoted claim should be read as a constructed ladder:

1. each closed slot boundary admits at most one admissible public execution object and every conflicting fully specified candidate admits a short objective obstruction witness,
2. durable state advances only through canonical collapse on that same close-or-abort surface,
3. irreversible effects release only through that same collapse surface.

Dimension	AFT realizing surface	Promoted singular claim
Agreement object	canonical close-or-abort / canonical collapse surface	one durable decision surface in M_{AFT}
Safety carrier	compact commitments, short contradiction objects, cold-path recovery, endogenous historical retrievability	dense positive quorum intersection is no longer load-bearing
History / bootstrap	canonical-collapse / replay historical retrievability root plus recovered-state historical retrievability surface	recovered history remains endogenous AFT state
Liveness burden	direct authenticated dissemination, compressed echo, recurrence/reduction/totality/collapse chain, persistent churn simulator	completed theorem surface under the AFT model delta

Target adversary/scheduler model and discharged liveness kernel. The singular theorem is interpreted under one explicit liveness model rather than a vague aspiration. Byzantine authorities may equivocate, omit, reorder, withhold reveals, withhold historical-retrievability objects, rotate maliciously, and crash/restart arbitrarily. Before stabilization the scheduler may delay or interleave publication, registry, recovery, archival, and restart traffic arbitrarily. After stabilization, authenticated channels between correct parties eventually deliver direct publisher sends and compact echo objects; repeatedly published admissible artifacts become visible and content-hash fetchable to correct parties; and governing profile/activation churn quiesces into bounded windows long enough for progress to compose. Under that model, the liveness bridge is the conjunction of five obligations: frontier-generation progress, canonical-resolution progress, recovery-completion progress, restart/historical-retrievability re-entry progress, and infinite composition of those four obligations across reassignment, outage, profile rotation, and archival page boundaries. That kernel is no longer open; it is the discharged bridge that supports the promoted AFT model-delta theorem.

The relevant liveness fact is not that AFT declines to claim a fully asynchronous theorem; the relevant fact is that it states the exact post-stabilization visibility resource used to halt the classical adversary. The protocol does not require universal timely receipt before the cutoff. It requires eventual delivery, persistence, and hash-fetchability of repeatedly published admissible artifacts after stabilization, realized by direct authenticated dissemination and compressed echo rather than by a trusted publication service.

Current artifact map and completion status. That kernel is now grounded in concrete repository artifacts rather than prose alone. Frontier-generation progress lands on the live `PublicationFrontier` path and its contradiction objects; canonical-resolution progress lands on canonical-order certificates, `CanonicalCollapseObject`, omission dominance, and recursive continuity; recovery-completion progress lands on the coded recovery-family contract and the recovered publication / close-or-abort surface; and restart/historical-retrievability progress lands on the AFT recovered-state surface plus canonical-history retrievability anchors and bounded-memory paging. Those obligations now have both concrete runtime carriers and a completed formal bridge. The formal package contains bounded churn witnesses, recurring cycle cores, a recurrence theorem, a finite reduction, a total classical- agreement history bridge, and a directly discharged semantic-collapse wrapper ending in `NestedGuardianRecoveryClassicalAgreementCollapse.tla`. The exact source of that recurrence / reduction / totality / collapse bridge is embedded in Appendix A, including the final wrapper in Appendix A.6.5. The recurrence / reduction / totality / collapse chain is not narrative glue. Its exact artifact chain is embedded verbatim in Appendix A, ending in the classical-agreement collapse wrapper itself. The runtime mirrors the same story through a persistent historical-retrievability churn/restart simulator over one evolving recovered-history state. The remaining work is therefore no longer theorem completion, but cleanup, stress expansion, and maintenance of the promoted singular claim.

2 Problem Statement

Classical permissioned Byzantine fault tolerance ties deterministic safety to dense positive voting by a large honest majority. Under that model, honest inclusion, total-order agreement, and finality all depend on most validators remaining both correct and available. That is the right model when the protocol is allowed to pay dense coordination cost and accept classical $3f + 1$ -style thresholds. It is the wrong model for a system that wants all of the following simultaneously:

- high-throughput publication and chain progression,
- equal-authority participation at the validator layer,
- deterministic rejection of incomplete or conflicting ordered views,
- stronger accountability than “someone must have voted wrong,”
- deterministic gating of irreversible effects.

AFT therefore changes the object of agreement.
Instead of asking:

- “Which order did the validator quorum choose?”

it asks:

- “Which order certificate is uniquely valid for this slot, and which effects are admissible after deterministic collapse?”

The family-level design goals are:

- keep ordering and publication sparse on the hot path,

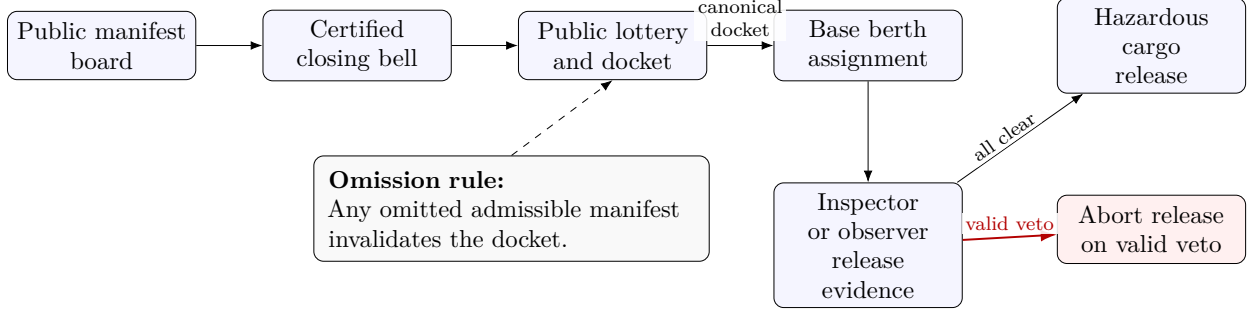


Figure 1: The thought experiment behind AFT: public admission, canonical ordering, base progress, and fail-closed release.

- keep effect release fail-closed,
- make omissions and conflicts objectively provable,
- keep all validators equally eligible to reveal the canonical order,
- move authority from dense yes-votes to verifiable evidence.

3 Thought Experiment: The Port of Public Manifests and Release Seals

Imagine an international port that handles ordinary cargo and hazardous cargo.

Every ship that wants to unload during tide h must post its cargo manifest to a public harbor board before the closing bell τ_h . The bell is not a private clock in the harbor master’s office. It is a certified public event. Once the bell rings, the harbor office takes every admissible manifest, applies the published tide lottery R_h , and derives one canonical docking docket O_h .

The office does not ask every captain to vote on the exact order of every container. It asks a simpler question: has someone produced the one valid docket certificate for the manifests that were publicly posted before the bell? If any dockworker can prove that an admissible manifest was omitted from the announced docket, that docket is rejected immediately.

Ordinary unloading may begin once the berth assignment is base-final. Hazardous cargo is stricter. It may leave the port only after one of two things happens:

- the required independent inspector guilds each sign a release certificate, or
- a deterministically assigned set of external captains publishes a public inspection transcript surface, the public red-flag board remains empty through the close bell, and the port issues the unique release docket implied by that surface.

Any valid red flag aborts release. The cargo does not “probably” leave the port. It does not leave.

Most of the port can be chaotic. Captains can lie, stall, or collude. What they cannot do is create a second valid docket for the same tide or a second valid hazardous-cargo release order, provided the public manifest board remains recoverable and the decisive proof surface stays objectively checkable.

The correspondence is direct:

Port object	AFT object
Public manifest board	Bulletin / DA surface B_h
Certified closing bell	Canonical cutoff τ_h
Tide lottery	Public randomness beacon R_h
Docking docket	Canonical order O_h
Docket certificate	Canonical-order certificate $OCert_h$
Omitted manifest challenge	Omission proof $\Omega(h, tx)$
Base berth assignment	BaseFinal
Independent inspector guilds	Witness committees
Sampled external captains	Equal-authority observers
Red-flag report	Canonical observer challenge
Hazardous cargo release seal	SealObject under SealedFinal

This thought experiment captures the essential AFT move: sparse operational progress, dense cheap verification, objective omission, and deterministic collapse for irreversible consequences.

4 Protocol Family Overview

AFT is best understood as four planes that compose:

Plane	Purpose	Throughput-critical work	Authority object
Dissemination / bulletin plane	Publish the candidate transaction surface	Data dissemination and bulletin commitments	Public bulletin commitment
Base-finality plane	Advance the chain and commit block identity	Proposal, QC formation, guardianized certification	Validator QC + guardian certificate
Canonical-ordering plane	Prove the unique slot order	Succinct verification, omission dominance	$OCert_h$
Sealed-effect plane	Gate irreversible effects	Asynchronous witness / observer evidence collection	SealedFinalityProof and SealObject

The protocol modes are:

Mode	Role	Current status
GuardianMajority	Production guardian-backed base finality	Production mode
CanonicalOrdering	Proof-carrying equal-authority ordering	Live through CommittedSurfaceV1
Asymptote	Two-tier sealing and sealed-effect release	Production sealing path
NestedGuardian	Witness-augmented layered threshold mode	Experimental operational mode / completed theorem bridge

The key invariant is:

$$\text{throughput-critical work} \neq \text{authority-critical work}$$

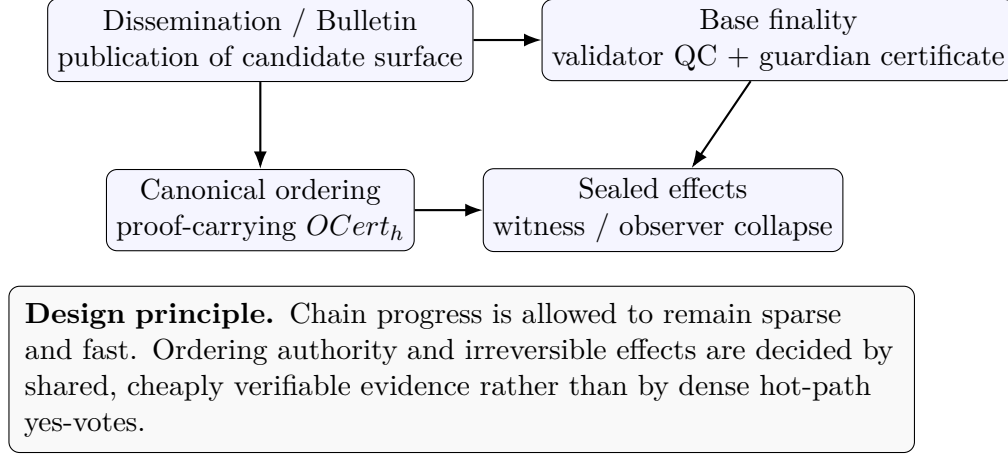


Figure 2: AFT separates publication, base progress, canonical ordering, and sealed effect release, then recomposes them through shared evidence.

5 System Model and Notation

5.1 Actors

- **Validator:** participates in proposal, vote, verification, and chain execution.
- **Guardian committee:** the registered threshold-signing committee protecting one validator’s proposal decisions.
- **Guardian member:** an individual signer within a guardian committee.
- **Witness committee:** an external threshold-signing committee used on the sealing path or in *NestedGuardian*.
- **Equal-authority observer:** a validator sampled to audit a slot under *Asymptote*.
- **Registry:** the on-chain source of truth for manifests, policies, epochs, witness sets, and verifier descriptors.
- **Transparency log:** an append-only checkpointed evidence log for guardian and witness statements.
- **External verifier or gateway:** a party that checks *SealObjects* and enforces replay-safe release.

5.2 Bulletin-board time and slot model

Consensus proceeds in heights and views. A slot is identified by $(height, view)$. Epoch-scoped policy and committee state bind the admissible evidence for that slot. The family should be read under a model-delta publication and recoverability discipline rather than a pure point-to-point timely receipt discipline: safety depends on whether the closed boundary fixes a unique public evidence surface, while liveness depends on correct-party delivery and eventual visibility of admissible content-addressed artifacts after stabilization.

Slots are externally paced by a public cutoff object τ_h and a public randomness beacon R_h . Honest validators that observe the same closed public boundary derive the same ordering or sealing close-or-abort result. In the runtime that derivation is no longer keyed off cutoff closure alone: the bulletin plane exports a *canonical bulletin-close object* binding the bulletin commitment, the bulletin-availability certificate, and the canonical cutoff into one public close surface. In this instantiation, positive canonical-order publication is fail-closed at this boundary: a closed-slot order certificate is admitted only if the published bulletin entries deterministically extract the bulletin surface bound by the canonical bulletin-close object and the carried bulletin-availability certificate. Universal timely receipt is not required.

Let

$$\begin{aligned} h &= \text{slot height}, \quad v = \text{slot view}, \\ B_h &= \text{bulletin surface admitted before cutoff } \tau_h, \\ R_h &= \text{public randomness beacon for slot } h, \\ E_h &= \{tx \mid \text{Included}(tx, B_h) \wedge \text{Eligible}(tx, \text{root}_{h-1})\}, \\ O_h &= \text{CanonicalOrder}(R_h, E_h), \\ \text{root}_h &= \text{Execute}(\text{root}_{h-1}, O_h). \end{aligned}$$

5.3 Classical lower bound and AFT model delta

Let M_{DLS} denote the standard authenticated partially synchronous message-passing model used by the classical DLS / PBFT lower-bound argument. Its adversary may delay messages so that two honest groups cannot distinguish “the other group is slow” from “the other group is faulty” for the duration required by the indistinguishability construction.

Let M_{AFT} denote the AFT model delta. It keeps fixed authenticated authorities, deterministic local verification, and partial synchrony among correct parties, but it removes one adversary power: after stabilization, an admissible content-addressed protocol artifact repeatedly published by a correct party cannot be indefinitely suppressed from correct parties. Payload delivery is realized by direct authenticated dissemination from the publisher to validators or deterministic custodians; equivocation, omission, and retrievability detection are realized by compact echo objects rather than by payload relay.

Adversary-execution trace. Run the classical indistinguishability adversary against AFT:

1. The adversary may schedule delays, split honest validators into views, and make Byzantine validators equivocate or omit.
2. The adversary may hide a Byzantine sender’s conflicting statements until an honest validator publishes a protocol-admissible commitment, receipt, frontier, challenge, or reconstruction / abort artifact.
3. The first illegal step is indefinite suppression of that published admissible artifact from another correct validator after stabilization. In M_{DLS} , that suppression power is available to the adversary long enough to drive the lower-bound execution. In M_{AFT} , the artifact eventually becomes visible and content-hash fetchable.

That stopped step is the model delta. The rest of AFT is protocol design inside M_{AFT} : compact frontiers bind the claimed surface, contradiction objects kill conflicts, retrievability objects reconstruct or abort the full surface, and canonical collapse gates durability.

Lemma 1 (Direct dissemination plus compressed echo realizes the delta). *Assume authenticated point-to-point channels among correct parties that eventually deliver after stabilization. An honest publisher sends its artifact or erasure-coded shards directly to the validators or deterministic custodians named by the slot policy. No non-sender validator must relay the payload for honest-artifact delivery. Validators then publish compact commitments, availability receipts, publication frontiers, observer transcripts, challenges, and reconstruction / abort objects that expose what they received or what the public surface lacks. The payload path discharges honest delivery; the echo path discharges equivocation, omission, and retrievability detection. The classical adversary’s indefinite-suppression step is therefore unavailable without introducing a trusted relay, coordinator, TEE, bootstrap oracle, or external recovery service.*

5.4 Core assumptions

The family-level safety claims rely on the following explicit assumptions:

- **Objective publication:** admitted bulletin entries, transcripts, and challenges have stable hashes and objective inclusion paths.
- **Authenticated direct dissemination:** after stabilization, authenticated channels between correct parties eventually deliver the publisher’s direct artifact or shard sends to the validators or deterministic custodians named by the slot policy.
- **Compressed echo visibility:** validators publish compact commitments, receipts, frontiers, transcripts, challenges, and reconstruction / abort artifacts that make equivocation, omission, stale lineage, and retrievability failures objectively visible without echoing full payloads.
- **Verifiable bulletin availability:** the protocol exports a bulletin-availability object, compact publication-frontier metadata that binds the same slot surface, and a canonical bulletin-close object for each closed ordering slot; the remaining assumption is that direct dissemination and the compressed echo layer satisfy the retrieval semantics claimed by those objects. The live runtime also requires successful closed-slot extraction before positive canonical-order admission.
- **Compact frontier consistency:** any carried `PublicationFrontier` must bind the same-slot bulletin commitment, ordered surface, and compact availability receipt, and same-slot conflicts or stale predecessor links must admit short objective contradiction objects. These frontier objects summarize the slot surface; they do not reconstruct it.
- **Objective validity only:** theorem-critical rejection or acceptance does not depend on private local facts.
- **Public derivability over universal receipt:** honest validators that observe the same closed public boundary derive the same close-or-abort result; universal timely receipt is not required.
- **Independent echo path:** suppressing negative evidence requires capture of both the payload-delivery path and the compact publication / registry echo surface.
- **Sealed-effect gating:** irreversible effects are released only after surviving the canonical close-or-abort surface.
- **Fixed epoch identities:** the validator set is epoch-scoped and stable while the accountable rules of that epoch are enforced.

Here and throughout, the AFT permissioned model means fixed authenticated authorities, deterministic local verification, authenticated point-to-point channels among correct parties, public hash-addressed protocol objects, epoch-scoped identity state, and eventual visibility / persistence / hash-fetchability of admissible protocol artifacts after stabilization, with no trusted relay, privileged coordinator, TEE, theorem-side bootstrap oracle, or hidden external recovery service. Fixed authenticated membership is part of the theorem statement, not a missing crutch. The hidden-helper exclusions are relay, coordinator, TEE, bootstrap oracle, and external recovery service — not epoch-scoped authenticated identity itself.

5.5 Evidence objects

The family-level evidence objects are:

- validator quorum certificates over block proposals,
- guardian quorum certificates over slot decisions,
- witness certificates over guardianized slots,
- canonical-order certificates over bulletin surfaces,
- omission proofs over ordered-set incompleteness,
- observer transcripts, observer challenges, and canonical close-or-abort objects over sealed finality,
- sealed-effect proof objects with replay-safe nullifiers.

All honest nodes are required to derive the same acceptance result from the same admissible evidence set.

5.6 Accountable publication rules

Objective AFT faults are not merely audit artifacts. In the runtime they are first-class accountable protocol objects:

- `OmissionProof` names the validator accountable for the dominated positive order object.
- `AsymptoteObserverChallenge` determines accountability by kind: `MissingTranscript` and `ConflictingTranscript` blame the assigned observer; `TranscriptMismatch`, `VetoTranscriptPresent`, and `InvalidCanonicalClose` blame the producer / positive-close path.
- Valid objective fault evidence is replay-deduplicated on chain.
- Policy-controlled membership updates are aftermath only: the same evidence may trigger best-effort immediate quarantine and stage next-epoch eviction, but the decisive negative object remains valid even when those membership updates are disabled, delayed, or skipped.

These rules do not change the hot path. They strengthen the adversary model: arbitrary behavior is tolerated only insofar as it remains publicly revealable and penalty-bearing.

6 Public State Continuity with Extractable Obstructions

AFT’s deeper theorem target is not a new round gadget. It is a new source of uniqueness. Classical Byzantine agreement derives uniqueness from overlap of dense positive quorums. AFT instead seeks a rigidity theorem over public slot objects: once the slot boundary is closed, the space of admissible public continuations should collapse to one canonical object or one canonical abort.

6.1 Boundary-conditioned execution language

For a closed slot boundary define

$$\partial_h := (\text{root}_{h-1}, \beta_h, \tau_h, R_h, p_h),$$

where β_h is the canonical bulletin-close object binding the bulletin commitment, bulletin-availability certificate, and canonical cutoff whose recoverable surface is B_h ; τ_h is the canonical cutoff object; R_h is the slot randomness beacon; and p_h is the policy descriptor governing admissible checks for slot h .

Let a *public execution object* for slot h be a normalized public codeword

$$X_h := (B_h, E_h, O_h, \text{root}_h, \sigma_h),$$

where E_h is the eligible subset, O_h is the canonical ordered object, root_h is the resulting state commitment, and σ_h is the slot-local sealing surface needed to determine whether the slot closes or aborts. The important point is that X_h is not a distributed transcript. It is the public quotient of the slot after irrelevant schedule variation is factored out.

Define the boundary-conditioned language

$$\mathcal{L}(\partial_h) := \{X \mid \exists \pi \text{ Accept}(\partial_h, X, \pi) = 1\}.$$

Here π is a succinct admissibility witness. For canonical ordering, π is the proof-carrying order certificate witness. For deterministic observer sealing, π is the witness that the canonical transcript surface, challenge surface, and close-or-abort object satisfy the slot policy.

6.2 Obstruction extractability

The companion rejection relation is

$$\text{Reject}(\partial_h, Y, \omega) = 1,$$

where Y is a fully specified candidate public execution object and ω is a short public objective contradiction witness.

AFT seeks *obstruction-complete local testability*: for a fixed closed boundary, every fully specified candidate lies in exactly one of two states,

$$\left(\exists \pi \text{ Accept}(\partial_h, Y, \pi) = 1 \right) \oplus \left(\exists \omega \text{ Reject}(\partial_h, Y, \omega) = 1 \right).$$

This xor formulation is the software-only analogue of state continuity. It is stronger than merely having a valid proof system. It says every non-admissible candidate exposes a short public contradiction witness.

In the live repository, the obstruction basis is not abstract:

- ordering uses a first-class canonical abort taxonomy over the canonical bulletin surface: missing certificates, bulletin-surface reconstruction failures, bulletin-surface mismatches, invalid bulletin-close formation, omission-dominated candidates, certificate-height mismatches, randomness mismatches, ordered-transactions-root mismatches, resulting-state-root mismatches, invalid public-input bindings, invalid bulletin-availability bindings, and invalid proof bindings,
- deterministic observer sealing uses objective challenge kinds over the canonical transcript and challenge surface,
- the accountable-adversary layer turns those same objects into penalty-bearing public fault evidence.

6.3 Unique collapse object

The slot should not merely admit at most one positive object. It should admit at most one *collapse object*:

$$\text{Collapse}(\partial_h) \in \{\text{Close}(X_h), \text{Abort}(\Omega_h)\}.$$

Here $\text{Close}(X_h)$ denotes the unique admissible positive object when the obstruction surface is empty, while $\text{Abort}(\Omega_h)$ denotes the unique negative object induced by a non-empty valid obstruction surface. Negative evidence does not create a second truth. It forces the slot to land on its canonical abort object instead of a close object.

In the runtime this collapse object is not merely an abstract proof target. Validator finalization publishes a first-class `CanonicalCollapseObject` through `guardian_registry`, and later consensus verification cross-checks any published copy against the same proof-carried ordering and sealing surface when parent-state data is available. In the current recursive-continuity slice, `CanonicalCollapseObject` also carries a rolling `previous_canonical_collapse_commitment_hash`, and in the current clean-break runtime slice it also carries `continuity_accumulator_hash` together with `continuity_recursive_proof`, so the current slot’s collapse object is linked to the previously persisted or published collapse object rather than standing as an isolated per-slot fact. The newest proposal-surface slice pushes that same predecessor link into the signed header itself: `BlockHeader` now carries `previous_canonical_collapse_commitment_hash` in its signed preimage and also carries a typed `CanonicalCollapseExtensionCertificate`. That certificate now carries the predecessor commitment plus the predecessor recursive-proof hash, while the public predecessor `CanonicalCollapseObject` now carries only a single recursive proof step rather than a carried predecessor-proof tree. Each recursive proof step still binds a predecessor collapse commitment, predecessor commitment hash, predecessor payload hash, and previous proof hash; the rolling `continuity_accumulator_hash` remains in place as the companion accumulator over the same chain. The implementation surface distinguishes the reference `HashPcdV1` carrier from a backend slot `SuccinctSp1V1`, and the runtime exposes a dedicated continuity-verifier seam for those recursive public inputs via `ioi-api` and `zk-driver-succinct`. That seam is now wired into the live protocol: `GuardianMajority` invokes it while validating proposal and QC continuity for carried or anchored `SuccinctSp1V1` proof steps, and validator durable-state checks invoke the same backend before a persisted `CanonicalCollapseObject` is treated as authoritative. Proposal verification checks that the carried predecessor commitment hash matches the signed predecessor link, checks that the predecessor commitment’s resulting state root matches the signed parent-state root, and checks that the carried predecessor proof hash matches the anchored predecessor collapse object already persisted or published in protocol state. Leaders in `Asymptote` stall rather than propose when the required extension certificate or anchored predecessor collapse object is unavailable, proposal

verification rechecks that implied predecessor commitment against both the signed predecessor commitment link and the parent-state root, checks the carried predecessor proof hash against the anchored predecessor proof on the public collapse object, and QC progression only treats continuity-linked headers as progress-authoritative.

Theorem 1 (Public State Continuity with Extractable Obstructions). *Assume a closed boundary ∂_h , objective publication, a protocol-native retrievability plane, compact publication-frontier consistency, objective validity only, and proof soundness. Then the instantiated and normative theorem shape for AFT is:*

1. $\mathcal{L}(\partial_h)$ contains at most one admissible public execution object,
2. every fully specified candidate $Y \notin \mathcal{L}(\partial_h)$ admits a short public objective obstruction witness ω such that $\text{Reject}(\partial_h, Y, \omega) = 1$,
3. therefore the slot admits at most one admissible collapse object, either the unique close object or the unique abort object.

This theorem is the precise form of the software-only continuity primitive that the instantiated AFT runtime realizes. The later ordering, collapse, recovery, restart, and sealed-effect sections are concrete theorem instances and composition layers of this same continuity statement, not separate aspirations. It does not claim that the classical DLS / PBFT frontier disappeared inside the standard dense-vote model. The point is different: the uniqueness source has moved from quorum overlap into the public-evidence language itself. In the instantiated runtime, compact publication frontiers internalize same-slot and predecessor consistency inside the live protocol, while the protocol-native retrievability plane discharges closed-bulletin extraction-or-abort on the cold path.

6.4 Lemma program

The clean proof path is a filtration rather than a single opaque argument:

$$\mathcal{L}_0(\partial_h) \supseteq \mathcal{L}_1(\partial_h) \supseteq \cdots \supseteq \mathcal{L}_k(\partial_h),$$

where each refinement adds one structural constraint and each failed refinement admits a short witness.

The repository's theorem program is:

1. **Boundary Fixing Lemma.** The closed boundary fixes the admissible publication domain.
2. **Bulletin Close Lemma.** The canonical bulletin-close object fixes the unique recoverable bulletin surface for the slot.
3. **Eligibility Determinism Lemma.** The predecessor commitment and boundary fix the eligible subset.
4. **Order Determinism Lemma.** The boundary and eligible subset fix the canonical ordered object.
5. **Transition Determinism Lemma.** The ordered object fixes the next state commitment.
6. **Collapse Exclusivity Lemma.** A slot cannot admit both a canonical close object and a canonical abort object.

7. **Obstruction Soundness Lemma.** Every valid omission proof or observer challenge objectively rejects its target candidate.
8. **Obstruction Completeness Lemma.** Every invalid fully specified candidate contains at least one minimal public obstruction witness.
9. **Extractor Sufficiency Lemma.** Given the canonical bulletin-close object, any honest validator can run the protocol-defined bulletin extractor to reconstruct the same bulletin surface; in this instantiation, admission of the positive ordering object already requires this extraction to succeed over the published closed-slot surface. One honest validator remains sufficient to surface the resulting positive or negative object under the instantiated runtime model.

One honest validator is surfacing-sufficient but never trust-bearing: correctness does not attach to the revealer’s identity, because any honest validator observing the same closed boundary derives the same decisive object.

The theorem surfaces of this paper are instances of that program:

- **CanonicalOrdering** instantiates unique admissibility and extractable obstruction through proof-carrying order certificates plus omission dominance.
- **Asymptote** instantiates unique collapse through canonical transcript/challenge surfaces plus close-or-abort exclusivity.

7 The AFT Deterministic Base Engine

AFT modes share a deterministic core engine for proposal flow, QC formation, locking, and view progression.

7.1 Deterministic leadership and view progression

For a given height and view, the implementation selects the proposer by deterministic round-robin over the active validator set:

$$\text{leader}(h, v) = V_h[(h - 1 + v) \bmod |V_h|].$$

Here V_h is the active validator set for height h after operational quarantine filters are applied.

View 0 proposals do not carry a timeout certificate. Any non-zero view proposal must carry a valid timeout certificate for the previous unsuccessful view. The pacemaker advances views monotonically and resets timers on observed progress.

7.2 Validator quorum certificates

In **GuardianMajority**, **Asymptote**, and **NestedGuardian**, validator quorum is a strict simple majority of active validator weight, not a classical $2/3 + 1$ quorum:

$$\text{QCQuorum}(V_h) > \frac{1}{2} \text{Weight}(V_h).$$

This is safe only because the validator quorum is *not* the sole authority object. A proposal must also satisfy guardian-layer admissibility.

Timeout certificates use the same simple-majority rule over view-change votes.

7.3 Locking, 2-chain commit, and divergence guard

Validators maintain a lock on the highest safe parent QC they have observed. A validator votes only when the proposal extends the lock or is in a strictly higher view than the lock.

Base commitment uses a 2-chain rule:

- let QC_{parent} certify parent block P ,
- let QC_{child} certify child block C ,
- if C directly extends P and the views are consecutive, then P becomes a pending commit.

The implementation delays final durability by a guard window Δ_{guard} before a pending commit is released. The purpose of the guard window is operational, not purely algebraic: it gives divergence evidence or panic signals time to propagate before a conflicting branch is treated as durable.

This produces a fail-closed operational behavior:

- sparse chain progress happens on the hot path,
- conflicting evidence is surfaced before irreversible consequences,
- divergence proofs quarantine the affected node rather than silently allowing fork continuation.

8 GuardianMajority: Production Base Finality

GuardianMajority is the production fault model for AFT base finality. Validator votes alone never make a proposal admissible. Every valid proposal must also carry a guardian committee certificate that verifies against current registered committee state.

8.1 Guardian certificate object

The guardian certificate binds:

- the guardian committee manifest hash,
- the active epoch,
- the canonical decision hash for the slot payload,
- a monotonic guardian counter,
- a guardian trace root,
- a measurement root,
- the signer bitfield,
- the aggregated BLS signature,
- a transparency-log checkpoint,
- and, in **NestedGuardian**, an optional experimental witness certificate.

The guardian decision payload is slot-scoped. Honest guardians are assumed not to sign two different decisions for the same slot.

8.2 Verification rule

A guardianized proposal is admissible only if all of the following hold:

1. the referenced guardian committee manifest is registered on-chain for the active epoch,
2. the certificate manifest hash matches the registered manifest,
3. the certificate epoch matches the manifest epoch and current epoch,
4. the decision hash matches the slot’s canonical guardian decision payload,
5. the measurement root matches the decision payload,
6. the signer set is valid, unique, and meets the committee threshold,
7. the aggregated signature verifies against the manifest public keys,
8. the attached transparency-log checkpoint verifies against the anchored log descriptor and append-only proof.

The transparency log is an accountability layer. It does not replace threshold safety. It makes issued decisions, checkpoints, and later slashing evidence publicly checkable.

8.3 Committee threshold model

Let a guardian committee have threshold t and size n . Conflicting slot certificates require enough equivocators to cover the minimum quorum intersection:

$$f < 2t - n.$$

Production committees therefore use strict majority thresholds:

$$t = \left\lfloor \frac{n}{2} \right\rfloor + 1.$$

This guarantees pairwise quorum intersection. It also exposes an important operational nuance:

- odd-sized simple-majority committees tolerate zero equivocating guardian members at the committee layer,
- even-sized majority committees tolerate one equivocator before threshold intersection is exhausted.

Production safety therefore relies on both threshold intersection and stronger guardian non-equivocation guarantees.

8.4 Safety statement

Proposition 1 (GuardianMajority Safety). *No two conflicting blocks can both finalize for the same (height, view) if guardian committee manifests are current, committee thresholds are majority thresholds, honest guardians do not equivocate for the same slot, transparency-log checkpoints and registry state are current enough to verify admissibility, and validators finalize only guardian-admissible blocks.*

This is not a classical $3f + 1$ theorem. It is a composed threshold theorem over validators, guardians, registry state, and shared evidence. That statement is local to the guardian-backed base-finality layer. It does not delimit the paper’s promoted theorem surface. The promoted durable-agreement claim is carried by the later proof-carrying close-or-abort and canonical- collapse architecture, which is precisely why the classical $3f + 1$ threshold is not load-bearing there.

8.5 Liveness model

Liveness is secondary to safety. Progress requires:

- enough active validators to form the simple-majority validator QC,
- enough guardian members to satisfy the committee threshold,
- current registry state for the epoch,
- current enough checkpoint and log availability.

If these degrade, the protocol prefers stalling over accepting unsafe evidence.

9 Canonical Ordering

`CanonicalOrdering` is the proof-carrying ordering path that gives AFT its all-but-one ($n - 1$ of n) equal-authority ordering consensus claim in the AFT model delta.

9.1 Objective

For each slot h , define a unique canonical ordered set O_h and a certificate $OCert_h$ such that:

- every honest verifier can check $OCert_h$ cheaply,
- any omission is objectively provable,
- the ordered set is recoverable from public data,
- the live hot path carries a compact publication frontier that binds the same-slot bulletin / ordering / availability surface and predecessor continuity,
- conflicting valid certificates for the same slot are impossible,
- arbitrary behavior by almost all validators cannot create a conflicting valid ordering outcome because the closed bulletin boundary admits one deterministically derivable close-or-abort result.

9.2 Canonical objects

For slot h ,

$$\begin{aligned} B_h &= \text{bulletin surface admitted before } \tau_h, \\ R_h &= \text{public randomness beacon for slot } h, \\ E_h &= \{tx \mid \text{Included}(tx, B_h) \wedge \text{Eligible}(tx, \text{root}_{h-1})\}, \\ O_h &= \text{CanonicalOrder}(R_h, E_h), \\ \text{root}_h &= \text{Execute}(\text{root}_{h-1}, O_h). \end{aligned}$$

The abstract certificate is

$$OCert_h = (h, \text{root}_{h-1}, \text{cutoff}_h, \text{bulletin_commitment}_h, \text{ordered_commitment}_h, \text{root}_h, \text{witness}_h).$$

9.3 Bulletin, cutoff, and admissibility assumptions

The ordering theorem is conditional on the following named assumptions.

A1. Objective publication. Each admitted transaction or batch in B_h has a stable content hash, a verifiable inclusion path into the bulletin commitment, and an objective publication time or publication position relative to the slot cutoff.

A2. Endogenous retrievability. The bulletin contents committed by $\text{bulletin_commitment}_h$ are reconstructed or deterministically aborted from protocol-native retrievability objects rooted in the canonical bulletin close for slot h . Compact publication-frontier summaries and availability certificates bind that claim on the hot path, while retrievability profiles, shard manifests, custody assignments, custody receipts, custody responses, and objective challenge objects close the extraction-or-abort loop on the cold path. Without that plane, uniqueness may still hold while timely materialization of the same close-or-abort surface can fail.

A3. Append-only or equivocation-evident commitment. The bulletin commitment and any carried publication frontier for slot h must be append-only or otherwise make same-slot equivocation or stale predecessor linkage objectively detectable at the slot boundary.

A4. Eligibility determinism. $\text{Eligible}(tx, \text{root}_{h-1})$ is deterministic and locally checkable from the committed predecessor root and the transaction object. Honest validators therefore derive the same eligible set E_h from the same public inputs.

A5. Proof soundness. $\text{VerifyProof}(\text{witness}_h) = \text{true}$ implies that the witness obligations for cutoff binding, bulletin binding, ordering, state transition, retrievability commitments, and omission commitments actually hold.

The cutoff τ_h is not a private local clock read. It is a protocol-bound object. In the instantiated runtime, the cutoff is bound directly into the published bulletin commitment and into the canonical public-input set. Eligibility is deterministic with respect to the predecessor state root and the transaction object. This prevents local reinterpretation of the eligible set.

9.4 Current runtime instantiation

The runtime surfaces the abstract objects above as the following concrete objects:

Runtime object	Current fields
BulletinCommitment	(height, cutoff_timestamp_ms, bulletin_root, entry_count)
BulletinSurfaceEntry	(height, tx_hash)
BulletinAvailability Certificate	(height, bulletin_commitment_hash, recoverability_root) (height, bulletin_commitment_hash, bulletin_availability_certificate_hash, recoverability_root, shard_count, recovery_threshold, custody_threshold)
BulletinRetrievabilityProfile	(height, bulletin_commitment_hash, bulletin_availability_certificate_hash, bulletin_retrievability_profile_hash, recoverability_root, entry_count, shard_count, recovery_threshold, shard_commitment_root)
BulletinShardManifest	(height, bulletin_retrievability_profile_hash, bulletin_shard_manifest_hash, validator_set_commitment_hash, assignments)
BulletinCustodyAssignment	(height, bulletin_commitment_hash, bulletin_availability_certificate_hash, bulletin_retrievability_profile_hash, bulletin_shard_manifest_hash, custodian_count, custody_threshold, custody_root)
BulletinCustodyReceipt	(height, bulletin_commitment_hash, bulletin_availability_certificate_hash, bulletin_retrievability_profile_hash, bulletin_shard_manifest_hash, bulletin_custody_assignment_hash, bulletin_custody_receipt_hash, served_shards)
BulletinCustodyResponse	(height, kind, bulletin_commitment_hash, bulletin_availability_certificate_hash, bulletin_retrievability_profile_hash, bulletin_shard_manifest_hash, bulletin_custody_assignment_hash, bulletin_custody_receipt_hash, bulletin_custody_response_hash, details)
BulletinRetrievabilityChallenge	(height, bulletin_commitment_hash, bulletin_availability_certificate_hash, bulletin_retrievability_profile_hash, bulletin_shard_manifest_hash, bulletin_custody_assignment_hash, bulletin_custody_receipt_hash, bulletin_custody_response_hash, canonical_bulletin_close_hash, canonical_order_certificate_hash, reconstructed_entry_count, reconstructed_bulletin_root)
BulletinReconstructionCertificate	(height, kind, bulletin_commitment_hash, bulletin_availability_certificate_hash, bulletin_retrievability_profile_hash, bulletin_shard_manifest_hash, bulletin_custody_receipt_hash, bulletin_retrievability_challenge_hash, canonical_order_abort_hash, details)
BulletinReconstructionAbort	(height, bulletin_commitment_hash, ordered_transactions_root_hash, resulting_state_root_hash, receipt_root)
PublicationAvailabilityReceipt	(height, view, counter, parent_frontier_hash, bulletin_commitment_hash, ordered_transactions_root_hash, availability_receipt_hash)
PublicationFrontier	(height, kind, candidate_frontier, reference_frontier)
PublicationFrontierContradiction	(height, cutoff_timestamp_ms, bulletin_commitment_hash, bulletin_availability_certificate_hash, bulletin_retrievability_profile_hash, bulletin_shard_manifest_hash, bulletin_custody_receipt_hash, entry_count)
CanonicalBulletin Close	(height, parent_state_root_hash, bulletin_commitment_hash, randomness_beacon, ordered_transactions_root_hash, resulting_state_root_hash, cutoff_timestamp_ms)
CanonicalOrder PublicInputs	concrete variants HashBindingV1 and CommittedSurfaceV1
CanonicalOrderProofSystem	(bulletin_availability_certificate_hash, omission_commitment_root, boundary_omission_justification_root)
CommittedSurface	(height, bulletin_commitment, bulletin_availability_certificate, randomness_beacon, ordered_transactions_root_hash, resulting_state_root_hash, proof, omission_proofs, boundary_tx_hashes, boundary_omission_justifications)
CanonicalOrderProof	(bulletin_commitment, bulletin_entries, bulletin_availability_certificate, bulletin_retrievability_profile, bulletin_shard_manifest, bulletin_custody_receipt, canonical_order_certificate)
CanonicalOrder Certificate	(height, reason, details, bulletin_commitment_hash, bulletin_availability_certificate_hash, bulletin_close_hash, canonical_order_certificate_hash)
CanonicalOrder PublicationBundle	
CanonicalOrder Abort	

HashBindingV1 is the reference verifier family: its proof bytes are a canonical hash binding over the public-input hash. **CommittedSurfaceV1** is the live proof family: its proof payload contains the recoverability root and the omission-commitment root for the committed surface. In the current runtime, positive ordering artifacts are published through the atomic **CanonicalOrderPublicationBundle**, carrying the bound retrievability profile / manifest / custody-receipt objects, while the registry deterministically materializes the coupled custody-assignment and custody-response objects over the same slot surface before admitting the positive lane. Positive committed blocks also carry a compact signed **PublicationFrontier**: it binds the same-slot bulletin commitment, ordered-transactions root, and compact publication-availability receipt, and it hash-links that summary to the previous slot's frontier. Short **PublicationFrontierContradiction** objects make same-slot conflict or stale predecessor linkage objectively rejectable on the hot path. Those frontier objects internalize compact publication consistency, but they do not replace the protocol-native retrievability plane that reconstructs or aborts the full bulletin surface. Validators also run a block-local derivation pass over the committed block boundary: they derive either a **CanonicalOrderExecutionObject** or a **CanonicalOrderAbort**, and publish the abort artifact when the proof-carried positive object cannot be derived. When the endogenous retrievability lane succeeds, the registry materializes a first-class **BulletinReconstructionCertificate** bound to the canonical close, canonical-order certificate, and the same retrievability objects. When endogenous retrievability evidence dominates that same surface, the registry also materializes a specialized **BulletinReconstructionAbort** object bound to the challenge and the paired **CanonicalOrderAbort**, so reconstruction-impossible failure is named as a first-class protocol object rather than only as a generic ordering abort.

9.5 Acceptance rule

Every validator applies the same deterministic rule:

$$\begin{aligned} \text{Accept}(OCert_h) \iff & \text{VerifyProof}(witness_h) \\ & \wedge \text{VerifyCutoff}(cutoff_h) \\ & \wedge \text{PredecessorAccepted}(root_{h-1}) \\ & \wedge \neg \exists tx : \text{ValidOmission}(h, tx). \end{aligned}$$

In the implementation this specializes to:

- the certificate height and bulletin height must match the block height,
- the randomness beacon must match the slot schedule,
- the bulletin commitment must match the published bulletin when that bulletin is available from anchored state,
- any carried **PublicationFrontier** must match the same-slot bulletin commitment, ordered-transactions root, and publication-availability receipt hash,
- when a predecessor frontier exists, the carried **PublicationFrontier** must extend it by counter and parent hash,
- no published **PublicationFrontierContradiction** may already dominate the slot,

- positive canonical-order admission in the runtime additionally requires successful closed-slot extraction over the published bulletin entries, the carried bulletin-availability certificate, and the canonical bulletin-close object,
- the ordered-transactions root and resulting state root must match the block header,
- the proof must match the canonical public inputs,
- the recoverability root must match the certificate commitments,
- the omission-commitment root must match the omission-proof set.

If the omission-proof set is non-empty, the candidate certificate is rejected immediately.

No dense positive vote on the order is required once a valid $OCert_h$ is surfaced. The current verifier entry point for this rule is `verify_canonical_order_certificate`.

9.6 Omission dominance

An omission is objective when a transaction:

- was included in B_h before τ_h ,
- was eligible under $root_{h-1}$,
- and is absent from the committed ordered set O_h .

Define the omission proof

$$\Omega(h, tx) = \left(\begin{array}{l} inclusion_proof(tx, bulletin_h), \\ cutoff_admissibility_proof(tx, cutoff_h), \\ eligibility_proof(tx, root_{h-1}), \\ nonmembership_proof(tx, O_h) \end{array} \right).$$

Its semantics are:

$$\text{Valid}(\Omega(h, tx)) \Rightarrow \text{Reject}(OCert_h).$$

This is the central AFT ordering move. A candidate does not need to win a dense round of positive endorsements if any incomplete view can be killed objectively.

Omission dominance is a correctness rule, not a punishment rule. A valid omission proof kills the candidate immediately whether or not slashing, quarantine, or next-epoch removal ever fires.

9.7 Uniqueness, publication frontiers, and recoverability

Theorem 2 (Uniqueness). *If $\text{ValidOrderCert}(h, C_1)$ and $\text{ValidOrderCert}(h, C_2)$, then C_1 and C_2 commit the same canonical ordered set O_h .*

The reason is structural:

- the predecessor root is fixed,
- the cutoff object is canonical,
- the bulletin commitment is canonical,

- eligibility is deterministic,
- the ordering function is deterministic,
- proof soundness forbids a witness for a different ordered set.

The live publication-frontier slice sits between uniqueness and recoverability. A frontier gives a compact signed summary of the claimed bulletin / ordering surface and its predecessor link, so same-slot conflict or stale lineage is objectively rejectable without reconstructing the full bulletin on the hot path. But the frontier only binds hashes and receipt roots; it is not itself the recoverable bulletin surface.

Theorem 3 (Endogenous Retrievability). *If ValidOrderCert(h, C) and the canonical bulletin close plus protocol-native retrievability plane for slot h are present and unchallenged, then every honest verifier can reconstruct the same ordered set O_h from protocol objects alone: the canonical close, bound frontier / availability commitments, retrievability profile, shard manifest, custody receipt, positive reconstruction certificate, and published bulletin entries.*

Endogenous retrievability matters because the accepted order is not merely unique. The compact frontier tells validators which surface is being claimed; the retrievability plane tells them that the full surface can actually be reconstructed or objectively aborted from protocol evidence.

The hot path and the cold path do not carry different truths. The frontier binds the claimed surface compactly; the retrievability plane reconstructs or objectively aborts that same surface; both terminate on the same canonical close-or-abort object.

9.8 All-but-one ($n - 1$ of n) equal-authority ordering consensus

AFT’s ordering claim is a proof-carrying consensus theorem whose decisive object is revealed rather than manufactured by dense positive yes-votes. Revelation is not a weaker post hoc reporting layer here; it is the equal- authority surfacing mechanism for the uniquely valid public execution object already fixed by the closed public boundary.

Theorem 4 (All-But-One ($n - 1$ of n) Equal-Authority Ordering Consensus in the AFT Model Delta). *Assume, in the AFT model delta (the assumed common publication boundary):*

1. *the closed bulletin surface B_h bound by the carried publication frontier and bulletin commitment is governed by a canonical bulletin close and protocol-native retrievability plane that deterministically yields extraction or abort,*
2. *the cutoff object for slot h is canonical,*
3. *bulletin commitments, compact publication-frontier bindings, and omission proofs are objectively verifiable,*
4. *same-slot frontier conflicts and stale predecessor links admit short objective contradiction objects,*
5. *the proof system is sound.*

Then even if all but one validator ($n - 1$ of n) behaves arbitrarily, they cannot create a conflicting valid canonical-order outcome for slot h . More strongly, every honest validator that observes the same closed ordering boundary derives the same positive order object or omission-dominated abort.

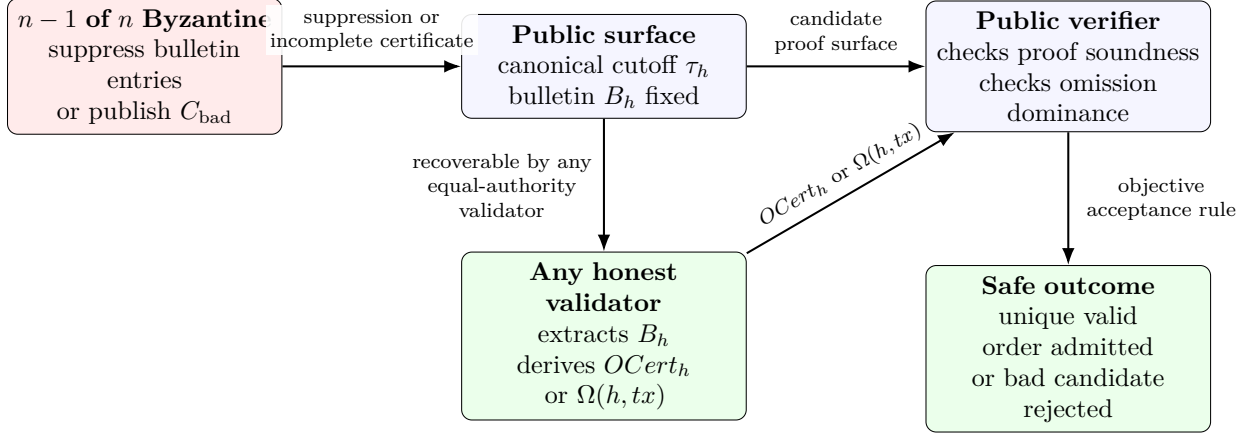


Figure 3: Omission-dominance attack and deterministic extraction. Even if a suppressing super-majority tries to hide admissible bulletin entries or promote an incomplete candidate certificate, any honest validator that reconstructs the same public surface derives the same valid $OCert_h$ or publishes a valid omission proof $\Omega(h, tx)$ that kills the bad candidate.

Compact publication frontiers therefore internalize cheap same-slot and predecessor consistency checks on the live path, while recoverability remains the stronger assumption needed to turn that compact claim into a publicly reconstructable ordered set.

The strongest shorthand, in the AFT model delta, is:

$n - 1$ of n validators may behave arbitrarily,

because the closed ordering boundary admits one deterministically

derivable close-or-abort outcome.

This is equal-authority because any validator may reveal the winning certificate. It tolerates all but one Byzantine validator ($n - 1$ of n) because correctness does not depend on a dense majority of positive votes. This section isolates the ordering lane, not the theorem ceiling. Once composed with endogenous retrievability, canonical collapse, recovery and restart continuity, and collapse-bound sealed-effect gating, the promoted AFT theorem surface is exactly the paper’s relay-free deterministic all-but-one ($n - 1$ of n) Byzantine safety claim in the AFT model delta.

With the accountable publication rules of Section 5, invalid positive ordering attempts are not only rejectable; they are also penalty-bearing and next-epoch removing.

10 Asymptote: Sealed Finality and Irreversible Effects

Asymptote is the two-tier settlement mode in the AFT family. It preserves fast **BaseFinal** chain progression while requiring stronger evidence before irreversible effects are released.

10.1 Finality tiers and collapse states

The finality tiers are:

- **BaseFinal**: validator QC plus guardian certificate.
- **SealedFinal**: **BaseFinal** plus sufficient sealing evidence.

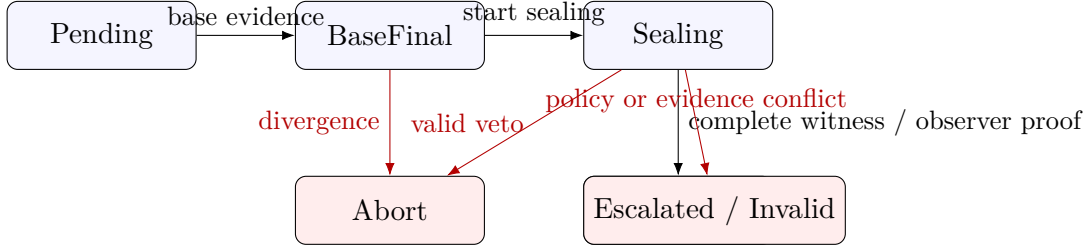


Figure 4: The deterministic collapse surface for **Asymptote**.

The slot collapse states are:

- Pending
- BaseFinal
- Sealing
- Abort
- SealedFinal
- Escalated
- Invalid

All honest nodes must derive the same collapse state from the same admissible evidence set.

10.2 SealedFinalityProof

The sealing plane gathers a proof object that binds:

- the epoch,
- the achieved finality tier,
- the collapse state,
- the guardian manifest hash,
- the guardian decision hash,
- the guardian counter,
- the guardian trace root,
- the guardian measurement root,
- the policy hash,
- witness certificates, when the witness path is used,
- observer transcripts, transcript commitment, observer challenges, challenge commitment, and exactly one canonical observer outcome object when the observer path is used,

- divergence signals gathered during sealing.

This object is block-scoped and guardian-bound. A `SealedFinalityProof` that is not bound to the slot's guardian evidence is invalid.

10.3 Witness-backed sealing

The witness path assigns one witness committee per required certification stratum.

Witness assignment is deterministic over:

- the epoch seed,
- the producer account id,
- the slot (*height*, *view*),
- the reassignment depth,
- the witness stratum id,
- the witness manifest hash.

Sealed finality is reached on the witness path when:

- `BaseFinal` holds,
- each required witness stratum contributes a valid assigned witness certificate,
- each witness certificate verifies against the registered witness manifest,
- required checkpoints and append-only proofs are current enough under policy.

Witness omission, stale registry participation, checkpoint inconsistency, or conflicting witness certificates produce explicit evidence rather than silent best-effort behavior.

10.4 Equal-authority observer sealing

The normative observer path is `CanonicalChallengeV1`. It treats sampled validators as deterministic duty assignees whose outputs must become public, recoverable protocol objects.

Observer assignment is deterministic over:

- the epoch seed,
- the producer account id,
- the slot (*height*, *view*),
- the observation round,
- the observer account id.

The producer is excluded from its own observer pool. The runtime may also filter assignments through a correlation budget over provider, region, host class, and key-authority class.

Each assigned observer publishes one of two things:

- a guardian-backed `AsymptoteObserverTranscript`, or
- an objective `AsymptoteObserverChallenge`.

A transcript binds the canonical slot surface:

- the epoch and assignment,
- the candidate block hash,
- the guardian manifest, decision hash, counter, trace hash, and measurement root,
- the guardian checkpoint root,
- the observer verdict and veto kind,
- the observer evidence hash.

The admissible challenge basis is public-evidence-only:

- `MissingTranscript`,
- `TranscriptMismatch`,
- `VetoTranscriptPresent`,
- `ConflictingTranscript`,
- `InvalidCanonicalClose`.

Each challenge carries its own public evidence object: an assignment, an offending observation request, an offending transcript, or an offending canonical close. The challenge `evidence_hash` must bind exactly that carried evidence.

The slot then carries three canonical bindings:

- a transcript commitment over the transcript surface,
- a challenge commitment over the challenge surface,
- exactly one canonical outcome object: `AsymptoteObserverCanonicalClose` or `AsymptoteObserverCanonicalA`

The collapse rule is close-or-abort and challenge-dominant:

- **SealedFinal** if **BaseFinal** holds, the deterministic assignment set is fully covered by valid canonical `Ok` transcripts, the transcript and challenge commitments verify, the challenge surface is empty, and the canonical close object is valid.
- **Abort** if the transcript and challenge commitments verify, the challenge surface is non-empty, and the canonical abort object is valid.
- **Pending** otherwise.

Challenge dominance is a theorem-level veto, not an operational alarm. A valid challenge collapses the slot to abort before any penalty, governance, or membership aftermath is considered.

This makes observer sealing structurally parallel to canonical ordering: public evidence, canonical commitments, a unique positive object, a unique negative object, and deterministic local close-or-abort derivation from the same public surface.

10.5 Divergence signals and escalation

The sealing plane may also emit divergence signals, including:

- conflicting guardian certificates,
- witness omission,
- stale registry use,
- stale checkpoints,
- transparency-log consistency failure.

These signals support **Escalated** and **Invalid** outcomes and give operators a fail-closed evidence trail.

10.6 Sealed effects and replay-safe release

Irreversible effects are gated by finality tier. The effect plane distinguishes between:

- **BaseFinal** effects, which are allowed on the fast path when policy permits,
- **SealedFinal** effects, which require a valid **SealedFinalityProof**.

For proof-enabled irreversible effects, **Asymptote** carries a self-contained **SealObject** whose current instantiation includes:

- an **EffectIntent**,
- an **EffectPublicInputs** object, including **canonical_collapse_hash**,
- an **EffectProofEnvelope**,
- a replay-safe nullifier.

In the runtime, **SealedFinal** secure-egress requests carry both the **SealedFinalityProof** and the slot's **CanonicalCollapseObject**. The guardian-side seal builder and downstream receipt verifier recompute the same **canonical_collapse_hash** from that collapse object together with the proof-carried observer surface, so a challenge-dominated or otherwise mismatched slot cannot authorize irreversible release.

The release rule is

$$\text{Release}(e) \iff \text{BaseFinal}(\text{slot}(e)) \wedge \text{SealedFinal}(\text{slot}(e)) \wedge \text{VerifySealObject}(e) \wedge \text{CollapseHash}(e) = \text{Hash}(\text{Collapse}(e))$$

This keeps the hot path sparse while making released consequences deterministic and replay-safe.

10.7 Deterministic observer theorem

The deterministic **Asymptote** observer model now proves the following kernel:

- base certificates are unique per slot,
- canonical observer close objects are unique per slot,
- canonical observer close and canonical observer abort cannot coexist for the same slot,
- a challenge-dominated slot cannot produce a valid sealed release.

Theorem 5 (Deterministic Equal-Authority Observer Sealing). *Assume sound deterministic observer assignment, signature-sound guardian-backed observer transcripts and challenges, canonical transcript and challenge commitments, append-only registry and log publication once published, and deterministic transcript/challenge derivation over the closed public observer surface. Then arbitrary behavior by the rest of the validator set cannot create a conflicting valid canonical observer close, cannot make canonical close and canonical abort coexist for the same slot, and cannot authorize a valid sealed irreversible effect release for a challenge-dominated slot.*

This theorem is the observer-lane analogue of canonical ordering’s public certificate theorem: public evidence, canonical commitments, negative-authority dominance, and deterministic local close-or-abort derivation from the same public observer surface.

Under the accountable publication rules of Section 5, the same observer challenges are also penalty-bearing objective faults. Unsafe positive close attempts therefore become not only rejectable, but accountable and epoch-removing.

10.8 Legacy sampled comparison

Before **CanonicalChallengeV1**, the observer lane was typically described through a sampled affirmative bound. Let:

- h be the honest fraction of equal-authority validators,
- A_s^j be the sampled observer committee for slot s and round j ,
- $k_j = |A_s^j|$,
- $p_{\text{beacon_bias}}$ be the probability of sufficient beacon bias,
- $p_{\text{veto_suppression}}$ be the probability that a valid veto surface is fully suppressed.

Then the old affirmative sampled lane admitted the analytical bound

$$P_{\text{unsafe}}(s) \leq p_{\text{beacon_bias}} \cdot p_{\text{veto_suppression}} \cdot \prod_j (1 - h)^{k_j}.$$

For constant committee size k across r rounds:

$$P_{\text{unsafe}}(s) \leq p_{\text{beacon_bias}} \cdot p_{\text{veto_suppression}} \cdot (1 - h)^{kr}.$$

That formula remains a useful historical comparison, but it is no longer the normative theorem for the observer lane. Under **CanonicalChallengeV1**, sampling determines duties, not safety. Safety comes from the canonical public transcript surface, the canonical public challenge surface, and the close-or-abort split that those surfaces induce.

The runtime publishes the canonical observer transcript, commitment, challenge, challenge commitment, and close-or-abort objects as ordinary **guardian_registry** transactions after the sealed header is formed. The same surface is carried inside the proof for immediate block verification.

11 The Common Boundary and the Seal Ring

The AFT-CB program restated the strong-tier settlement layer as a single construction: the *Common Boundary*. Where the Asymptote surface of the previous section describes the deployed observer-sealing runtime, this section describes the mechanized seal-ring architecture that the theorem corpus of Section 14 proves and that the runtime is converging onto. Status labels are exact and load-bearing: the kernels are mechanized (Appendix A), the membership types, membership simulator, registry wiring, hardened signer, and the runtime inversions below are landed in the repository, and the full multi-member ring runtime is future work behind an honestly labeled reference seam.

11.1 The Unanimous Boundary Close

One signer configuration C_v of n members forms one *Unanimous Boundary Close* per seal slot: an n -of- n certificate of individually verifiable final-ack shares over a domain-separated tuple binding the slot’s closed boundary root. Three disciplines carry the entire safety story:

- **Journal-guarded single final-ack (A3).** An honest member journals its chosen root for a slot in the same atomic step that lets the share exist; recovery replays bytes and never re-decides. The journal is the only thing standing between $n - 1$ Byzantine equivocators and a double seal — and by T1 it suffices.
- **n -of- n close.** Every current member’s share over the exact (slot, root) pair is required; no smaller quorum exists anywhere in the construction. The forced trade (L2) is accepted openly: one withholder freezes seal cadence, never the chain.
- **Validate-and-hold as the signature’s meaning (T3).** The final-ack *is* the custody commitment; availability standing derives from the close itself, and a close verifies with zero audit records.

11.2 Two-tier separation

The live tier (GuardianMajority) produces, orders, and commits blocks under its own weight-and-synchrony model; the seal ring closes boundaries above it. The only protocol edge from ring to live tier is the release gate for effects *typed* irreversible: block production consults no seal, and a ring stall delays irreversible-effect release and nothing else (unsealed-over-unsafe, T4a). The trade runs in both directions — the live tier’s fault bound never contaminates the ring’s weight-independent uniqueness, and the ring’s unanimity requirement never throttles chain progress.

11.3 The membership plane

Ring membership is event-driven and type-enforced (`crates/types/src/app/ring_membership.rs` plus the guardian registry’s service wiring):

- **Bonded registration and the public queue.** Registrants post a bond and enter a public activation queue with an event-depth delay (D_{act}), so admission cannot be timed adversarially and the honest set always has public warning of a capture attempt.
- **Event-driven versioned configurations.** A configuration lives until a close record names its successor; no calendar time exists anywhere in the plane.

- **The assurance-preserving handover is the only strong-ring transition.** It is constructible solely from signed old-ring-unanimous approvals plus acceptance from every new member. No type represents non-response, so a transition “from silence” has no expressible input — the proof-of-silence attack is unrepresentable rather than merely rejected (T5c’).
- **Typed re-genesis.** An anchored re-genesis is a lineage root, never continuity; the lineage query cannot answer “continuous across a root.”
- **Live-tier-only ejection.** Liveness faults eject a member from live-tier duty on live-tier weight; the ejection types are fenced from every seal and ring-configuration surface, so strong-ring standing exits only via handover or re-genesis.
- **Custody succession gates bond release.** A departing member’s bond releases only against a custody handover receipt carrying the successor’s acknowledgement — succession without reconstruction is refused.
- **Riders.** Seat assignment is deterministic sortition over a beacon abstraction across constituency diversity floors (the beacon is the reference ordering beacon until a vetted VDF lands — an honest label, not a security claim); watchtower countersignatures are accepted from anyone and gate nothing.

The plane is exercised by a membership simulator at $n \leq 8$ whose gates are the corpus’s scenario proofs: a withholder freezes cadence while the live tier produces; three-Byzantine-of-four equivocation yields zero conflicting seals; succession without reconstruction refuses bond release; the handover ceremony completes end to end with lineage preserved while a re-genesis severs it; sortition reproduces from the beacon and the seal outcome is byte-identical with and without watchtower records.

11.4 The hardened signer

The seal path’s signer (`crates/validator/.../guardian/seal_signer.rs`) implements the wire discipline T7 is proven against, plus the A8 everlasting-safety mechanism:

- **Attribution-preserving shares.** Every share carries its own per-seal public key and signature over a domain-separated tuple — individually verifiable, never an opaque aggregate — so forensic extraction names every double-signer from the shares alone.
- **Per-seal key evolution with verified immediate erasure.** One keypair per seal index derives from a hash-ratcheting seed; the spent seed is overwritten in the emitting call, the erasure is byte-verified against the persisted state, and a spent index is structurally unreachable. Compromise after seal i yields only future seeds.
- **Emitting is publishing.** Share emission returns the publication record in the same act, so withholding requires deliberately discarding what the signer already handed back — the honest-publication duty as mechanism rather than assumption.

11.5 Runtime inversions landed by the program

Four inversions moved the production ordering path from internally-consistent to boundary-honest, each with its defect class named and killed:

- **Boundary-before-block.** The committed-surface certificate builder *consumes* a previously closed boundary; the bulletin-from-block-transactions path is deleted. Certificates carry their boundary and typed omission justifications, verification is self-contained (the included set — boundary minus justified omissions — must reproduce the header’s transaction root), and the program’s baseline commit records the pre-inversion code verifying a censoring block as valid. The current closer is an honestly labeled reference-single-member seam that the ring plane replaces.
- **Custody-attested availability.** The self-attested availability-certificate constructor is deleted; what remains is a deterministic binding derivable by anyone, and availability *standing* derives only from a verified close. An additive audit lane exists for slashing evidence and is consulted by no verification path.
- **Immutable closes.** An abort supersedes only pre-seal state; an abort against a sealed close is a typed rejection whose attempt is itself recorded. Post-close evidence lands in an append-only resolution log (slashing evidence, forward remedies, descendant fences) — history is never rewritten, and the sealed object is byte-identical after any attempt.
- **Real-or-refuse succinct verification.** The succinct continuity lane verifies only through the real proving backend against a governed, test-pinned verifying key; a build without the backend refuses rather than simulating, and no mock constructor survives anywhere in the release tree (a CI gate holds the absence).

11.6 The assumption lattice at the type layer

The T6 lattice is implemented in the collapse types: every certificate profile carries a machine-readable assumption label, adding a profile without deciding its label is a compile error, and a collapse object’s effective guarantee is computed as the meet of its constituents — minimum rank, union of consumed assumptions, AND of the post-quantum bit. Evidence-only constituents (continuity bindings, typed roots) contribute assumptions and the pq bit but neither grant nor degrade rank, and a finality receipt is issuable only for a class the meet honestly supports.

11.7 Continuous conformance

The trace-conformance lane closes the loop between code and model: a reference driver executes boundary-ring steps under the kernel’s exact guards and emits a committed trace; a generator turns that trace into a TLC-checkable module extending `BoundaryRing`; the replay runs on every build with deadlock detection as the divergence signal. A step the code takes that the model refuses is a red build, which makes the proven-spec-beside-divergent-code defect class — the class the program was founded on — structurally visible.

12 NestedGuardian: Witness-Augmented Research Mode

`NestedGuardian` explores a layered threshold construction:

- validators run the AFT deterministic message flow,
- each proposal carries its guardian committee certificate,
- external witness committees cross-check the slot,

- chain state anchors witness assignments, witness manifests, and slashing evidence.

The instantiated runtime scope includes:

- deterministic witness assignment from the active witness set,
- registered on-chain witness manifests,
- witness certificates bound into guardianized slot certificates,
- safety proofs for the witness-augmented kernel,
- bounded model checking for reassignment, outage, and checkpoint transitions.

The instantiated proof package consists of:

- safety formalization,
- bounded operational model checking for reassignment, outage, and checkpoint transitions,
- a discharged composed-liveness bridge under the target eventual-fair scheduler through the recurrence/reduction/totality/collapse chain together with a matching persistent churn/restart simulator.

NestedGuardian is therefore a witness-augmented mode with a finished theorem bridge. It is experimental as an operational mode, not as a theorem crutch. Its witness-coded family supplies an explicit constructive recovery carrier and completed bridge artifacts; it does not reduce the promoted AFT theorem surface to an experimental-only claim. That constructive recovery carrier is a witness-coded publication-bundle recovery contract with compact hot-path bindings and cold-path registry-driven close-or-abort resolution. The abstract coded recovery-family contract is realized by the parametric **SystematicXorK0fKPlus1V1** parity family plus the bounded true non-parity **SystematicGf256K0fNV1** family with at least two parity shares, implemented with systematic Cauchy-style coefficients and exercised in bounded **2-of-4**, **3-of-5**, **3-of-7**, **4-of-6**, and **4-of-7** lanes. These lanes resolve from threshold-many public reveals, objective recovered-support conflict, or deterministic missingness. The bounded **3-of-5**, **4-of-6**, and **4-of-7** lanes restore or exceed threshold-support overlap while the bounded **3-of-7** lane shows that recovered-support conflict remains fail-closed even when overlap is absent. A bounded conformance harness checks that every threshold-sized reveal subset reconstructs and every below-threshold subset fails across the shipped XOR and GF256 realizations of that contract. Those recovered reveals lift deterministically into an explicit positive close-extraction payload and then into an explicit extractable bulletin-surface payload that binds the verifying publication bundle, verifying canonical bulletin close, canonical bulletin-availability bytes, and sorted bulletin entries.

The completed bounded family also carries the ordinary durable and restart surfaces. A bounded recovered-only two-slot harness shows that consecutive recovered surfaces derive the ordinary publication-frontier predecessor chain and the same authoritative **CanonicalCollapseObject** predecessor chain without the ordinary publication-bundle lane. A second bounded recovered-only harness shows a close/abort/close bulletin window with a recovered omission-dominated middle slot: recovered data alone materializes the ordinary positive closes and extracted bulletin surfaces on the outer slots, materializes the ordinary omission-dominated abort in the middle while keeping omission evidence and recovered bulletin entries public, and preserves the same authoritative collapse continuity chain. A bounded read-side harness shows that the same recovered-only durable chain

yields the same compact replay-prefix / state-root continuity view that the ordinary lane exposes plus a bounded recovered canonical-header prefix. The execution module verifies that replay tip as the same restart anchor the ordinary lane uses, retains both recovered prefixes in execution state, and uses the recovered canonical-header ancestry for bounded anchored reads plus parent-block / parent-state-root continuity after restart when ordinary recent blocks are absent.

Validator consensus orchestration derives the same bounded recovered restart parent anchor from the recovered canonical-header / collapse surface when the ordinary committed block is absent, and the **GuardianMajority** engine consumes that bounded recovered canonical-header prefix as restart-time parent/QC context for synthetic parent-height QC continuity when the full committed block is absent. The same recovered surface yields a bounded recovered certified-header window plus a restart-only recovered **BlockHeader** / QC cache window for restart-time header / QC lookup plus bounded QC-certified recovered branch reconciliation over the configured local five-entry window. Those windows fold by exact overlap with a configurable budget, exercised at five windows into a longer seventeen-step recovered certified branch without ordinary committed headers. Those same cold-path loaders consume a configurable exact-overlap segment budget, exercised at four exact-overlap segments into a longer fifty-three-step recovered certified branch with direct registry-extraction parity and explicit interior-overlap conflict rejection. Those same restart loaders also compose two overlapping four-segment exact-overlap folds into a longer eighty-nine-step recovered certified branch with direct registry-extraction parity and explicit inter-fold overlap conflict rejection, and tests exercise the same recursive carrier at three overlapping four-segment folds into a one-hundred-twenty-five-step recovered certified branch. The runtime has a bounded conformance harness over one, two, and three stitched segment folds, matching the same production-loader / registry-extraction carrier at the corresponding fifty-three-step, eighty-nine-step, and one-hundred-twenty-five-step branches.

The restart path also pages older exact-overlap segment folds on demand beneath that bounded suffix via an overlap-checked cursor while keeping only the bounded recent suffix plus the streamed page live in engine caches, and tests exercise that paged carrier at a longer two-hundred-thirty-three-step recovered certified branch with direct registry-extraction parity plus explicit duplicate-page, missing-gap, and late-page overlap rejection. Restart therefore streams recovered certified ancestry until target height or recovered-history exhaustion without a theorem-relevant fixed depth bound.

The archival internalization layer uses an active archived-recovered-history profile naming retention horizon, exact-overlap paging geometry, segment / fold geometry, checkpoint update rule, and profile evolution. Archived segments, archived restart pages, archival checkpoints, and archival retention receipts bind to historically referenced profile hashes and governing activation hashes under a published profile-activation chain before archived continuation is trusted, and ordinary canonical collapse history names the archived checkpoint / activation pair that archived restart is authorized to follow. Archived replay and archived publication correctness are historical and index-free: they follow the canonical-collapse-anchored or object-carried activation hash plus predecessor/checkpoint history rather than whichever archived activation is merely latest in local state.

When the recovered certificate carries objective omission proofs, the same recovered lane routes into the ordinary **OmissionDominated** abort path while keeping omission evidence and recovered bulletin entries public. The recovery-family contract, canonical-collapse / replay retrievability anchor, and recovered-state historical retrievability surface are therefore part of ordinary endogenous AFT history rather than a narrower theorem-side qualifier. The directly discharged recurrence/reduction/totality/collapse chain closes the former distance to the relay-free deterministic all-but-one ($n-1$ of n) Byzantine safety theorem in M_{AFT} , so no recoverability or architecture-specific semantic gap remains at the AFT theorem surface.

13 Security and Assumption Surface

The AFT family depends on explicit assumptions. They are not optional prose.

Layer	Required assumptions	Failure consequence
Base finality	current manifests, current epoch state, majority thresholds, guardian non-equivocation, valid checkpoints, enough validators and guardians to reach quorum	conflicting base-finality evidence or loss of progress
Canonical ordering	objective publication, compact publication-frontier binding and contradiction rules, canonical cutoff, deterministic eligibility, append-only bulletin commitment, proof soundness, protocol-native retrievability profile / manifest / custody / challenge surface, canonical bulletin-close extraction-or-abort, cheap omission verification, accountable omission publication	loss of compact frontier consistency, deterministic extraction-or-abort, accountability, or the ordering theorem
Sealed finality	valid asymptote policy, current witness seed, valid witness or observer assignments, checkpoint freshness, valid log consistency proofs, deterministic transcript/challenge derivation, accountable challenge publication	incorrect sealed release, loss of accountability, or fail-closed stalling
Sealed effects	correct finality-tier policy, sound verifier metadata, nullifier uniqueness, intent/public-input binding	replay or unsafe irreversible effect release
NestedGuardian	sound witness assignment and registration, correct reassignment handling, witness committee non-equivocation	degraded safety or open liveness behavior

The family-level discipline is simple: if an assumption fails, AFT should degrade to stalling, aborting, or withholding release before it degrades to unsafe irreversible behavior.

14 The Common-Boundary Theorem Corpus (AFT-CB)

The AFT Common-Boundary research program (AFT-CB) restated the strong-tier settlement story as a theorem corpus with a single machine-checked discipline: every theorem carries an explicit *Assumes* line drawn from a ten-entry assumption ledger, every positive theorem is paired with the lower bound that makes its trade forced, and every claim any AFT surface prints is gated by checkers that fail the build on an unconditional or over-rounded assertion. The full paper proofs live in the corpus document (`specs/common_boundary_theorems.md`); their mechanized successors are the TLAPS/TLC kernels of Appendix A; the ledger’s single owner is the public whitepaper’s research-candidate section, mirrored here; and the claim surface is bound by the program’s claim adjudication (`specs/p4_claim_adjudication.md`). This section is the citation spine the rest of the paper hangs from.

14.1 The assumption ledger (A1–A10)

Every deterministic statement in the corpus consumes only entries from this ledger, named on its own *Assumes* line. Machine discipline (enforced in CI): no safety theorem may cite A5 (synchrony is

confined to liveness); A9 appears only where the physical clock is genuinely consumed; A10 belongs to the design-open succession extension alone.

Id	Assumption
A1	Signature unforgeability and hash collision resistance.
A2	Minimal Honesty Axiom (MHA): at least one honest, non-equivocating signer per publicly committed signer configuration. Economically bought (bonds, diversity floors, activation queues), never proven, and stated so.
A3	Honest single-final-ack discipline over crash-consistent storage: the journal entry is written in the same atomic step that lets the share exist, and recovery replays bytes without re-deciding.
A4	Reachability to at least one honest signer — for submitters (completeness) and for retrievers (availability’s serving path).
A5	Post-GST synchrony as a full honest mesh (pairwise delivery within the bound between honest parties) and bounded dishonest bonded weight. Liveness, seal cadence, and the live tier only; no strong-lineage safety claim cites A5.
A6	A historical-freshness anchor, exactly one deployed, each separately priced: a recent authenticated checkpoint; forward-secure or puncturable signatures with secure key erasure; or an external objective checkpoint. Verifiable-delay elapsed history is a hardening layer over an anchor, never a standalone anchor.
A7	Proof-system soundness (succinct verification only; full-replay verification needs A1 alone).
A8	Per-seal key evolution with verified immediate erasure: no signing-capable material ever persists to a medium that outlives its slot, so later compromise of former members — including full-media imaging — forges nothing (everlasting safety).
A9	Verifiable-delay-function sequentiality with a bounded, published adversary hardware advantage (a physical assumption). The protocol’s objective clock wherever ticks are read; safety-critical consumption is confined to the elapsed-history hardening and the design-open succession extension.
A10	Succession-path observation: a bound on the acting successor’s view of pre-consented-succession evidence. PROVISIONAL — consumed by the succession extension alone, whose claims are withdrawn to design-open; the uniqueness and lineage ladder never cites it.

14.2 Theorems T1–T9

Statements are condensed; the corpus document carries each full proof, and each entry names its mechanization artifact and its lower-bound pairing. Throughout, a *Unanimous Boundary Close* (UBC) for a configuration C_v and seal slot s is an n -of- n certificate of individually verifiable final-ack shares over a domain-separated tuple.

T1 — Uniqueness within one configuration. *Assumes A1, A2, A3.* At most one UBC exists per (v, s) : two closes over different boundary roots imply every member of C_v final-acked both, so zero members are honest ($\neg A2$). The claim is weight-independent and holds under adversarial delivery and scheduling; retries never weaken it because pre-acks are attempt-tagged and can never be reinterpreted as final-ack shares (a wire-format premise, stated as such). *Mechanization:* `BoundaryRing.t1a` + TLAPS inductive invariant; TLC at the one-honest-member corner. *Pairing:* L1.

T2 — Completeness (censorship at the seal layer). *Assumes A1, A2, A4.* An artifact received by at least one honest signer before that signer’s local cutoff is contained in every boundary that signer final-acks, hence in the unique close. Censoring an artifact out of a sealed boundary requires corrupting all n signers ($\neg A2$) or preventing it from reaching any honest signer ($\neg A4$). Post-close omission from a sealed batch is either carried by a typed, verifiable omission justification or

invalidates the seal. *Mechanization*: completeness invariant over declared sets (`BoundaryRing.tla`). *Pairing*: L-E (eclipse).

T3 — Availability (custody-attested). *Assumes* $A1, A2, A4$. Validate-and-hold is a signing obligation: the final-ack signature *means* the signer obtained the bytes, recomputed the root, and committed to retain and serve. A UBC therefore implies every compliant signer holds; under $A2$ one signer actually does, so a sealed boundary’s bytes are held and served through the horizon with *zero* audit records — the audit lane produces slashing evidence and is proven non-load-bearing. Deletion by the other $n - 1$ members is harmless; retrieval additionally needs $A4$ ’s reach edge. *Mechanization*: `CustodyObligation.tla` (close-time replication at compliant signers; zero-audit witness). *Pairing*: L-H (holder necessity).

T4a — Live-tier liveness. *Assumes* $A5$. Block production advances under the live engine’s own fault bound and partial-synchrony assumption, independently of the ring: no Boundary Ring rule appears in the proposal, vote, or commit path. A ring stall delays only the release of effects *typed* irreversible (unsealed-over-unsafe); the engine’s asynchronous-fallback obligation is a named residual (RES-R10), deliberately not claimed. *Mechanization*: the two-tier separation property of `BoundaryLiveness.tla`. *Pairing*: L-OPEN — the row closes only when the sim-exercised fallback lands.

T4b — Seal cadence. *Assumes* $A2, A3, A4, A5$. With all n members responsive and honest-behaving post-GST, seals advance at bounded cadence (each pipeline phase completes within a bounded number of message delays, per-slot volume bounded by the deployment parameter). One non-responsive member freezes seal cadence — never the chain (T4a) — until a handover or a labeled re-genesis; no rule substitutes a smaller quorum. *Mechanization*: `BoundaryLiveness.tla`, model-checked at $n \leq 4$ in both the advancement and freeze directions. *Pairing*: L2 (unanimity forces the one-withholder stall).

T5a — Membership canonicity. *Assumes* $A1, A2, A3$. For lineage segments whose transitions are UBC-closed, two lineages diverging at version v imply full-configuration self-incrimination at the transition slot (T1’s argument verbatim), so under per-configuration MHA the configuration lineage from a root is unique. *Mechanization*: `MembershipTransition.tla`. *Pairing*: L1, applied per configuration.

T5b — Bootstrap (validity and freshness, separately). *Assumes* $A1, A7, A6$ (*anchor as deployed; $A9$ iff the elapsed-history hardening is enabled*). A newcomer that verifies the lineage’s recursive proof chain from genesis and runs exactly one deployed $A6$ freshness mechanism obtains validity under $A7$ (or $A1$ alone via full replay) and freshness under the deployed mechanism’s own assumption, cited by name. A newcomer with no live $A6$ mechanism is out of model — stated, not smoothed. The VDF chain is a hardening layer only: tick length evidences work and elapsed time, never *which* history is live. *Mechanization*: bootstrap-freshness obligation (`MembershipTransition.tla`), $A6$ -conditional in the statement itself. *Pairing*: L-LR (long-range indistinguishability, a necessity bound).

T5c’ — Assurance-preserving reconfiguration. *Assumes* $A1, A2, A3$. Every strong-ring transition carries old-ring unanimity *and* new-ring acceptance, so configuration lineage is MHA-inductive end to end with no weight anywhere in the strong chain. No proof-of-silence path exists:

the transition action set contains no silence-derived action — asserted over the formal action set and made unrepresentable in the runtime’s types. An anchored re-genesis is a typed lineage root, never continuity. *Mechanization:* `MembershipTransition.tla` (transition obligation + the no-silence action-set assertion); the membership plane’s type layer. *Pairing:* L2 and L1.

T5d — Pre-consented succession safety (withdrawn). *Assumes A1, A2, A3, A9, A10.* The succession extension’s safety claim is *withdrawn for this cycle*: three successive formulations were refuted by the program’s own adversarial reviews, and the operative exits from a dead ring remain the unanimous handover and the labeled re-genesis. `SuccessionClock.tla` survives as an honestly narrowed kernel; the conditions a fourth formulation must meet are recorded in the specification’s status banner. *Pairing:* L-OPEN, claim withdrawn.

T6 — Composition (the lattice meet). *Assumes A1.* Every certificate profile carries a machine-readable assumption label; the collapse verifier computes the meet over all constituents; a composed object’s effective guarantee is exactly its weakest constituent’s, including the post-quantum bit (AND). Reporting the meet is exact — anything stronger is unsound (L-M), anything weaker discards guarantee — and exhaustive matching over profiles makes a silently unlabeled new profile a compile error. *Mechanization:* the runtime lattice’s unit gates (meet, pq-meet, exhaustive-match-by-construction). *Pairing:* L-M (above-meet unsoundness).

T7 — Forensic accountability (accountable degradation). *Assumes A1.* Proven against the wire format, not an abstraction: above the fault threshold, either no seal exists for a slot, or any party holding both conflicting seals extracts a cryptographically attributable set containing every participant necessary for the violation — for n -of- n UBCs, the entire configuration. The extraction runs on the two certificates alone; the wire-format smallnesses (individually verifiable shares, domain separation, broadcast duty) are premises, and the theorem is false without them. Holder-relativity is stated: an all-Byzantine set can complete a second certificate privately; surfacing is guaranteed by use, honest broadcast, and watchtowers, never by wire-format magic. *Mechanization:* `ForensicAccountability.tla` over the wire-level certificate structure, including the staged all-Byzantine double-seal trace extracting the full set. *Pairing:* L9.

T8 — Selection supply (probabilistic, forever separate). *Assumes: none — an economic model, not a ledger-consuming theorem.* Under a stated model (adversary budget, bond schedule, constituency-correlation structure over the diversity floors, an adaptive-corruption window from sortition unpredictability, and a standby-capture term), the probability that open selection yields at least one honest signer in a formed configuration is published *as a probability* with all model inputs. The composition rule is absolute: deterministic safety is only ever “conditioned on the selection event,” and no surface may convert this probability into a deterministic Byzantine-tolerance figure — the corpus’s checkers enforce the ban. *Mechanization:* none (the economics memo owns the published computation). *Pairing:* L-OPEN — the cheapest-capture supply bound is analysis work, recorded open.

T9 — Maximal accountable safety. *Assumes A1.* Any seal-uniqueness violation cryptographically convicts the entire violating coalition — all n signers — with slashable floor $n \times$ bond. The convicted-to-necessary ratio is 1.0, which is maximal: L9 states no protocol can attribute beyond the signers of the conflicting certificates, and T9 meets that ceiling. The lineage-escape

maneuver (laundering a double-seal through a self-serving re-genesis) is closed upstream by root-admissibility rules, so the maneuver changes the offense’s name, not its price. *Mechanization:* `ForensicAccountability.tla` (attribution completeness); share-level verification in the hardened signer. *Pairing:* L9 (the attribution cap).

14.3 Lower-bound pairing

Each positive theorem is paired with the necessity bound that makes its trade forced; three rows are honestly open. Per the program’s claim ladder, no flagship-completeness rung prints while any row is open — the corpus’s own checkers enforce the gate (Section 19).

Positive theorem	Lower bound	Status
T1 uniqueness	L1 (zero-honest equivocation)	cited
T2 completeness	L-E (eclipse)	cited
T3 availability	L-H (holder necessity)	cited
T4a live-tier liveness	—	L-OPEN (RES-R10: the engine’s asynchronous bound)
T4b seal cadence	L2 (unanimity forces the one-withholder stall)	cited
T5a membership canonicity	L1 (per configuration)	cited
T5b bootstrap	L-LR (long-range indistinguishability; a necessity bound that forces <i>some</i> anchor)	cited
T5c’ reconfiguration	L2 + L1	cited
T5d succession safety	—	L-OPEN; claim withdrawn
T6 composition	L-M (above-meet unsoundness)	cited
T7 forensic accountability	L9 (attribution cap)	cited
T8 selection supply	—	L-OPEN (cheapest-capture supply bound)
T9 maximal accountable safety	L9 (ratio 1.0 is the cap)	cited

15 Formal Results

The repository’s machine-checked surfaces align with the protocol family as follows:

Formal module	Core proved results
GuardianMajorityProof GuardianMajority	QuorumCertificatesIntersect, InvariantImpliesSafety executable bounded checks for finalization witness soundness and slot safety InvariantImpliesCertifiedUniqueness, InvariantImpliesRecoverability,
CanonicalOrderingProof CanonicalOrdering	InvariantImpliesOmissionDominates executable bounded checks for publication, cutoff, certification, omission, and recoverability InvariantImpliesBaseSafety, InvariantImpliesCanonicalCloseSafety, InvariantImpliesCloseAbortExclusive,
AsymptoteProof Asymptote	InvariantImpliesAbortDominatesSealed executable bounded checks for transcript publication, challenge publication, canonical close, and canonical abort witness-assignment soundness, witness-certificates-stay-assigned,
NestedGuardianProof NestedGuardian	InvariantImpliesSafety executable bounded checks for reassignment, outage, and checkpoint admissibility
BoundaryRingProof	T1 uniqueness via a TLAPS inductive invariant over an unconstrained Byzantine adversary; no admitted-equals-canonical definition anywhere in the module
BoundaryRing BoundaryLiveness	TLC exploration at the one-honest-member corner ($n = 3$) and at $n = 4$ two-tier separation (live progress under a ring stall) and seal-cadence advancement plus the freeze direction, model-checked at $n \leq 4$
MembershipTransitionProof	T5a lineage canonicity, the T5c' transition obligation, the no-silence action-set assertion, and typed lineage roots
CustodyObligationProof	T3 validate-and-hold: close-time replication at compliant signers, the zero-audit witness, and reconstruct-before-release
ForensicAccountabilityProof	T7 against the wire-level certificate structure, including the staged all-Byzantine double-seal extraction
SuccessionClockProof	the honestly narrowed succession kernel (T5d itself withdrawn; the kernel header states the narrowing)
BoundaryRingTraceReference	the trace-conformance lane: a committed reference trace replays against BoundaryRing with deadlock detection as the divergence signal, on every build

Every formal artifact named in this section, and every additional proof artifact cited by file name below, is embedded verbatim in Appendix A so that the served yellow paper remains self-contained outside the repository.

These formal surfaces prove the deterministic safety kernels that the broader classical-agreement bridge builds on. The full promoted all-but-one ($n - 1$ of n) Byzantine agreement sentence — conditional on the AFT model delta — is then completed by the recurring, reduction, totality, and collapse layers discussed above, rather than by these module-local safety kernels in isolation. Compact publication-frontier summaries internalize same-slot and predecessor consistency on the live path, while the protocol-native retrievability plane internalizes closed-bulletin extraction-or-abort as an endogenous part of the realizing architecture rather than a purely algebraic consequence of the authenticated message transcript alone.

The accountable-adversary variant therefore composes these formal kernels with runtime accountability rules rather than replacing them: replay-deduplicated objective fault publication and optional policy-controlled membership updates live in the implementation and policy layer above the proved deterministic core.

15.1 Complete omission-dominance witness artifact

The formal package includes an executable TLC witness module, `internal-docs/architecture/protocols/aft/f` paired with `internal-docs/architecture/protocols/aft/formal/canonical_ordering/CanonicalOrderingO`. The exact module and configuration are embedded in Appendix A.5.1 and Appendix A.5.2; the summary below highlights the executed public-evidence shape that the witness reaches.

Executed witness trace. The module intentionally asks TLC to violate the state predicate `NoOmissionDominanceWitness` so that the checker emits a concrete public evidence trace. The executed trace is:

1. `PublishStep`: `tx1` enters slot 1’s bulletin surface.
2. `PublishStep`: `tx2` enters the same bulletin surface.
3. `CloseCutoffStep`: the canonical cutoff closes and the recoverable set becomes $\{tx1, tx2\}$.
4. `CandidateCertifyStep`: a candidate certificate for $\{tx1\}$ appears.
5. `ProveOmissionStep`: an omission proof for `tx2` against that candidate certificate appears.

The same formal package carries a bounded recursive-continuity model, `internal-docs/architecture/protocols/aft/formal/canonical_ordering/CanonicalCollapseR` paired with `internal-docs/architecture/protocols/aft/formal/canonical_ordering/CanonicalCollapseR`. The exact source of both artifacts is embedded in Appendix A.5.3 and Appendix A.5.4. That model captures the reference `HashPcdV1` continuity carrier directly: deterministic recursive proof steps, predecessor-proof hashing, `CanonicalCollapseExtensionCertificate` carriage, and the rule that non-genesis header admission depends on the anchored predecessor proof relation. The runtime exposes a `SuccinctSp1V1` backend seam for those same recursive public inputs, and that seam is exercised by consensus verification and durable-state gating. The formal model still stops at the reference carrier rather than mechanizing a cryptographic recursive accumulator or ZK backend.

At the final state, the incomplete candidate certificate remains *unadmitted* while the omission proof is present. In the executable model this happens because the admitted object must already equal the canonical set; the trace therefore does not replace the theorem. Its purpose is narrower and useful: it gives a bounded, executed witness of the exact public-evidence shape from which omission dominance follows. Readers can run the trace directly and inspect the six-state witness without reconstructing the paper’s argument by hand.

16 Implementation Correspondence

This section is non-normative. It names the exact runtime symbols that carry the protocol objects defined above.

16.1 Core ordering symbols

The core ordering data structures and free functions are public Rust symbols re-exported through `ioi_types::app`. They are the runtime carriers for the abstract ordering objects and verifier rules defined above.

Abstract object	Current runtime symbols
Bulletin surface commitment	BulletinCommitment, BulletinSurfaceEntry
Compact publication availability binding	PublicationAvailabilityReceipt
Compact publication frontier	PublicationFrontier
Publication-frontier contradiction	PublicationFrontierContradiction
Canonical ordering proof families	CanonicalOrderProofSystem, with concrete variants HashBindingV1 and CommittedSurfaceV1
Canonical public-input tuple	CanonicalOrderPublicInputs
Live succinct witness payload	CommittedSurfaceCanonicalOrderProof
Omission evidence	OmissionProof
Order certificate	CanonicalOrderCertificate
Canonical ordering function	canonical_order_tx_hashes canonical_publication_availability _receipt_hash, canonical_publication_frontier_hash, canonical_publication_frontier _contradiction_hash build_publication_availability_receipt, build_publication_frontier, verify_publication_frontier, verify_publication_frontier_contradiction
Publication-frontier hashing	
Publication-frontier construction / verification	canonical_order_public_inputs, canonical_order_public_inputs_hash build_reference_canonical_order _proof_bytes build_committed_surface_canonical _order_certificate verify_canonical_order_certificate
Public-input hashing	
Reference proof construction	
Live certificate construction	
Certificate verification	

16.2 Guardian and sealing symbols

The guardian, witness, observer, and sealed-effect objects are likewise public Rust symbols re-exported through `ioi_types::app`. They encode the exact evidence surface checked by validators and downstream gateways.

Abstract object	Current runtime symbols
Guardian slot certificate	GuardianQuorumCertificate
Witness certificate	GuardianWitnessCertificate
Equal-authority observer assignment	AsymptoteObserverAssignment
Correlation budget	AsymptoteObserverCorrelationBudget
Observer observation request	AsymptoteObserverObservationRequest
Observer transcript	AsymptoteObserverTranscript
Observer transcript commitment	AsymptoteObserverTranscriptCommitment
Observer challenge	AsymptoteObserverChallenge, with kinds MissingTranscript, TranscriptMismatch, VetoTranscriptPresent, ConflictingTranscript, and InvalidCanonicalClose
Observer challenge commitment	AsymptoteObserverChallengeCommitment
Canonical observer close	AsymptoteObserverCanonicalClose
Canonical observer abort	AsymptoteObserverCanonicalAbort
Finality tier	FinalityTier, with variants BaseFinal and SealedFinal
Collapse state	CollapseState, with variants Pending, BaseFinal, Sealing, Abort, SealedFinal, Escalated, and Invalid
Sealing policy	AsymptotePolicy
Observer statement	AsymptoteObserverStatement
Observer certificate	AsymptoteObserverCertificate
Divergence signal	DivergenceSignal, DivergenceSignalKind
Sealed finality proof	SealedFinalityProof
Observer binding for sealed effects	sealed_finality_proof_observer_binding
Proof-carrying effect object	SealObject

16.3 Common-boundary and membership symbols

The AFT-CB program’s runtime surfaces are public Rust symbols on the same re-export path. They realize the objects of Section 11.

Abstract object	Current runtime symbols
Closed boundary (R1)	ClosedBoundary, BoundaryOmissionJustification, close_reference_single_member_boundary
Availability binding / standing (R2)	derive_bulletin_availability_binding, availability_standing_from_close, AvailabilityAuditRecord
Resolution log (R3)	AftResolutionRecord, AftResolutionKind, aft_resolution_log_entry_key
Assumption lattice (T6)	CertificateProfile, AssumptionLabel, assumption_meet, FinalityReceipt
Ring membership plane (R5)	RingMemberRegistration, RingActivationQueue, BoundaryRingConfig, AssurancePreservingHandover, AnchoredRegenesisRoot, LiveTierEjection, CustodyHandoverReceipt, assign_seats, WatchtowerCountersignRecord
Membership registry wiring (R5)	register_ring_member@v1, publish_ring_genesis_config@v1, publish_ring_handover@v1
Hardened seal signer (R9)	EvolvingSealSigner, SealShare, verify_seal_share, extract_double_signers
Trace conformance (R13)	BoundaryRingDriver, the committed reference trace, and the replay generator in the formal-floor CI job
Succinct continuity (R4c)	SuccinctDriverConfig::pinned, the governed verifying-key pin, and the no-mock CI gate

16.4 Verifier, orchestration, and persistence symbols

Verification and orchestration entry points are split across four public namespaces: `ioi_types::app`, `ioi_crypto::sign::guardian_committee`, `ioi_services::guardian_registry`, and the `GuardianMajorityEngine` runtime. Engine-local checks are named here by their exact method identifiers.

Responsibility	Current runtime symbols
Guardian certificate verification	verify_quorum_certificate
Witness certificate verification	verify_witness_certificate
Deterministic observer assignment	derive_asymptote_observer_assignments, derive_asymptote_observer_plan_entries GuardianContainer::observe_asymptote_request, GuardianContainer::request_remote_asymptote _observer_observation,
Observer-side transcript / challenge derivation	GuardianContainer::sign_consensus_with_guardian GuardianMajorityEngine::verify_asymptote _canonical_observer_sealed_finality, GuardianMajorityEngine::verify_asymptote _sealed_finality GuardianMajorityEngine::verify _canonical_order_enrichment; published CanonicalOrderAbort dominates positive order certificates and inconsistent mixed parent state
Sealed finality verification in the engine	GuardianMajorityEngine::verify _publication_frontier_enrichment; published PublicationFrontierContradiction dominates conflicting or stale frontiers
Canonical-order enrichment verification in the engine	GuardianMajorityEngine::verify _publication_frontier_enrichment; published PublicationFrontierContradiction dominates conflicting or stale frontiers
Publication-frontier enrichment verification in the engine	build_committed_surface_canonical _order_certificate during finalization
Block finalization attachment of order certificates	build_publication_frontier during finalization
Block finalization attachment of publication frontier	canonicalize_observer_sealed_finality_proof, publish_canonical_observer_artifacts build_http_egress_seal_object, verify_seal_object GuardianMajorityEngine::verify _guardianized_certificate
Canonical observer proof publication	SafetyGadget, Pacemaker, TimeoutCertificate load_bulletin_commitment, load_canonical_order_certificate GuardianRegistry::load_publication_frontier, GuardianRegistry::load_publication _frontier_contradiction derive_canonical_order_execution_object, derive_canonical_order_public_obstruction CanonicalOrderAbortReason::MissingOrderCertificate, BulletinSurfaceReconstructionFailure, BulletinSurfaceMismatch, InvalidBulletinClose, OmissionDominated, CertificateHeightMismatch, RandomnessMismatch, OrderedTransactionsRootMismatch, ResultingStateRootMismatch, InvalidPublicInputsHash, InvalidBulletinAvailabilityCertificate, InvalidProofBinding derive_canonical_collapse_object, GuardianMajorityEngine::verify_published _canonical_collapse_object
Sealed-effect construction and checking	
Guardianized proposal verification in the engine	
Base-engine safety and timing	
Bulletin and order-certificate state access	
Publication-frontier state access	
Ordering close-or-abort derivation	
Ordering abort taxonomy	
Protocol-wide collapse derivation and verification	
Proposal-surface recursive continuity	BlockHeader::previous_canonical_collapse_commitment_hash, BlockHeader::canonical_collapse_extension_certificate, CanonicalCollapseExtensionCertificate, CanonicalCollapseCommitment, verify_block_header_canonical_collapse_evidence, ConsensusDecision::ProduceBlock load_bulletin_commitment, load_canonical_order_certificate, load_canonical_order_abort, load_canonical_collapse_object publish_aft_publication_frontier@v1, publish_aft_canonical_order_artifact_bundle@v1, publish_aft_canonical_order_abort@v1, publish_aft_canonical_collapse_object@v1, report_aft_omission@v1, publish_asymptote_observer_transcript@v1, publish_asymptote_observer_transcript_commitment@v1, report_asymptote_observer_challenge@v1, publish_asymptote_observer_challenge_commitment@v1, publish_asymptote_observer_canonical_close@v1,
Bulletin, order, and collapse state access	

These symbols are named here so that the paper maps directly onto the current runtime without making the reader chase separate prose specifications. In the implementation, validator finalization publishes a protocol-visible `CanonicalCollapseObject` through `publish_aft_canonical_collapse_object@v1` after local post-commit header enrichment, `guardian_registry` exposes `load_canonical_collapse_object`, and `GuardianMajorityEngine::verify_publication` cross-checks any published copy against the proof-carried header surface when it is available. The externally finalized AFT commit path persists the same object keyed by slot height, rather than treating ordering and sealing close-or-abort evidence as loose enrichments detached from state persistence. The same post-commit path also builds `PublicationFrontier` via `build_publication_frontier` and publishes it through `publish_aft_publication_frontier@v1`; `GuardianMajorityEngine::verify_publication_frontier_enrichment` rechecks same-slot binding, predecessor linkage, and any dominating contradiction object. In the current proposal-surface continuity slice, validator orchestration now signs both `previous_canonical_collapse_commitment_hash` and a typed `CanonicalCollapseExtensionCertificate` into each produced AFT block header. In the current clean-break runtime slice, `CanonicalCollapseObject` itself carries `continuity_accumulator_hash` and `continuity_recursive_proof`, so the certificate now carries the predecessor commitment plus predecessor proof hash rather than an explicit carried ancestor vector, full predecessor collapse object, or predecessor recursive proof object. `Asymptote` leaders stall rather than produce when that required extension certificate is unavailable, proposal verification recursively checks that the carried predecessor commitment hash matches the signed predecessor link, checks that the predecessor commitment’s resulting state root matches the signed parent-state root, and rechecks those carried public inputs against the anchored predecessor collapse object when one is available. QC promotion therefore relies on a reference recursive proof-carrying continuity relation rather than on a recomputed predecessor digest alone, and only treats continuity-linked headers as progress-bearing.

17 Benchmark Context

The performance evidence in this paper has two sources:

- literature-reported baselines for `HotStuff`, `Narwhal/Tusk`, and `Bullshark`,
- the repository’s native `benchmark_throughput` target for matched `GuardianMajority` and `Asymptote` scenarios.

The purpose of this section is architectural positioning, not false equivalence across unmatched environments.

17.1 Paper-reported baselines

System	Reported throughput	Reported latency	Source type
HotStuff baseline	1,800 tx/s	1 s	Narwhal/Tusk comparison setting
Narwhal-HotStuff	130,000 tx/s	under 2 s	Narwhal/Tusk paper
Narwhal-HotStuff with additional workers	up to 600,000 tx/s	n/a	Narwhal/Tusk paper
Tusk	160,000 tx/s	about 3 s	Narwhal/Tusk paper
Bullshark	125,000 tx/s	about 2 s	Bullshark paper

The baseline interpretation is:

- **HotStuff** demonstrates the leader-based partially synchronous baseline,
- **Narwhal-HotStuff** demonstrates the gain from separating dissemination from ordering,
- **Bullshark** demonstrates high-throughput DAG-oriented ordering,
- **AFT** adds a new separation: order surfacing and effect release are no longer identified with dense positive voting.

17.2 Native repository measurements

No performance measurement in this section is load-bearing for any theorem in this paper; the promoted theorem surface stands independently of benchmark tuning, harness state, or optimization maturity.

The corrected native measurements below were produced on March 21, 2026 after bringing the repository’s native **AFT** benchmark path back into sync with the current **GuardianMajority / base_final** harness semantics:

- shared-tip readiness no longer treats a transient `get_status.height = 0` as authoritative when a resilient tip probe already sees committed blocks,
- submission alignment and leader-targeted ingress now use a resilient tip block rather than a potentially stale status height,
- authoritative chain-commit scans start from the resilient pre-submission tip instead of rescanning stale empty-block prefixes,
- sustained throughput for fast probes now stops on the authoritative full-chain commit scan for the highest committed height; sampled committed-status visibility is retained only as an auxiliary lag diagnostic,
- the benchmark surface now exposes alignment, spill, and replay diagnostics explicitly so missed due blocks or replay churn are not hidden behind a single TPS number.

The table below reports the current matched bulk-throughput proof point for the native **GuardianMajority** 4-validator **base_final** scenario under that corrected harness. This is a repository-native **AFT** proof point on the Jellyfish state path with a ‘1 s’ block interval, not an apples-to-apples claim against published Narwhal/Tusk/Bullshark environments.

scenario	validators	attempted	accepted	committed	blocks	inj. tps	sust. tps	p50	p95	p99	sealed p50	sealed p95
guardian_majority_4v_base_final_bulk	4	16384	16384	16384	1	21370.89	9647.66	1293.45	1603.73	1695.24	–	–

These measurements imply four things:

- the earlier single-digit and low-hundreds TPS reports were artifacts of benchmark-path bugs rather than of the protocol surface itself,
- the current repository now clears the lower edge of the intended **8k-16k TPS** range in a matched native 4-validator **GuardianMajority** bulk probe,

- on this corrected probe the batch lands in a single block, so the main remaining AFT work is ceiling-raising and aggregate-domain multiplication rather than recovering from a broken harness,
- these values should be read as corrected repository baselines, not as optimized ceilings and not as apples-to-apples superiority claims over published **Narwhal/Tusk/Bullshark** environments.

17.3 Measured costs, reproduction-classed

The AFT-CB program’s measured-cost discipline: every number carries its reproduction class, and a number without a reproduction path is not printed.

Class A — exact, zero variance (gate-pinned). The SCALE-encoded byte sizes of the canonical objects, pinned by a repository test so any wire-format change surfaces in review:

Quantity	Bytes	Reading
Validate-and-hold binding	72	per sealed slot, ring-size independent
Canonical bulletin close	180	the sealed-slot commitment
Optional audit record	106	strictly marginal: a close verifies with zero audit records, so the at-rest audit cost is zero and 106 B is paid only per recorded probe

Class B — reproduces within CPU variance. The real succinct continuity backend proves one recursion step in roughly 27 seconds of CPU (about 84 seconds wall for the three-step end-to-end chain, CPU backend), reproduced by the repository’s nightly proof lane. GPU or prover-network backends would shift these by an order of magnitude and are out of scope for an in-repository measurement.

Class C — box-variable, deferred. Boundary-close latency and sustained throughput at $n \in \{4, 8\}$ come from the existing paper-benchmark harness; the instrument is built, and the figures await a pinned benchmark runner because a number measured on a variable development box would fail its own reproduce-within-variance gate (Section 19.5). The native measurements earlier in this section predate this discipline and are retained as architectural positioning only.

18 Related Work

Geeq’s Proof of Honesty (PoH) is important prior work in the “99%” (their figure, not an AFT claim) direction: it argues for globally honest and canonical chain recovery in pure software under assumptions including at least one honest node and an honest relay, and it uses public evidence plus user-side verification to push beyond ordinary dense-vote intuition [9]. AFT differs in a key respect: it is relay-free and coordinator-free. Correctness does not depend on any designated honest relay, broadcaster, or privileged recovery role. Instead, durable ordering and sealed effects collapse through protocol-native public-evidence objects whose negative forms are immediately decisive.

Narwhal/Tusk and Bullshark separate dissemination from ordering, but they still ground ordering safety in dense positive certificates and quorum-intersection arguments over the ordered object itself [2, 3]. HoneyBadgerBFT and Dumbo move to asynchronous common-subset or MVBA-style agreement, but they still rely on classical Byzantine-majority thresholds rather than a public-state-continuity theorem over a canonical public surface [4, 5]. Chainlink CCIP and optimistic cross-chain

mechanisms add secondary monitoring layers, risk-management committees, or challenge windows to detect and halt bad transfers, but the decisive recovery rule is operational or economic rather than the immediate acceptance of a uniquely valid proof-carrying canonical object recoverable by any equal-authority validator [8]. Accountable subgroup multisigs and finality gadgets add equivocation evidence and slashing, yet still keep dense subgroup votes as the positive safety object [6, 7]. AFT differs by making ordering and sealed release relay-free public-evidence close-or-abort objects with dominant negative proofs: honest validators that observe the same closed public boundary derive the same decisive object, and publication is downstream of that deterministic derivation rather than its source.

19 Claims and Non-Claims

19.1 The headline claim

The claim this paper prints, adopted verbatim from the AFT-CB program’s claim adjudication and machine-gated in CI (no stronger sentence can enter this corpus without failing the build):

Deterministic all-but-one safety for certified boundaries; live consensus under separately bounded adversarial weight and eventual synchrony.

Both halves state their own conditions. The first half is T1 (Section 14.2): weight-independent, mechanized, and unconditional under the Minimal Honesty Axiom — which is economically bought, never proven, and priced rather than assumed away. The second half is T4a, and it *names* its two conditions (bounded adversarial weight; eventual synchrony), which is exactly why the named asynchronous residual does not undercut it: the claim never asserted asynchronous liveness. Stronger “completeness”-class rungs exist in the program’s claim ladder and are BLOCKED — they do not print here or anywhere, and the enumerated open gates live in the adjudication document.

19.2 The maximality claims

Two claims in this paper are superlatives, and both are theorems rather than comparisons — each is a cited positive-theorem/lower-bound pair in the corpus (Section 14.3), so the ceiling and the achievement print together:

- **Safety at the terminal threshold (T1 paired with L1).** With zero honest signers, two conflicting n -of- n certificates are constructible by any adversary in any model — equivocation is unpreventable, so *no consensus protocol can claim a stronger safety threshold than all-but-one*. Within the model delta, AFT’s certified-boundary safety meets that terminal threshold exactly, weight-independently, under A1–A3 alone, with the mechanized kernel of Appendix A.7.2 as the machine-checked proof. On the safety axis there is nothing above this point to claim; the axis ends here.
- **Accountability at the attribution ceiling (T9 paired with L9).** No protocol can attribute a violation beyond the signers of the conflicting certificates — ratio 1.0 of convicted to necessary is the cap. AFT meets the cap: a seal-uniqueness violation convicts the entire configuration, member by member, from the two certificates alone, with slashable floor $n \times$ bond. Safety violations are not merely detected; they are maximally expensive and maximally attributed, by construction.

The classical comparison, stated precisely: no protocol in the classical carrier exceeds $f < n/3$ for agreement, because the indefinite-suppression move is legal there. AFT does not out-tune that family inside its own model — it replaces the carrier, which makes the suppression move illegal, and in the replaced carrier the safety threshold ascends from a fraction to the information-theoretic maximum. Prior work has gestured at this territory with rounded prose figures; this paper’s form of the claim is exact ($n - 1$ of n , never a rounded percentage), mechanized (TLAPS, with the trace lane holding the runtime to the kernel on every build), and attributed (every violator convicted individually). The liveness price and the residual list (Section 19.5) print beside these claims and are part of them.

19.3 Claims

This paper additionally claims:

- a production guardian-backed base-finality model in which validator quorums are composed with guardian committee admissibility,
- proof-carrying canonical ordering with omission dominance and uniqueness, plus protocol-native endogenous retrievability,
- boundary-honest ordering: the committed-surface certificate consumes a previously closed boundary, carries typed omission justifications, and its verification is self-contained — an unjustified omission is an invalid block, so censorship is observable by construction,
- compact signed publication-frontier summaries and short contradiction objects that internalize same-slot and predecessor consistency on the live path,
- a public-state-continuity formulation in which closed slot boundaries admit at most one positive public execution object and conflicting candidates admit short objective obstruction witnesses,
- all-but-one ($n - 1$ of n) equal-authority ordering consensus in the AFT model delta, under explicit assumptions,
- deterministic sealed-effect collapse with challenge dominance, with sealed closes immutable and post-close evidence confined to an append-only resolution log,
- a relay-free, coordinator-free, pure-software model-delta theorem yielding deterministic all-but-one ($n - 1$ of n) Byzantine safety for durable ordering and sealed-effect safety over the explicit bulletin / retrievability surface, conditional on the AFT model delta,
- that the whole AFT stack breaks the classical indefinite-suppression lower bound, in the AFT model delta, by identifying the first illegal adversary step and realizing the visibility delta without a trusted relay — the classical threshold is not out-tuned but made moot by replacing its carrier,
- safety at the terminal threshold: no protocol in any model can exceed all-but-one for safety (L1), and AFT meets that ceiling weight-independently with a mechanized proof (Section 19.2),
- accountability at the attribution ceiling: convicted-to-necessary ratio 1.0 (T9/L9), every violator individually, from the two certificates alone,

- the common-boundary theorem corpus of Section 14: T1–T9 with explicit Assumes lines over the A1–A10 ledger, each theorem paired with its necessity bound, with the mechanized kernels of Appendix A as their machine-checked successors and a trace-conformance lane holding code to the uniqueness kernel on every build,
- forensic accountability at the attribution ceiling: a seal-uniqueness violation convicts the entire configuration individually from the two certificates alone (T7/T9), with the wire discipline implemented in the hardened signer,
- a witness-augmented mode for layered threshold constructions,
- machine-checked deterministic safety kernels for the major protocol surfaces.

19.4 Non-claims

This paper does **not** claim:

- that public state continuity arises for free inside the classical DLS / PBFT partially synchronous message-passing model,
- that the classical $n > 3f / f < n/3$ lower bound has been refuted inside that bare model; the claim is instead that AFT operates in the explicit M_{AFT} model delta where the adversary cannot indefinitely suppress published admissible artifacts from correct parties after stabilization. Remove that visibility resource, or realize it circularly through the same Byzantine-majority relay set, and the paper is no longer claiming the same result. This is not a retreat; it is the exact location of the break. AFT does not merely vary a parameter inside the dense-vote carrier. It replaces the carrier with direct dissemination plus compressed public echo and states the new proof obligation at theorem surface,
- freedom from bulletin-publication, protocol-native retrievability obligations, or proof-soundness assumptions,
- that compact publication-frontier summaries make the bulletin surface self-extracting inside the classical authenticated-message model,
- a full asynchronous liveness theorem under arbitrary registry, log, witness, and outage schedules,
- an apples-to-apples measured throughput superiority over **HotStuff**, **Narwhal**, **Tusk**, or **Bullshark** from the current repository alone.

19.5 Named residuals (the limitations list)

The AFT-CB program closed complete-with-residuals, and this paper carries the residual list verbatim rather than smoothing it. None of these is a silent gap; each names its closing condition in the program record.

- **The MHA is bought, not proven.** T1’s safety is conditional on at least one honest signer per configuration; the economics memo prices how open selection *supplies* that condition (T8, as a probability only), and no surface converts that price into a deterministic tolerance figure.

- **RES-R10 — asynchronous liveness.** The live engine’s recovery is timeout-driven and fully deterministic, so no asynchronous progress guarantee is possible without a common coin (an engine-structure change). The fallback is designed (view synchronizer, trigger, coin options) and not built; T4a’s pairing row stays open and every completeness-class claim rung stays gated on it.
- **RES-R11 — the VDF clock plane.** Every candidate construction requires externally vetted cryptography (class-group, isogeny, or checkpointed-hash approaches each fail a stated requirement today); selection and review are an owner action. Sortition’s beacon is the reference ordering beacon until this closes — an honest label, not a security bound.
- **RES-R12 and the T5d withdrawal — succession.** Three successive succession formulations were refuted by the program’s own adversarial reviews; the claim is withdrawn, the operative exits from a dead ring are the unanimous handover and the labeled re-genesis, and no path in the estate activates a successor ring from elapsed time or non-response. The conditions a fourth formulation must meet are recorded in the specification’s status banner.
- **RES-P4.3-throughput.** Close-latency and throughput figures at $n \in \{4, 8\}$ await a pinned benchmark runner; the instrument exists and the reproduction command is recorded, but a number measured on a variable box would fail its own reproduce-within-variance gate and is therefore not printed (Section 17.3).
- **R14 — the verified verifier.** A spec-extracted or refinement-proven UBC verifier remains a design-permitted residual; its outward complement (an independent twin implementation against exported conformance vectors) is filed as an owner packet.

19.6 Model-delta public-state-continuity theorem

The runtime supports a stronger theorem than the older accountable summary because the mechanism and system model are now stated directly at theorem surface.

Theorem 6 (Relay-Free Deterministic All-But-One ($n - 1$ of n) Byzantine Safety in M_{AFT}). *Assume the system model of Section 5, including the direct-dissemination / compressed-echo delta of Lemma 1, guardian non-equivocation, proof soundness, compact signed publication-frontier summaries whose same-slot and predecessor-link contradictions are objectively checkable, protocol-native retrievability objects plus canonical bulletin-close extraction-or-abort for the bulletin surface bound by those frontiers, deterministic transcript/challenge derivation for sealing, durable state advancement only through canonical collapse, and irreversible-effect release only through collapse-bound sealed proofs. Then all but one validator ($n - 1$ of n) may behave arbitrarily without creating a conflicting valid durable canonical-order outcome or a conflicting valid sealed-effect release outcome. Honest validators that observe the same closed public boundary derive the same close-or-abort result; publication, penalties, and validator-set updates only accelerate convergence and future hygiene. The classical $n > 3f / f < n/3$ threshold is not load-bearing in M_{AFT} , because the classical adversary’s indefinite-suppression step is no longer legal.*

The theorem does not claim that DLS / PBFT has been refuted inside M_{DLS} . It claims that AFT is not a member of that model: the first illegal lower-bound adversary step is indefinite suppression of a published admissible artifact from correct parties after stabilization. Bulletin close, publication frontier, retrievability, recovery, restart continuity, canonical collapse, and collapse-bound sealing are not theorem-side caveats; they are the protocol-internal realizing mechanism of M_{AFT} . A

criticism that AFT “changes the model” is therefore descriptive rather than fatal unless it identifies circular realization of the visibility resource or a hidden relay, coordinator, TEE, bootstrap oracle, or external recovery service.

The theorem is stronger than the older accountable summary because invalid positive objects are not merely rejectable or penalty-bearing. They are already dominated by deterministic negative objects and collapse-gated durable state before any penalty logic runs. Readers trained on dense-vote BFT will be tempted to treat the absence of quorum-intersection safety as disqualifying. In AFT, that is precisely the point: authority is carried by uniquely valid public execution objects and omission-dominant contradiction witnesses, with durable advancement gated by canonical collapse. Compact publication frontiers keep the live hot path cheap by internalizing consistency and lineage checks, while the protocol-native retrievability plane discharges deterministic extraction-or-abort on the full bulletin surface.

The recurrence / reduction / totality / collapse chain is not narrative glue. Its exact artifact chain is embedded verbatim in Appendix A, ending in the classical-agreement collapse wrapper itself.

Theorem 7 (Realizing-Mechanism Statement). *The new publication-frontier slice is not the theorem by itself; the theorem is the promoted AFT model-delta sentence, carried by the full AFT public-evidence continuity architecture. Compact signed publication-frontier summaries, compact availability receipts, short frontier contradiction objects, endogenous retrievability profiles, manifests, custody assignments, custody receipts, custody responses, retrievability challenges, canonical extraction-or-abort, canonical collapse, and historical retrievability together realize the direct-dissemination / compressed-echo visibility resource stated above. Removing that protocol-native retrievability plane removes part of the realizing mechanism rather than merely shaving off an optional refinement.*

Proposition 2 (Scoped Witness-Coded Constructive Corollary). *Assume the hypotheses of the relay-free deterministic all-but-one ($n - 1$ of n) Byzantine safety theorem in M_{AFT} , plus sound witness assignment and witness certificate binding, a completed witness-coded recovery family with compact hot-path bindings and cold-path share delivery / reveal / reconstruction, and availability of the retained recovered publication surface together with any older recovered-history pages needed for restart-time extraction. Then the **NestedGuardian** witness-coded carrier deterministically materializes the same positive close or canonical abort surface, the same authoritative collapse chain, the same replay-prefix / restart-anchor surface, and the same restart-time canonical-header / certified-header / block-header ancestry surface as the ordinary lane for the completed recovery families. Restart can therefore stream recovered certified ancestry until target height or recovered-history exhaustion without a theorem-relevant fixed depth bound, while keeping only compact bindings on the hot path and bounded-memory paging on the restart path.*

This runtime result is stronger than the separation closeout because it no longer stops at “compact frontiers bind a recoverability claim.” It shows that the live AFT stack can re-materialize the same durable and restart-critical surfaces without violating the throughput guardrails, while keeping deeper recovered history inside ordinary endogenous AFT state.

This is one constructive route to Byzantine agreement in a realizable permissioned model delta rather than a weaker contrast class. In the classical dense-vote presentation, worst-case progress is usually phrased in terms of $> 2/3$ honest participation and quorum intersection as the main safety mechanism. In AFT’s realization, authority is obtained instead through uniquely valid public-evidence objects plus dominant negative proofs, without trusted relays, privileged coordinators, TEEs, or dense positive quorum intersection. The live publication-frontier slice internalizes compact same-slot and predecessor consistency on the hot path, while explicit recoverability of the bound

bulletin surface, canonical collapse, and historical retrievability complete the same AFT model-delta theorem. Publication and accountability serve as reinforcement rather than the source of correctness. Liveness then follows from direct authenticated dissemination, compressed echo visibility, and eventual publication of the protocol-defined artifacts, and the recurrence/reduction/totality/collapse chain closes that route formally. That is the precise sense in which the classical $n > 3f / f < n/3$ barrier stops being load-bearing here: the promoted theorem no longer grants the adversary indefinite suppression of published admissible information.

The correct one-line summary is:

AFT is a proof-carrying, guardian-backed, omission-dominant consensus and settlement family in which base finality, canonical ordering, and irreversible-effect release are separated and then recomposed through shared evidence.

A Embedded Formal Artifacts

This appendix embeds verbatim every proof artifact referenced by name in the yellow paper, together with the repository path from which it is sourced. The goal is to keep the served paper self-contained even when it is read outside the repository. The artifacts are grouped by protocol surface and rendered with line numbers so the appendix reads as a standalone proof bundle rather than as a raw repository dump.

A.1 GuardianMajority

Core guardian-backed base-finality model and proof.

A.1.1 GuardianMajorityProof.tla

Repository path. `\detokenize{internal-docs/architecture/protocols/aft/formal/guardian_majority/GuardianMajorityProof.tla}`

```
1  ---- MODULE GuardianMajorityProof ----
2  EXTENDS Naturals, FiniteSets, TLAPS
3
4  CONSTANT Validators, Blocks, Slots, Epochs, QuorumSize
5
6  ASSUME QuorumIntersection ==
7    \A S, T \in SUBSET Validators :
8      /\ Cardinality(S) >= QuorumSize
9      /\ Cardinality(T) >= QuorumSize
10     => S \cap T # {}
11
12  VARIABLES votes, certs
13
14  vars == <<votes, certs>>
15
16  VoteDomain == Validators \X Slots \X Blocks \X Epochs
17  CertDomain == Slots \X Blocks \X Epochs \X (SUBSET Validators)
18
19  Init ==
20    /\ votes = {}
21    /\ certs = {}
22
23  HasVote(v, s) ==
24    \E b \in Blocks, e \in Epochs : <<v, s, b, e>> \in votes
25
26  VoteStep ==
27    \E v \in Validators, s \in Slots, b \in Blocks, e \in Epochs :
28      /\ ~HasVote(v, s)
29      /\ votes' = votes \cup {<<v, s, b, e>>}
30      /\ UNCHANGED certs
31
32  CertifyStep ==
33    \E s \in Slots, b \in Blocks, e \in Epochs, Q \in SUBSET Validators :
34      /\ Cardinality(Q) >= QuorumSize
35      /\ \A v \in Q : <<v, s, b, e>> \in votes
36      /\ certs' = certs \cup {<<s, b, e, Q>>}
37      /\ UNCHANGED votes
38
39  Next == VoteStep \/ CertifyStep
40
41  TypeInvariant ==
42    /\ votes \subseq VoteDomain
43    /\ certs \subseq CertDomain
44
45  NoDualVotes ==
46    \A v \in Validators, s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
47      /\ <<v, s, b1, e1>> \in votes
48      /\ <<v, s, b2, e2>> \in votes
49      => /\ b1 = b2
```

```

50      /\ e1 = e2
51
52 CertSoundness ==
53   \A s \in Slots, b \in Blocks, e \in Epochs, Q \in SUBSET Validators :
54     <<s, b, e, Q>> \in certs
55     => /\ Cardinality(Q) >= QuorumSize
56         /\ \A v \in Q : <<v, s, b, e>> \in votes
57
58 Invariant == TypeInvariant /\ NoDualVotes /\ CertSoundness
59
60 Safety ==
61   \A s \in Slots,
62     b1 \in Blocks,
63     b2 \in Blocks,
64     e1 \in Epochs,
65     e2 \in Epochs,
66     Q1 \in SUBSET Validators,
67     Q2 \in SUBSET Validators :
68     /\ <<s, b1, e1, Q1>> \in certs
69     /\ <<s, b2, e2, Q2>> \in certs
70     => /\ b1 = b2
71         /\ e1 = e2
72
73 Spec == Init /\ [] [Next]_vars
74
75 THEOREM InitImpliesTypeInvariant == Init => TypeInvariant
76   BY SMTT(30) DEF Init, TypeInvariant, VoteDomain, CertDomain
77
78 THEOREM InitImpliesNoDualVotes == Init => NoDualVotes
79   BY DEF Init, NoDualVotes
80
81 THEOREM InitImpliesCertSoundness == Init => CertSoundness
82   BY DEF Init, CertSoundness
83
84 THEOREM InitImpliesInvariant == Init => Invariant
85   BY InitImpliesTypeInvariant, InitImpliesNoDualVotes,
86     InitImpliesCertSoundness DEF Invariant
87
88 THEOREM VotePreservesTypeInvariant == TypeInvariant /\ VoteStep => TypeInvariant'
89   BY SMTT(30) DEF TypeInvariant, VoteStep, HasVote, VoteDomain, CertDomain
90
91 THEOREM VotePreservesNoDualVotes == NoDualVotes /\ VoteStep => NoDualVotes'
92   BY SMTT(30) DEF NoDualVotes, VoteStep, HasVote
93
94 THEOREM VotePreservesCertSoundness == CertSoundness /\ VoteStep => CertSoundness'
95   BY DEF CertSoundness, VoteStep
96
97 THEOREM VotePreservesInvariant == Invariant /\ VoteStep => Invariant'
98   BY VotePreservesTypeInvariant, VotePreservesNoDualVotes,
99     VotePreservesCertSoundness DEF Invariant
100
101 THEOREM CertifyPreservesTypeInvariant == TypeInvariant /\ CertifyStep => TypeInvariant'
102   BY SMTT(30) DEF TypeInvariant, CertifyStep, VoteDomain, CertDomain
103
104 THEOREM CertifyPreservesNoDualVotes == NoDualVotes /\ CertifyStep => NoDualVotes'
105   BY DEF NoDualVotes, CertifyStep
106
107 THEOREM CertifyPreservesCertSoundness == CertSoundness /\ CertifyStep => CertSoundness'
108   BY SMTT(30) DEF CertSoundness, CertifyStep
109
110 THEOREM CertifyPreservesInvariant == Invariant /\ CertifyStep => Invariant'
111   BY CertifyPreservesTypeInvariant, CertifyPreservesNoDualVotes,

```

```

112     CertifyPreservesCertSoundness DEF Invariant
113
114 THEOREM StepPreservesInvariant == Invariant /\ Next => Invariant'
115   BY VotePreservesInvariant, CertifyPreservesInvariant DEF Next
116
117 THEOREM StutterPreservesTypeInvariant ==
118   TypeInvariant /\ UNCHANGED vars => TypeInvariant'
119   BY DEF TypeInvariant, vars
120
121 THEOREM StutterPreservesNoDualVotes ==
122   NoDualVotes /\ UNCHANGED vars => NoDualVotes'
123   BY DEF NoDualVotes, vars
124
125 THEOREM StutterPreservesCertSoundness ==
126   CertSoundness /\ UNCHANGED vars => CertSoundness'
127   BY DEF CertSoundness, vars
128
129 THEOREM StutterPreservesInvariant == Invariant /\ UNCHANGED vars => Invariant'
130   BY StutterPreservesTypeInvariant, StutterPreservesNoDualVotes,
131     StutterPreservesCertSoundness DEF Invariant
132
133 THEOREM NextPreservesInvariant == Invariant /\ [Next]_vars => Invariant'
134   BY StepPreservesInvariant, StutterPreservesInvariant DEF vars
135
136 THEOREM QuorumCertificatesIntersect ==
137   \A s \in Slots,
138     b1 \in Blocks,
139     b2 \in Blocks,
140     e1 \in Epochs,
141     e2 \in Epochs,
142     Q1 \in SUBSET Validators,
143     Q2 \in SUBSET Validators :
144     /\ Invariant
145     /\ <<s, b1, e1, Q1>> \in certs
146     /\ <<s, b2, e2, Q2>> \in certs
147     => Q1 \cap Q2 # {}
148   BY SMTT(60), QuorumIntersection DEF Invariant, CertSoundness
149
150 THEOREM InvariantImpliesSafety == Invariant => Safety
151   BY SMTT(60), QuorumCertificatesIntersect DEF Invariant, Safety, NoDualVotes,
152     CertSoundness
153
154 =====

```

A.1.2 GuardianMajority.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/guardian_majority/GuardianMajority.tla}

```

1  ---- MODULE GuardianMajority ----
2  EXTENDS Naturals, FiniteSets, TLC, TLAPS
3
4  CONSTANT Validators, Blocks, Slots, Epochs, QuorumSize, InitialEpoch
5
6  ASSUME InitialEpochInEpochs == InitialEpoch \in Epochs
7
8  ASSUME QuorumIntersection ==
9    \A S, T \in SUBSET Validators :
10     /\ Cardinality(S) >= QuorumSize

```

```

11      /\ Cardinality(T) >= QuorumSize
12      => S \cap T # {}
13
14  VARIABLES votes, finalized, finalizerSets, registryEpoch, manifestEpoch, guardianReady,
15      ↪ checkpointLevel
16
17  vars == <<votes, finalized, finalizerSets, registryEpoch, manifestEpoch, guardianReady,
18      ↪ checkpointLevel>>
19
20  CheckpointLevels == 0..2
21
22  VoteEvent(v, s, b, e) == <<v, s, b, e>>
23  Finalization(s, b, e) == <<s, b, e>>
24
25  VoteDomain == {VoteEvent(v, s, b, e) : v \in Validators, s \in Slots, b \in Blocks, e \in Epochs}
26  FinalizationDomain == {Finalization(s, b, e) : s \in Slots, b \in Blocks, e \in Epochs}
27
28  Init ==
29      /\ votes = {}
30      /\ finalized = {}
31      /\ finalizerSets = [f \in FinalizationDomain |-> {}]
32      /\ registryEpoch = [v \in Validators |-> InitialEpoch]
33      /\ manifestEpoch = [v \in Validators |-> InitialEpoch]
34      /\ guardianReady = [v \in Validators |-> TRUE]
35      /\ checkpointLevel = [v \in Validators |-> 1]
36
37  HasVote(v, s) ==
38      \E b \in Blocks, e \in Epochs : VoteEvent(v, s, b, e) \in votes
39
40  CanVote(v, s, b, e) ==
41      /\ v \in Validators
42      /\ s \in Slots
43      /\ b \in Blocks
44      /\ e \in Epochs
45      /\ guardianReady[v]
46      /\ checkpointLevel[v] > 0
47      /\ registryEpoch[v] = e
48      /\ manifestEpoch[v] = e
49      /\ ~HasVote(v, s)
50
51  Vote(v, s, b, e) ==
52      /\ CanVote(v, s, b, e)
53      /\ votes' = votes \cup {VoteEvent(v, s, b, e)}
54      /\ UNCHANGED <<finalized, finalizerSets, registryEpoch, manifestEpoch, guardianReady,
55      ↪ checkpointLevel>>
56
57  EligibleVoters(s, b, e) ==
58      {v \in Validators :
59          /\ VoteEvent(v, s, b, e) \in votes
60          /\ guardianReady[v]
61          /\ checkpointLevel[v] > 0
62          /\ registryEpoch[v] = e
63          /\ manifestEpoch[v] = e}
64
65  CanFinalize(s, b, e) ==
66      /\ s \in Slots
67      /\ b \in Blocks
68      /\ e \in Epochs
69      /\ Cardinality(EligibleVoters(s, b, e)) >= QuorumSize
70
71  Finalize(s, b, e) ==
72      /\ CanFinalize(s, b, e)

```

```

70 /\ finalized' = finalized \cup {Finalization(s, b, e)}
71 /\ finalizerSets' = [finalizerSets EXCEPT ![Finalization(s, b, e)] = EligibleVoters(s, b, e)]
72 /\ UNCHANGED <<votes, registryEpoch, manifestEpoch, guardianReady, checkpointLevel>>
73
74 AdoptEpoch(v, e) ==
75 /\ v \in Validators
76 /\ e \in Epochs
77 /\ e >= registryEpoch[v]
78 /\ registryEpoch' = [registryEpoch EXCEPT ![v] = e]
79 /\ manifestEpoch' = [manifestEpoch EXCEPT ![v] = e]
80 /\ UNCHANGED <<votes, finalized, finalizerSets, guardianReady, checkpointLevel>>
81
82 RegistryRollback(v, e) ==
83 /\ v \in Validators
84 /\ e \in Epochs
85 /\ e < registryEpoch[v]
86 /\ registryEpoch' = [registryEpoch EXCEPT ![v] = e]
87 /\ UNCHANGED <<votes, finalized, finalizerSets, manifestEpoch, guardianReady, checkpointLevel>>
88
89 ManifestRollback(v, e) ==
90 /\ v \in Validators
91 /\ e \in Epochs
92 /\ e < manifestEpoch[v]
93 /\ manifestEpoch' = [manifestEpoch EXCEPT ![v] = e]
94 /\ UNCHANGED <<votes, finalized, finalizerSets, registryEpoch, guardianReady, checkpointLevel>>
95
96 GuardianOutage(v) ==
97 /\ v \in Validators
98 /\ guardianReady[v]
99 /\ guardianReady' = [guardianReady EXCEPT ![v] = FALSE]
100 /\ UNCHANGED <<votes, finalized, finalizerSets, registryEpoch, manifestEpoch, checkpointLevel>>
101
102 GuardianRecovery(v) ==
103 /\ v \in Validators
104 /\ ~guardianReady[v]
105 /\ guardianReady' = [guardianReady EXCEPT ![v] = TRUE]
106 /\ UNCHANGED <<votes, finalized, finalizerSets, registryEpoch, manifestEpoch, checkpointLevel>>
107
108 AdvanceCheckpoint(v, level) ==
109 /\ v \in Validators
110 /\ level \in CheckpointLevels
111 /\ level > checkpointLevel[v]
112 /\ checkpointLevel' = [checkpointLevel EXCEPT ![v] = level]
113 /\ UNCHANGED <<votes, finalized, finalizerSets, registryEpoch, manifestEpoch, guardianReady>>
114
115 CheckpointRollback(v, level) ==
116 /\ v \in Validators
117 /\ level \in CheckpointLevels
118 /\ level < checkpointLevel[v]
119 /\ checkpointLevel' = [checkpointLevel EXCEPT ![v] = level]
120 /\ UNCHANGED <<votes, finalized, finalizerSets, registryEpoch, manifestEpoch, guardianReady>>
121
122 Next ==
123 /\ E v \in Validators, s \in Slots, b \in Blocks, e \in Epochs : Vote(v, s, b, e)
124 /\ E s \in Slots, b \in Blocks, e \in Epochs : Finalize(s, b, e)
125 /\ E v \in Validators, e \in Epochs : AdoptEpoch(v, e)
126 /\ E v \in Validators, e \in Epochs : RegistryRollback(v, e)
127 /\ E v \in Validators, e \in Epochs : ManifestRollback(v, e)
128 /\ E v \in Validators : GuardianOutage(v)
129 /\ E v \in Validators : GuardianRecovery(v)
130 /\ E v \in Validators, level \in CheckpointLevels : AdvanceCheckpoint(v, level)
131 /\ E v \in Validators, level \in CheckpointLevels : CheckpointRollback(v, level)

```

```

132
133 TypeInvariant ==
134   /\ votes \subseq VoteDomain
135   /\ finalized \subseq FinalizationDomain
136   /\ finalizerSets \in [FinalizationDomain -> SUBSET Validators]
137   /\ registryEpoch \in [Validators -> Epochs]
138   /\ manifestEpoch \in [Validators -> Epochs]
139   /\ guardianReady \in [Validators -> BOOLEAN]
140   /\ checkpointLevel \in [Validators -> CheckpointLevels]
141
142 NoDualVotes ==
143   \A v \in Validators, s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
144     /\ VoteEvent(v, s, b1, e1) \in votes
145     /\ VoteEvent(v, s, b2, e2) \in votes
146     => /\ b1 = b2
147         /\ e1 = e2
148
149 FinalizationWitnessSoundness ==
150   \A s \in Slots, b \in Blocks, e \in Epochs :
151     Finalization(s, b, e) \in finalized
152     => /\ Cardinality(finalizerSets[Finalization(s, b, e)]) >= QuorumSize
153     /\ \A v \in finalizerSets[Finalization(s, b, e)] :
154       VoteEvent(v, s, b, e) \in votes
155
156 Invariant == TypeInvariant /\ NoDualVotes /\ FinalizationWitnessSoundness
157
158 Safety ==
159   \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
160     /\ Finalization(s, b1, e1) \in finalized
161     /\ Finalization(s, b2, e2) \in finalized
162     => b1 = b2
163
164 Spec ==
165   Init /\ [] [Next]_vars
166
167 THEOREM OneInCheckpointLevels == 1 \in CheckpointLevels
168   BY SMT DEF CheckpointLevels
169
170 THEOREM InitImpliesInvariant == Init => Invariant
171 PROOF
172 <1>1. Init => TypeInvariant
173   BY SMT, InitialEpochInEpochs, OneInCheckpointLevels
174   DEF Init, TypeInvariant, VoteDomain, FinalizationDomain
175 <1>2. Init => NoDualVotes
176   BY DEF Init, NoDualVotes
177 <1>3. Init => FinalizationWitnessSoundness
178   BY DEF Init, FinalizationWitnessSoundness
179 <1>4. QED
180   BY <1>1, <1>2, <1>3 DEF Invariant
181
182 THEOREM StepPreservesTypeInvariant == Invariant /\ Next => TypeInvariant'
183   BY SMTT(30) DEF Invariant, TypeInvariant, Next, Vote, Finalize, AdoptEpoch,
184     RegistryRollback, ManifestRollback, GuardianOutage, GuardianRecovery,
185     AdvanceCheckpoint, CheckpointRollback, CanVote, HasVote, CanFinalize,
186     EligibleVoters, VoteEvent, Finalization, VoteDomain, FinalizationDomain
187
188 THEOREM StepPreservesNoDualVotes == Invariant /\ Next => NoDualVotes'
189   BY SMT DEF Invariant, NoDualVotes, Next, Vote, Finalize, AdoptEpoch,
190     RegistryRollback, ManifestRollback, GuardianOutage, GuardianRecovery,
191     AdvanceCheckpoint, CheckpointRollback, CanVote, HasVote, VoteEvent
192
193 THEOREM StepPreservesFinalizationWitnessSoundness ==

```

```

194 Invariant /\ Next => FinalizationWitnessSoundness'
195 BY SMTT(30) DEF Invariant, FinalizationWitnessSoundness, Next, Vote, Finalize,
196     AdoptEpoch, RegistryRollback, ManifestRollback, GuardianOutage,
197     GuardianRecovery, AdvanceCheckpoint, CheckpointRollback,
198     CanFinalize, EligibleVoters, VoteEvent, Finalization
199
200 THEOREM StutterPreservesInvariant == Invariant /\ UNCHANGED vars => Invariant'
201 BY SMT DEF Invariant, vars
202
203 THEOREM NextPreservesInvariant == Invariant /\ [Next]_vars => Invariant'
204 PROOF
205 <1>1. Invariant /\ Next => Invariant'
206 BY StepPreservesTypeInvariant, StepPreservesNoDualVotes,
207     StepPreservesFinalizationWitnessSoundness DEF Invariant
208 <1>2. Invariant /\ UNCHANGED vars => Invariant'
209 BY StutterPreservesInvariant
210 <1>3. QED
211 BY <1>1, <1>2 DEF vars
212
213 THEOREM FinalizedWitnessesIntersect ==
214     \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
215     /\ Invariant
216     /\ <<s, b1, e1>> \in finalized
217     /\ <<s, b2, e2>> \in finalized
218     => \E v \in Validators :
219         /\ <<v, s, b1, e1>> \in votes
220         /\ <<v, s, b2, e2>> \in votes
221 PROOF
222 <1>1. TAKE s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs
223 <1>2. ASSUME Invariant,
224     <<s, b1, e1>> \in finalized,
225     <<s, b2, e2>> \in finalized
226 PROVE \E v \in Validators :
227     /\ <<v, s, b1, e1>> \in votes
228     /\ <<v, s, b2, e2>> \in votes
229 <2>1. finalizerSets[<<s, b1, e1>>] \subseq Validators
230 BY SMT, <1>2 DEF Invariant, TypeInvariant, FinalizationDomain, Finalization
231 <2>2. finalizerSets[<<s, b2, e2>>] \subseq Validators
232 BY SMT, <1>2 DEF Invariant, TypeInvariant, FinalizationDomain, Finalization
233 <2>3. Cardinality(finalizerSets[<<s, b1, e1>>]) >= QuorumSize
234 BY SMT, <1>2 DEF Invariant, FinalizationWitnessSoundness
235 <2>4. Cardinality(finalizerSets[<<s, b2, e2>>]) >= QuorumSize
236 BY SMT, <1>2 DEF Invariant, FinalizationWitnessSoundness
237 <2>5. finalizerSets[<<s, b1, e1>>] \cap finalizerSets[<<s, b2, e2>>] # {}
238 BY <2>1, <2>2, <2>3, <2>4, QuorumIntersection
239 <2>6. PICK v \in finalizerSets[<<s, b1, e1>>] \cap finalizerSets[<<s, b2, e2>>] : TRUE
240 BY <2>5
241 <2>7. <<v, s, b1, e1>> \in votes
242 BY SMT, <1>2, <2>6 DEF Invariant, FinalizationWitnessSoundness
243 <2>8. <<v, s, b2, e2>> \in votes
244 BY SMT, <1>2, <2>6 DEF Invariant, FinalizationWitnessSoundness
245 <2>9. QED
246 BY SMT, <2>6, <2>7, <2>8
247 <1>3. QED
248 BY <1>2
249
250 THEOREM InvariantImpliesSafety ==
251     \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
252     /\ Invariant
253     /\ <<s, b1, e1>> \in finalized
254     /\ <<s, b2, e2>> \in finalized
255     => /\ b1 = b2

```

```

256         /\ e1 = e2
257 PROOF
258 <1>1. TAKE s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs
259 <1>2. ASSUME Invariant,
260         <<s, b1, e1>> \in finalized,
261         <<s, b2, e2>> \in finalized
262     PROVE /\ b1 = b2
263         /\ e1 = e2
264     <2>1. \E v \in Validators :
265         /\ <<v, s, b1, e1>> \in votes
266         /\ <<v, s, b2, e2>> \in votes
267     BY SMT, FinalizedWitnessesIntersect, <1>2
268     <2>2. PICK v \in Validators :
269         /\ <<v, s, b1, e1>> \in votes
270         /\ <<v, s, b2, e2>> \in votes
271     BY <2>1
272     <2>3. QED
273     BY SMT, <1>2, <2>2 DEF Invariant, NoDualVotes
274 <1>3. QED
275     BY <1>2
276
277 =====

```

A.2 CanonicalOrdering

Proof-carrying ordering model and its main proof surface.

A.2.1 CanonicalOrderingProof.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/canonical_ordering/CanonicalOrderingProof.tla}

```
1  ---- MODULE CanonicalOrderingProof ----
2  EXTENDS Naturals, FiniteSets, TLAPS
3
4  CONSTANT Slots, Transactions
5
6  VARIABLES bulletins, closedCutoffs, admittedCerts, omissionProofs, recoveredSets
7
8  vars == <<bulletins, closedCutoffs, admittedCerts, omissionProofs, recoveredSets>>
9
10 CertEvent(s, c) == <<s, c>>
11 OmissionEvent(s, c, tx) == <<s, c, tx>>
12
13 CanonicalSet(s) == bulletins[s]
14 RecoverableSet(s) == IF s \in closedCutoffs THEN CanonicalSet(s) ELSE {}
15
16 BulletinDomain == [Slots -> SUBSET Transactions]
17 CutoffDomain == SUBSET Slots
18 CertDomain == Slots \X (SUBSET Transactions)
19 OmissionDomain == Slots \X (SUBSET Transactions) \X Transactions
20 RecoveredDomain == [Slots -> SUBSET Transactions]
21
22 TypeInvariant ==
23   /\ bulletins \in BulletinDomain
24   /\ closedCutoffs \in CutoffDomain
25   /\ admittedCerts \subseq CertDomain
26   /\ omissionProofs \subseq OmissionDomain
27   /\ recoveredSets \in RecoveredDomain
28
29 CertifiedSoundness ==
30   \A s \in Slots, c \in SUBSET Transactions :
31     CertEvent(s, c) \in admittedCerts
32     => /\ s \in closedCutoffs
33         /\ c = CanonicalSet(s)
34
35 OmissionSoundness ==
36   \A s \in Slots, c \in SUBSET Transactions, tx \in Transactions :
37     OmissionEvent(s, c, tx) \in omissionProofs
38     => /\ s \in closedCutoffs
39         /\ tx \in CanonicalSet(s)
40         /\ tx \notin c
41
42 RecoveredSoundness ==
43   \A s \in Slots :
44     recoveredSets[s] = RecoverableSet(s)
45
46 Invariant ==
47   /\ TypeInvariant
48   /\ CertifiedSoundness
49   /\ OmissionSoundness
```

```

50 /\ RecoveredSoundness
51
52 CertifiedUniqueness ==
53   \A s \in Slots, c1 \in SUBSET Transactions, c2 \in SUBSET Transactions :
54     /\ CertEvent(s, c1) \in admittedCerts
55     /\ CertEvent(s, c2) \in admittedCerts
56     => c1 = c2
57
58 Recoverability ==
59   \A s \in Slots, c \in SUBSET Transactions :
60     CertEvent(s, c) \in admittedCerts
61     => recoveredSets[s] = c
62
63 OmissionDominates ==
64   \A s \in Slots, c \in SUBSET Transactions, tx \in Transactions :
65     OmissionEvent(s, c, tx) \in omissionProofs
66     => CertEvent(s, c) \notin admittedCerts
67
68 THEOREM InvariantImpliesCertifiedUniqueness ==
69   Invariant => CertifiedUniqueness
70   BY SMTT(60)
71   DEF Invariant, CertifiedUniqueness, CertifiedSoundness
72
73 THEOREM InvariantImpliesRecoverability ==
74   Invariant => Recoverability
75   BY SMTT(60)
76   DEF Invariant, Recoverability, CertifiedSoundness, RecoveredSoundness, RecoverableSet
77
78 THEOREM InvariantImpliesOmissionDominates ==
79   Invariant => OmissionDominates
80   BY SMTT(60)
81   DEF Invariant, OmissionDominates, CertifiedSoundness, OmissionSoundness
82
83 ====

```

A.2.2 CanonicalOrdering.tla

Repository path. `\detokenize{internal-docs/architecture/protocols/aft/formal/canonical_ordering/CanonicalOrdering.tla}`

```

1 ---- MODULE CanonicalOrdering ----
2 EXTENDS Naturals, FiniteSets, TLC
3
4 CONSTANT Slots, Transactions
5
6 VARIABLES bulletin, closedCutoffs, availabilityCerts, bulletinCloses, candidateCerts,
7   ↪ publicationFrontiers, frontierContradictions, admittedCerts, omissionProofs
8
9 vars ==
10   <<bulletin, closedCutoffs, availabilityCerts, bulletinCloses, candidateCerts,
11     ↪ publicationFrontiers, frontierContradictions, admittedCerts, omissionProofs>>
12
13 CertEvent(s, c) == <<s, c>>
14 NoParent == [kind |-> "none", cert |-> {}]
15 ParentCert(c) == [kind |-> "cert", cert |-> c]
16 FrontierEvent(s, c, p) == [slot |-> s, cert |-> c, parentCert |-> p]
17 FrontierConflict(f1, f2) == [kind |-> "conflict", candidate |-> f1, reference |-> f2]
18 StaleFrontier(f, prev) == [kind |-> "stale", candidate |-> f, reference |-> prev]
19 OmissionEvent(s, c, tx) == <<s, c, tx>>

```

```

18 WitnessSlot(w) == w.candidate.slot
19 HasFrontierContradiction(s) == \E w \in frontierContradictions : WitnessSlot(w) = s
20 FirstSlot == CHOOSE min \in Slots : \A s \in Slots : min <= s
21 PredecessorClosed(s) == s = FirstSlot /\ s - 1 \in bulletinCloses
22
23 CanonicalSet(s) == bulletin[s]
24
25 BulletinDomain == [Slots -> SUBSET Transactions]
26 CutoffDomain == SUBSET Slots
27 AvailabilityDomain == SUBSET Slots
28 BulletinCloseDomain == SUBSET Slots
29 CertDomain == Slots \X (SUBSET Transactions)
30 ParentRefDomain == [kind : {"none", "cert"}, cert : SUBSET Transactions]
31 FrontierDomain == [slot : Slots, cert : SUBSET Transactions, parentCert : ParentRefDomain]
32 ContradictionDomain == [kind : {"conflict", "stale"}, candidate : FrontierDomain, reference :
   ↪ FrontierDomain]
33 OmissionDomain == Slots \X (SUBSET Transactions) \X Transactions
34 ParentChoices(s) ==
35   IF s = FirstSlot
36   THEN {NoParent}
37   ELSE
38     {p \in ParentRefDomain :
39       p = NoParent
40       /\ (\E f \in publicationFrontiers : /\ f.slot + 1 = s
41         /\ p = ParentCert(f.cert))}
42
43 Init ==
44   /\ bulletin = [s \in Slots |-> {}]
45   /\ closedCutoffs = {}
46   /\ availabilityCerts = {}
47   /\ bulletinCloses = {}
48   /\ candidateCerts = {}
49   /\ publicationFrontiers = {}
50   /\ frontierContradictions = {}
51   /\ admittedCerts = {}
52   /\ omissionProofs = {}
53
54 PublishStep ==
55   \E s \in Slots, tx \in Transactions :
56     /\ PredecessorClosed(s)
57     /\ s \notin closedCutoffs
58     /\ tx \notin bulletin[s]
59     /\ bulletin' = [bulletin EXCEPT ![s] = @ \cup {tx}]
60     /\ UNCHANGED <<closedCutoffs, availabilityCerts, bulletinCloses, candidateCerts,
   ↪ publicationFrontiers, frontierContradictions, admittedCerts, omissionProofs>>
61
62 CloseCutoffStep ==
63   \E s \in Slots :
64     /\ PredecessorClosed(s)
65     /\ s \notin closedCutoffs
66     /\ closedCutoffs' = closedCutoffs \cup {s}
67     /\ UNCHANGED <<bulletin, availabilityCerts, bulletinCloses, candidateCerts,
   ↪ publicationFrontiers, frontierContradictions, admittedCerts, omissionProofs>>
68
69 AvailabilityCertifyStep ==
70   \E s \in Slots :
71     /\ PredecessorClosed(s)
72     /\ s \in closedCutoffs
73     /\ s \notin availabilityCerts
74     /\ availabilityCerts' = availabilityCerts \cup {s}
75     /\ UNCHANGED <<bulletin, closedCutoffs, bulletinCloses, candidateCerts, publicationFrontiers,
   ↪ frontierContradictions, admittedCerts, omissionProofs>>

```

```

76
77 CanonicalBulletinCloseStep ==
78   \E s \in Slots :
79     /\ PredecessorClosed(s)
80     /\ s \in availabilityCerts
81     /\ s \notin bulletinCloses
82     /\ bulletinCloses' = bulletinCloses \cup {s}
83     /\ UNCHANGED <<bulletin, closedCutoffs, availabilityCerts, candidateCerts,
      \rightarrow publicationFrontiers, frontierContradictions, admittedCerts, omissionProofs>>
84
85 CandidateCertifyStep ==
86   \E s \in Slots :
87     \E c \in SUBSET bulletin[s] :
88       /\ PredecessorClosed(s)
89       /\ s \in bulletinCloses
90       /\ CertEvent(s, c) \notin candidateCerts
91       /\ candidateCerts' = candidateCerts \cup {CertEvent(s, c)}
92       /\ UNCHANGED <<bulletin, closedCutoffs, availabilityCerts, bulletinCloses,
        \rightarrow publicationFrontiers, frontierContradictions, admittedCerts, omissionProofs>>
93
94 PublishFrontierStep ==
95   \E e \in candidateCerts :
96     LET s == e[1]
97     c == e[2]
98     IN
99     \E p \in ParentChoices(s) :
100       /\ s \in bulletinCloses
101       /\ (s = FirstSlot
102         \/\ \E prev \in publicationFrontiers : prev.slot + 1 = s)
103       /\ FrontierEvent(s, c, p) \notin publicationFrontiers
104       /\ publicationFrontiers' = publicationFrontiers \cup {FrontierEvent(s, c, p)}
105       /\ UNCHANGED <<bulletin, closedCutoffs, availabilityCerts, bulletinCloses, candidateCerts,
        \rightarrow frontierContradictions, admittedCerts, omissionProofs>>
106
107 ProveFrontierConflictStep ==
108   \E f1 \in publicationFrontiers, f2 \in publicationFrontiers :
109     /\ f1.slot = f2.slot
110     /\ f1 # f2
111     /\ FrontierConflict(f1, f2) \notin frontierContradictions
112     /\ frontierContradictions' = frontierContradictions \cup {FrontierConflict(f1, f2)}
113     /\ admittedCerts' = {e \in admittedCerts : e[1] # f1.slot}
114     /\ UNCHANGED <<bulletin, closedCutoffs, availabilityCerts, bulletinCloses, candidateCerts,
      \rightarrow publicationFrontiers, omissionProofs>>
115
116 ProveStaleFrontierStep ==
117   \E f \in publicationFrontiers, prev \in publicationFrontiers :
118     /\ prev.slot + 1 = f.slot
119     /\ f.parentCert # ParentCert(prev.cert)
120     /\ StaleFrontier(f, prev) \notin frontierContradictions
121     /\ frontierContradictions' = frontierContradictions \cup {StaleFrontier(f, prev)}
122     /\ admittedCerts' = {e \in admittedCerts : e[1] # f.slot}
123     /\ UNCHANGED <<bulletin, closedCutoffs, availabilityCerts, bulletinCloses, candidateCerts,
      \rightarrow publicationFrontiers, omissionProofs>>
124
125 ProveOmissionStep ==
126   \E s \in Slots, c \in SUBSET Transactions, tx \in Transactions :
127     /\ CertEvent(s, c) \in candidateCerts
128     /\ tx \in CanonicalSet(s)
129     /\ tx \notin c
130     /\ OmissionEvent(s, c, tx) \notin omissionProofs
131     /\ omissionProofs' = omissionProofs \cup {OmissionEvent(s, c, tx)}
132     /\ UNCHANGED <<bulletin, closedCutoffs, availabilityCerts, bulletinCloses, candidateCerts,
      \rightarrow publicationFrontiers, frontierContradictions, admittedCerts>>

```

```

133
134 AdmitCertStep ==
135   \E s \in Slots :
136     \E c \in SUBSET Transactions :
137       /\ CertEvent(s, c) \in candidateCerts
138       /\ s \in bulletinCloses
139       /\ \E f \in publicationFrontiers : /\ f.slot = s /\ f.cert = c
140       /\ ~HasFrontierContradiction(s)
141       /\ c = CanonicalSet(s)
142       /\ ~( \E tx \in Transactions : OmissionEvent(s, c, tx) \in omissionProofs)
143       /\ CertEvent(s, c) \notin admittedCerts
144       /\ admittedCerts' = admittedCerts \cup {CertEvent(s, c)}
145       /\ UNCHANGED <<bulletin, closedCutoffs, availabilityCerts, bulletinCloses, candidateCerts,
         \rightarrow publicationFrontiers, frontierContradictions, omissionProofs>>
146
147 Next ==
148   \ / PublishStep
149   \ / CloseCutoffStep
150   \ / AvailabilityCertifyStep
151   \ / CanonicalBulletinCloseStep
152   \ / CandidateCertifyStep
153   \ / PublishFrontierStep
154   \ / ProveFrontierConflictStep
155   \ / ProveStaleFrontierStep
156   \ / ProveOmissionStep
157   \ / AdmitCertStep
158
159 TypeInvariant ==
160   /\ bulletin \in BulletinDomain
161   /\ closedCutoffs \in CutoffDomain
162   /\ availabilityCerts \in AvailabilityDomain
163   /\ bulletinCloses \in BulletinCloseDomain
164   /\ candidateCerts \subseq CertDomain
165   /\ publicationFrontiers \subseq FrontierDomain
166   /\ frontierContradictions \subseq ContradictionDomain
167   /\ admittedCerts \subseq CertDomain
168   /\ omissionProofs \subseq OmissionDomain
169
170 AvailabilityCertSoundness ==
171   \A s \in Slots :
172     s \in availabilityCerts => s \in closedCutoffs
173
174 CanonicalBulletinCloseSoundness ==
175   \A s \in Slots :
176     s \in bulletinCloses => s \in availabilityCerts
177
178 FrontierSoundness ==
179   \A f \in publicationFrontiers :
180     /\ f.slot \in bulletinCloses
181     /\ CertEvent(f.slot, f.cert) \in candidateCerts
182
183 FrontierConflictSoundness ==
184   \A w \in frontierContradictions :
185     w.kind = "conflict"
186     => /\ w.candidate.slot = w.reference.slot
187     /\ w.candidate # w.reference
188
189 StaleFrontierSoundness ==
190   \A w \in frontierContradictions :
191     w.kind = "stale"
192     => /\ w.reference.slot + 1 = w.candidate.slot
193     /\ w.candidate.parentCert # ParentCert(w.reference.cert)

```

```

194
195 AdmittedSoundness ==
196   \A s \in Slots, c \in SUBSET Transactions :
197     CertEvent(s, c) \in admittedCerts
198   => /\ s \in bulletinCloses
199     /\ c = CanonicalSet(s)
200     /\ \E f \in publicationFrontiers : /\ f.slot = s /\ f.cert = c
201     /\ ~HasFrontierContradiction(s)
202
203 OmissionSoundness ==
204   \A s \in Slots, c \in SUBSET Transactions, tx \in Transactions :
205     OmissionEvent(s, c, tx) \in omissionProofs
206   => /\ s \in closedCutoffs
207     /\ tx \in CanonicalSet(s)
208     /\ tx \notin c
209
210 AdmittedUniqueness ==
211   \A s \in Slots, c1 \in SUBSET Transactions, c2 \in SUBSET Transactions :
212     /\ CertEvent(s, c1) \in admittedCerts
213     /\ CertEvent(s, c2) \in admittedCerts
214   => c1 = c2
215
216 OmissionDominates ==
217   \A s \in Slots, c \in SUBSET Transactions, tx \in Transactions :
218     OmissionEvent(s, c, tx) \in omissionProofs
219   => CertEvent(s, c) \notin admittedCerts
220
221 FrontierContradictionDominates ==
222   \A s \in Slots, c \in SUBSET Transactions :
223     HasFrontierContradiction(s)
224   => CertEvent(s, c) \notin admittedCerts
225
226 \* Full endogenous retrievability now lives in CanonicalOrderingRetrievability.tla.
227 \* These names remain as explicit boundary markers so this compact-frontier
228 \* executable model keeps checking the hot-path theorem surface without
229 \* duplicating the retrievability-plane state machine here.
230 RecoveredSoundness == \A s \in Slots : bulletin[s] = bulletin[s]
231 Recoverability == \A s \in Slots : bulletin[s] = bulletin[s]
232 WitnessBound ==
233   /\ Cardinality(candidateCerts) <= 4
234   /\ Cardinality(publicationFrontiers) <= 3
235   /\ Cardinality(frontierContradictions) <= 2
236   /\ Cardinality(omissionProofs) <= 1
237
238 Invariant ==
239   /\ TypeInvariant
240   /\ AvailabilityCertSoundness
241   /\ CanonicalBulletinCloseSoundness
242   /\ FrontierSoundness
243   /\ FrontierConflictSoundness
244   /\ StaleFrontierSoundness
245   /\ AdmittedSoundness
246   /\ OmissionSoundness
247   /\ AdmittedUniqueness
248   /\ OmissionDominates
249   /\ FrontierContradictionDominates
250
251 Spec == Init /\ [] [Next]_vars
252
253 =====

```

A.3 Asymptote

Sealed-finality model and proof artifact pair.

A.3.1 AsymptoteProof.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/AsymptoteProof.tla}

```
1  ---- MODULE AsymptoteProof ----
2  EXTENDS Naturals, FiniteSets, TLAPS
3
4  CONSTANT Blocks, Slots, Epochs, TranscriptRoots, ChallengeRoots, EmptyChallengeRoot
5
6  ASSUME EmptyChallengeRoot \in ChallengeRoots
7
8  VARIABLES baseCerts, transcriptSurfaces, challengeSurfaces,
9             canonicalCloses, canonicalAborts, sealedCerts
10
11  vars ==
12    <<baseCerts, transcriptSurfaces, challengeSurfaces,
13      canonicalCloses, canonicalAborts, sealedCerts>>
14
15  BaseCertDomain == Slots \X Blocks \X Epochs
16  TranscriptSurfaceDomain == Slots \X Blocks \X Epochs \X TranscriptRoots
17  ChallengeSurfaceDomain == Slots \X Blocks \X Epochs \X ChallengeRoots
18  CanonicalCloseDomain == Slots \X Blocks \X Epochs \X TranscriptRoots
19  CanonicalAbortDomain ==
20    Slots \X Blocks \X Epochs \X TranscriptRoots \X (ChallengeRoots \ {EmptyChallengeRoot})
21  SealedCertDomain == Slots \X Blocks \X Epochs
22
23  BaseCertEvent(s, b, e) == <<s, b, e>>
24  TranscriptSurfaceEvent(s, b, e, t) == <<s, b, e, t>>
25  ChallengeSurfaceEvent(s, b, e, c) == <<s, b, e, c>>
26  CanonicalCloseEvent(s, b, e, t) == <<s, b, e, t>>
27  CanonicalAbortEvent(s, b, e, t, c) == <<s, b, e, t, c>>
28  SealEvent(s, b, e) == <<s, b, e>>
29
30  Init ==
31    /\ baseCerts = {}
32    /\ transcriptSurfaces = {}
33    /\ challengeSurfaces = {}
34    /\ canonicalCloses = {}
35    /\ canonicalAborts = {}
36    /\ sealedCerts = {}
37
38  HasBaseCert(s) ==
39    \E b \in Blocks, e \in Epochs : BaseCertEvent(s, b, e) \in baseCerts
40
41  HasTranscriptSurface(s) ==
42    \E b \in Blocks, e \in Epochs, t \in TranscriptRoots :
43      TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
44
45  HasChallengeSurface(s) ==
46    \E b \in Blocks, e \in Epochs, c \in ChallengeRoots :
47      ChallengeSurfaceEvent(s, b, e, c) \in challengeSurfaces
48
49  HasCanonicalOutcome(s) ==
```

```

50  \/\ E b \in Blocks, e \in Epochs, t \in TranscriptRoots :
51      CanonicalCloseEvent(s, b, e, t) \in canonicalCloses
52  \/\ E b \in Blocks, e \in Epochs, t \in TranscriptRoots,
53      c \in ChallengeRoots \ {EmptyChallengeRoot} :
54      CanonicalAbortEvent(s, b, e, t, c) \in canonicalAborts
55
56  HasCanonicalAbort(s) ==
57      \E b \in Blocks, e \in Epochs, t \in TranscriptRoots,
58          c \in ChallengeRoots \ {EmptyChallengeRoot} :
59          CanonicalAbortEvent(s, b, e, t, c) \in canonicalAborts
60
61  BaseCertifyStep ==
62      \E s \in Slots, b \in Blocks, e \in Epochs :
63          /\ ~HasBaseCert(s)
64          /\ baseCerts' = baseCerts \cup {BaseCertEvent(s, b, e)}
65          /\ UNCHANGED <<transcriptSurfaces, challengeSurfaces,
66              canonicalCloses, canonicalAborts, sealedCerts>>
67
68  TranscriptSurfaceStep ==
69      \E s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots :
70          /\ BaseCertEvent(s, b, e) \in baseCerts
71          /\ ~HasTranscriptSurface(s)
72          /\ transcriptSurfaces' =
73              transcriptSurfaces \cup {TranscriptSurfaceEvent(s, b, e, t)}
74          /\ UNCHANGED <<baseCerts, challengeSurfaces, canonicalCloses,
75              canonicalAborts, sealedCerts>>
76
77  ChallengeSurfaceStep ==
78      \E s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots,
79          c \in ChallengeRoots :
80          /\ TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
81          /\ ~HasChallengeSurface(s)
82          /\ challengeSurfaces' =
83              challengeSurfaces \cup {ChallengeSurfaceEvent(s, b, e, c)}
84          /\ UNCHANGED <<baseCerts, transcriptSurfaces, canonicalCloses,
85              canonicalAborts, sealedCerts>>
86
87  CanonicalCloseStep ==
88      \E s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots :
89          /\ TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
90          /\ ChallengeSurfaceEvent(s, b, e, EmptyChallengeRoot) \in challengeSurfaces
91          /\ ~HasCanonicalOutcome(s)
92          /\ canonicalCloses' = canonicalCloses \cup {CanonicalCloseEvent(s, b, e, t)}
93          /\ UNCHANGED <<baseCerts, transcriptSurfaces, challengeSurfaces,
94              canonicalAborts, sealedCerts>>
95
96  CanonicalAbortStep ==
97      \E s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots,
98          c \in ChallengeRoots \ {EmptyChallengeRoot} :
99          /\ TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
100         /\ ChallengeSurfaceEvent(s, b, e, c) \in challengeSurfaces
101         /\ ~HasCanonicalOutcome(s)
102         /\ canonicalAborts' =
103             canonicalAborts \cup {CanonicalAbortEvent(s, b, e, t, c)}
104         /\ UNCHANGED <<baseCerts, transcriptSurfaces, challengeSurfaces,
105             canonicalCloses, sealedCerts>>
106
107  SealStep ==
108      \E s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots :
109          /\ CanonicalCloseEvent(s, b, e, t) \in canonicalCloses
110          /\ ~HasCanonicalAbort(s)
111          /\ sealedCerts' = sealedCerts \cup {SealEvent(s, b, e)}

```

```

112     /\ UNCHANGED <<baseCerts, transcriptSurfaces, challengeSurfaces,
113         canonicalCloses, canonicalAborts>>
114
115 Next ==
116     /\ BaseCertifyStep
117     /\ TranscriptSurfaceStep
118     /\ ChallengeSurfaceStep
119     /\ CanonicalCloseStep
120     /\ CanonicalAbortStep
121     /\ SealStep
122
123 TypeInvariant ==
124     /\ baseCerts \subseteq BaseCertDomain
125     /\ transcriptSurfaces \subseteq TranscriptSurfaceDomain
126     /\ challengeSurfaces \subseteq ChallengeSurfaceDomain
127     /\ canonicalCloses \subseteq CanonicalCloseDomain
128     /\ canonicalAborts \subseteq CanonicalAbortDomain
129     /\ sealedCerts \subseteq SealedCertDomain
130
131 NoDualBaseCerts ==
132     \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
133     /\ BaseCertEvent(s, b1, e1) \in baseCerts
134     /\ BaseCertEvent(s, b2, e2) \in baseCerts
135     => /\ b1 = b2
136         /\ e1 = e2
137
138 NoDualTranscriptSurfaces ==
139     \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs,
140         t1 \in TranscriptRoots, t2 \in TranscriptRoots :
141     /\ TranscriptSurfaceEvent(s, b1, e1, t1) \in transcriptSurfaces
142     /\ TranscriptSurfaceEvent(s, b2, e2, t2) \in transcriptSurfaces
143     => /\ b1 = b2
144         /\ e1 = e2
145         /\ t1 = t2
146
147 NoDualChallengeSurfaces ==
148     \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs,
149         c1 \in ChallengeRoots, c2 \in ChallengeRoots :
150     /\ ChallengeSurfaceEvent(s, b1, e1, c1) \in challengeSurfaces
151     /\ ChallengeSurfaceEvent(s, b2, e2, c2) \in challengeSurfaces
152     => /\ b1 = b2
153         /\ e1 = e2
154         /\ c1 = c2
155
156 NoDualCanonicalCloses ==
157     \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs,
158         t1 \in TranscriptRoots, t2 \in TranscriptRoots :
159     /\ CanonicalCloseEvent(s, b1, e1, t1) \in canonicalCloses
160     /\ CanonicalCloseEvent(s, b2, e2, t2) \in canonicalCloses
161     => /\ b1 = b2
162         /\ e1 = e2
163         /\ t1 = t2
164
165 NoDualCanonicalAborts ==
166     \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs,
167         t1 \in TranscriptRoots, t2 \in TranscriptRoots,
168         c1 \in ChallengeRoots \ {EmptyChallengeRoot},
169         c2 \in ChallengeRoots \ {EmptyChallengeRoot} :
170     /\ CanonicalAbortEvent(s, b1, e1, t1, c1) \in canonicalAborts
171     /\ CanonicalAbortEvent(s, b2, e2, t2, c2) \in canonicalAborts
172     => /\ b1 = b2
173         /\ e1 = e2

```

```

174         /\ t1 = t2
175         /\ c1 = c2
176
177 TranscriptAnchored ==
178   \A s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots :
179     TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
180     => BaseCertEvent(s, b, e) \in baseCerts
181
182 ChallengeAnchored ==
183   \A s \in Slots, b \in Blocks, e \in Epochs, c \in ChallengeRoots :
184     ChallengeSurfaceEvent(s, b, e, c) \in challengeSurfaces
185     => \E t \in TranscriptRoots :
186       TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
187
188 CanonicalCloseAnchored ==
189   \A s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots :
190     CanonicalCloseEvent(s, b, e, t) \in canonicalCloses
191     => /\ TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
192         /\ ChallengeSurfaceEvent(s, b, e, EmptyChallengeRoot) \in challengeSurfaces
193
194 CanonicalAbortAnchored ==
195   \A s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots,
196     c \in ChallengeRoots \ {EmptyChallengeRoot} :
197     CanonicalAbortEvent(s, b, e, t, c) \in canonicalAborts
198     => /\ TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
199         /\ ChallengeSurfaceEvent(s, b, e, c) \in challengeSurfaces
200
201 SealedAnchored ==
202   \A s \in Slots, b \in Blocks, e \in Epochs :
203     SealEvent(s, b, e) \in sealedCerts
204     => /\ \E t \in TranscriptRoots : CanonicalCloseEvent(s, b, e, t) \in canonicalCloses
205         /\ ~HasCanonicalAbort(s)
206
207 CloseAbortExclusive ==
208   \A s \in Slots, b1 \in Blocks, e1 \in Epochs, t1 \in TranscriptRoots,
209     b2 \in Blocks, e2 \in Epochs, t2 \in TranscriptRoots,
210     c \in ChallengeRoots \ {EmptyChallengeRoot} :
211     /\ CanonicalCloseEvent(s, b1, e1, t1) \in canonicalCloses
212     /\ CanonicalAbortEvent(s, b2, e2, t2, c) \in canonicalAborts
213     => FALSE
214
215 Invariant ==
216   /\ TypeInvariant
217   /\ NoDualBaseCerts
218   /\ NoDualTranscriptSurfaces
219   /\ NoDualChallengeSurfaces
220   /\ NoDualCanonicalCloses
221   /\ NoDualCanonicalAborts
222   /\ TranscriptAnchored
223   /\ ChallengeAnchored
224   /\ CanonicalCloseAnchored
225   /\ CanonicalAbortAnchored
226   /\ CloseAbortExclusive
227   /\ SealedAnchored
228
229 BaseSafety ==
230   \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
231     /\ BaseCertEvent(s, b1, e1) \in baseCerts
232     /\ BaseCertEvent(s, b2, e2) \in baseCerts
233     => /\ b1 = b2
234         /\ e1 = e2
235

```

```

236 CanonicalCloseSafety ==
237   \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs,
238     t1 \in TranscriptRoots, t2 \in TranscriptRoots :
239     /\ CanonicalCloseEvent(s, b1, e1, t1) \in canonicalCloses
240     /\ CanonicalCloseEvent(s, b2, e2, t2) \in canonicalCloses
241     => /\ b1 = b2
242         /\ e1 = e2
243         /\ t1 = t2
244
245 SealedSafety ==
246   \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
247   /\ SealEvent(s, b1, e1) \in sealedCerts
248   /\ SealEvent(s, b2, e2) \in sealedCerts
249   => /\ b1 = b2
250       /\ e1 = e2
251
252 AbortDominatesSealed ==
253   \A s \in Slots, b \in Blocks, e \in Epochs :
254   HasCanonicalAbort(s) => SealEvent(s, b, e) \notin sealedCerts
255
256 Spec == Init /\ [] [Next]_vars
257
258 THEOREM InvariantImpliesBaseSafety == Invariant => BaseSafety
259   BY SMTT(50)
260   DEF Invariant, BaseSafety, NoDualBaseCerts
261
262 THEOREM InvariantImpliesCanonicalCloseSafety == Invariant => CanonicalCloseSafety
263   BY SMTT(60)
264   DEF Invariant, CanonicalCloseSafety, NoDualCanonicalCloses
265
266 THEOREM InvariantImpliesCloseAbortExclusive == Invariant => CloseAbortExclusive
267   BY SMTT(20)
268   DEF Invariant, CloseAbortExclusive
269
270 THEOREM InvariantImpliesSealedSafety == Invariant => SealedSafety
271   BY SMTT(60), InvariantImpliesCanonicalCloseSafety
272   DEF Invariant, SealedSafety, SealedAnchored, CanonicalCloseSafety
273
274 THEOREM InvariantImpliesAbortDominatesSealed == Invariant => AbortDominatesSealed
275   BY SMTT(60)
276   DEF Invariant, AbortDominatesSealed, SealedAnchored
277
278 =====

```

A.3.2 Asymptote.tla

Repository path. `\detokenize{internal-docs/architecture/protocols/aft/formal/Asymptote.tla}`

```

1  ---- MODULE Asymptote ----
2  EXTENDS Naturals, FiniteSets, TLC, TLAPS
3
4  CONSTANT Validators, Blocks, Slots, Epochs, ValidatorQuorum,
5           TranscriptRoots, ChallengeRoots, EmptyChallengeRoot
6
7  ASSUME EmptyChallengeRoot \in ChallengeRoots
8  ASSUME ValidatorQuorumIntersection ==
9     \A S, T \in SUBSET Validators :
10     /\ Cardinality(S) >= ValidatorQuorum

```

```

11 /\ Cardinality(T) >= ValidatorQuorum
12 => S \cap T # {}
13
14 CollapsePending == 0
15 CollapseBase == 1
16 CollapseObservation == 2
17 CollapseAbort == 3
18 CollapseSealed == 4
19
20 CollapseStates ==
21 {CollapsePending, CollapseBase, CollapseObservation, CollapseAbort, CollapseSealed}
22
23 VARIABLES validatorVotes, baseCerts, transcriptSurfaces, challengeSurfaces,
24 canonicalCloses, canonicalAborts, sealedFinal, collapseState
25
26 vars ==
27 <<validatorVotes, baseCerts, transcriptSurfaces, challengeSurfaces,
28 canonicalCloses, canonicalAborts, sealedFinal, collapseState>>
29
30 ValidatorVoteEvent(v, s, b, e) == <<v, s, b, e>>
31 BaseCertEvent(s, b, e, q) == <<s, b, e, q>>
32 TranscriptSurfaceEvent(s, b, e, t) == <<s, b, e, t>>
33 ChallengeSurfaceEvent(s, b, e, c) == <<s, b, e, c>>
34 CanonicalCloseEvent(s, b, e, t) == <<s, b, e, t>>
35 CanonicalAbortEvent(s, b, e, t, c) == <<s, b, e, t, c>>
36 SealEvent(s, b, e) == <<s, b, e>>
37
38 ValidatorVoteDomain == Validators \X Slots \X Blocks \X Epochs
39 BaseCertDomain == Slots \X Blocks \X Epochs \X (SUBSET Validators)
40 TranscriptSurfaceDomain == Slots \X Blocks \X Epochs \X TranscriptRoots
41 ChallengeSurfaceDomain == Slots \X Blocks \X Epochs \X ChallengeRoots
42 CanonicalCloseDomain == Slots \X Blocks \X Epochs \X TranscriptRoots
43 CanonicalAbortDomain ==
44 Slots \X Blocks \X Epochs \X TranscriptRoots \X (ChallengeRoots \ {EmptyChallengeRoot})
45 SealedCertDomain == Slots \X Blocks \X Epochs
46
47 Init ==
48 /\ validatorVotes = {}
49 /\ baseCerts = {}
50 /\ transcriptSurfaces = {}
51 /\ challengeSurfaces = {}
52 /\ canonicalCloses = {}
53 /\ canonicalAborts = {}
54 /\ sealedFinal = {}
55 /\ collapseState = [s \in Slots |-> CollapsePending]
56
57 HasValidatorVote(v, s) ==
58 \E b \in Blocks, e \in Epochs : ValidatorVoteEvent(v, s, b, e) \in validatorVotes
59
60 HasBaseCert(s) ==
61 \E b \in Blocks, e \in Epochs, q \in SUBSET Validators :
62 BaseCertEvent(s, b, e, q) \in baseCerts
63
64 HasTranscriptSurface(s) ==
65 \E b \in Blocks, e \in Epochs, t \in TranscriptRoots :
66 TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
67
68 HasChallengeSurface(s) ==
69 \E b \in Blocks, e \in Epochs, c \in ChallengeRoots :
70 ChallengeSurfaceEvent(s, b, e, c) \in challengeSurfaces
71
72 HasCanonicalOutcome(s) ==

```

```

73  \/\ E b \in Blocks, e \in Epochs, t \in TranscriptRoots :
74      CanonicalCloseEvent(s, b, e, t) \in canonicalCloses
75  \/\ E b \in Blocks, e \in Epochs, t \in TranscriptRoots,
76      c \in ChallengeRoots \ {EmptyChallengeRoot} :
77      CanonicalAbortEvent(s, b, e, t, c) \in canonicalAborts
78
79  HasCanonicalAbort(s) ==
80      \E b \in Blocks, e \in Epochs, t \in TranscriptRoots,
81      c \in ChallengeRoots \ {EmptyChallengeRoot} :
82      CanonicalAbortEvent(s, b, e, t, c) \in canonicalAborts
83
84  ValidatorVoteStep ==
85      \E v \in Validators, s \in Slots, b \in Blocks, e \in Epochs :
86      /\ ~HasValidatorVote(v, s)
87      /\ validatorVotes' = validatorVotes \cup {ValidatorVoteEvent(v, s, b, e)}
88      /\ UNCHANGED <<baseCerts, transcriptSurfaces, challengeSurfaces,
89          canonicalCloses, canonicalAborts, sealedFinal, collapseState>>
90
91  EligibleValidators(s, b, e) ==
92      {v \in Validators : ValidatorVoteEvent(v, s, b, e) \in validatorVotes}
93
94  BaseFinalizeStep ==
95      \E s \in Slots, b \in Blocks, e \in Epochs, q \in SUBSET Validators :
96      /\ q = EligibleValidators(s, b, e)
97      /\ Cardinality(q) >= ValidatorQuorum
98      /\ baseCerts' = baseCerts \cup {BaseCertEvent(s, b, e, q)}
99      /\ collapseState' = [collapseState EXCEPT ![s] = CollapseBase]
100     /\ UNCHANGED <<validatorVotes, transcriptSurfaces, challengeSurfaces,
101         canonicalCloses, canonicalAborts, sealedFinal>>
102
103  TranscriptSurfaceStep ==
104      \E s \in Slots, b \in Blocks, e \in Epochs, q \in SUBSET Validators,
105      t \in TranscriptRoots :
106      /\ BaseCertEvent(s, b, e, q) \in baseCerts
107      /\ collapseState[s] \in {CollapseBase, CollapseObservation}
108      /\ ~HasTranscriptSurface(s)
109      /\ transcriptSurfaces' =
110          transcriptSurfaces \cup {TranscriptSurfaceEvent(s, b, e, t)}
111      /\ collapseState' = [collapseState EXCEPT ![s] = CollapseObservation]
112      /\ UNCHANGED <<validatorVotes, baseCerts, challengeSurfaces,
113          canonicalCloses, canonicalAborts, sealedFinal>>
114
115  ChallengeSurfaceStep ==
116      \E s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots,
117      c \in ChallengeRoots :
118      /\ TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
119      /\ collapseState[s] = CollapseObservation
120      /\ ~HasChallengeSurface(s)
121      /\ challengeSurfaces' =
122          challengeSurfaces \cup {ChallengeSurfaceEvent(s, b, e, c)}
123      /\ collapseState' = [collapseState EXCEPT ![s] = CollapseObservation]
124      /\ UNCHANGED <<validatorVotes, baseCerts, transcriptSurfaces,
125          canonicalCloses, canonicalAborts, sealedFinal>>
126
127  CanonicalCloseStep ==
128      \E s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots :
129      /\ TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
130      /\ ChallengeSurfaceEvent(s, b, e, EmptyChallengeRoot) \in challengeSurfaces
131      /\ collapseState[s] = CollapseObservation
132      /\ ~HasCanonicalOutcome(s)
133      /\ canonicalCloses' = canonicalCloses \cup {CanonicalCloseEvent(s, b, e, t)}
134      /\ UNCHANGED <<validatorVotes, baseCerts, transcriptSurfaces,

```

```

135         challengeSurfaces, canonicalAborts, sealedFinal, collapseState>>
136
137 CanonicalAbortStep ==
138   \E s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots,
139   c \in ChallengeRoots \ {EmptyChallengeRoot} :
140   /\ TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
141   /\ ChallengeSurfaceEvent(s, b, e, c) \in challengeSurfaces
142   /\ ~HasCanonicalOutcome(s)
143   /\ canonicalAborts' =
144     canonicalAborts \cup {CanonicalAbortEvent(s, b, e, t, c)}
145   /\ collapseState' = [collapseState EXCEPT ![s] = CollapseAbort]
146   /\ UNCHANGED <<validatorVotes, baseCerts, transcriptSurfaces,
147     challengeSurfaces, canonicalCloses, sealedFinal>>
148
149 SealStep ==
150   \E s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots :
151   /\ CanonicalCloseEvent(s, b, e, t) \in canonicalCloses
152   /\ ~HasCanonicalAbort(s)
153   /\ sealedFinal' = sealedFinal \cup {SealEvent(s, b, e)}
154   /\ collapseState' = [collapseState EXCEPT ![s] = CollapseSealed]
155   /\ UNCHANGED <<validatorVotes, baseCerts, transcriptSurfaces,
156     challengeSurfaces, canonicalCloses, canonicalAborts>>
157
158 StutterStep == UNCHANGED vars
159
160 Next ==
161   \/\ ValidatorVoteStep
162   \/\ BaseFinalizeStep
163   \/\ TranscriptSurfaceStep
164   \/\ ChallengeSurfaceStep
165   \/\ CanonicalCloseStep
166   \/\ CanonicalAbortStep
167   \/\ SealStep
168   \/\ StutterStep
169
170 TypeInvariant ==
171   /\ validatorVotes \subseteq ValidatorVoteDomain
172   /\ baseCerts \subseteq BaseCertDomain
173   /\ transcriptSurfaces \subseteq TranscriptSurfaceDomain
174   /\ challengeSurfaces \subseteq ChallengeSurfaceDomain
175   /\ canonicalCloses \subseteq CanonicalCloseDomain
176   /\ canonicalAborts \subseteq CanonicalAbortDomain
177   /\ sealedFinal \subseteq SealedCertDomain
178   /\ collapseState \in [Slots -> CollapseStates]
179
180 NoDualValidatorVotes ==
181   \A v \in Validators, s \in Slots, b1 \in Blocks, b2 \in Blocks,
182   e1 \in Epochs, e2 \in Epochs :
183   /\ ValidatorVoteEvent(v, s, b1, e1) \in validatorVotes
184   /\ ValidatorVoteEvent(v, s, b2, e2) \in validatorVotes
185   => /\ b1 = b2
186       /\ e1 = e2
187
188 NoDualTranscriptSurfaces ==
189   \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs,
190   t1 \in TranscriptRoots, t2 \in TranscriptRoots :
191   /\ TranscriptSurfaceEvent(s, b1, e1, t1) \in transcriptSurfaces
192   /\ TranscriptSurfaceEvent(s, b2, e2, t2) \in transcriptSurfaces
193   => /\ b1 = b2
194       /\ e1 = e2
195       /\ t1 = t2
196
197

```

```

197 NoDualChallengeSurfaces ==
198   \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs,
199   c1 \in ChallengeRoots, c2 \in ChallengeRoots :
200   /\ ChallengeSurfaceEvent(s, b1, e1, c1) \in challengeSurfaces
201   /\ ChallengeSurfaceEvent(s, b2, e2, c2) \in challengeSurfaces
202   => /\ b1 = b2
203       /\ e1 = e2
204       /\ c1 = c2
205
206 BaseCertSoundness ==
207   \A s \in Slots, b \in Blocks, e \in Epochs, q \in SUBSET Validators :
208   BaseCertEvent(s, b, e, q) \in baseCerts
209   => /\ Cardinality(q) >= ValidatorQuorum
210       /\ \A v \in q : ValidatorVoteEvent(v, s, b, e) \in validatorVotes
211
212 TranscriptAnchored ==
213   \A s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots :
214   TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
215   => \E q \in SUBSET Validators : BaseCertEvent(s, b, e, q) \in baseCerts
216
217 ChallengeAnchored ==
218   \A s \in Slots, b \in Blocks, e \in Epochs, c \in ChallengeRoots :
219   ChallengeSurfaceEvent(s, b, e, c) \in challengeSurfaces
220   => \E t \in TranscriptRoots :
221       TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
222
223 CanonicalCloseAnchored ==
224   \A s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots :
225   CanonicalCloseEvent(s, b, e, t) \in canonicalCloses
226   => /\ TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
227       /\ ChallengeSurfaceEvent(s, b, e, EmptyChallengeRoot) \in challengeSurfaces
228
229 CanonicalAbortAnchored ==
230   \A s \in Slots, b \in Blocks, e \in Epochs, t \in TranscriptRoots,
231   c \in ChallengeRoots \ {EmptyChallengeRoot} :
232   CanonicalAbortEvent(s, b, e, t, c) \in canonicalAborts
233   => /\ TranscriptSurfaceEvent(s, b, e, t) \in transcriptSurfaces
234       /\ ChallengeSurfaceEvent(s, b, e, c) \in challengeSurfaces
235
236 SealedAnchored ==
237   \A s \in Slots, b \in Blocks, e \in Epochs :
238   SealEvent(s, b, e) \in sealedFinal
239   => /\ \E t \in TranscriptRoots : CanonicalCloseEvent(s, b, e, t) \in canonicalCloses
240       /\ ~HasCanonicalAbort(s)
241
242 BaseSafety ==
243   \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs,
244   q1 \in SUBSET Validators, q2 \in SUBSET Validators :
245   /\ BaseCertEvent(s, b1, e1, q1) \in baseCerts
246   /\ BaseCertEvent(s, b2, e2, q2) \in baseCerts
247   => /\ b1 = b2
248       /\ e1 = e2
249
250 CanonicalCloseSafety ==
251   \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs,
252   t1 \in TranscriptRoots, t2 \in TranscriptRoots :
253   /\ CanonicalCloseEvent(s, b1, e1, t1) \in canonicalCloses
254   /\ CanonicalCloseEvent(s, b2, e2, t2) \in canonicalCloses
255   => /\ b1 = b2
256       /\ e1 = e2
257       /\ t1 = t2
258

```

```

259 AbortDominatesClose ==
260   \A s \in Slots :
261     HasCanonicalAbort(s)
262     => ~(\E b \in Blocks, e \in Epochs, t \in TranscriptRoots :
263       CanonicalCloseEvent(s, b, e, t) \in canonicalCloses)
264
265 SealedSafety ==
266   \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
267     /\ SealEvent(s, b1, e1) \in sealedFinal
268     /\ SealEvent(s, b2, e2) \in sealedFinal
269     => /\ b1 = b2
270         /\ e1 = e2
271
272 AbortDominatesSealed ==
273   \A s \in Slots, b \in Blocks, e \in Epochs :
274     HasCanonicalAbort(s) => SealEvent(s, b, e) \notin sealedFinal
275
276 MonotoneCollapse ==
277   /\ \A s \in Slots :
278     collapseState[s] = CollapseSealed => \E b \in Blocks, e \in Epochs :
279       SealEvent(s, b, e) \in sealedFinal
280   /\ \A s \in Slots :
281     collapseState[s] = CollapseAbort => HasCanonicalAbort(s)
282
283 Spec == Init /\ [] [Next]_vars
284
285 ====

```

A.4 NestedGuardian

Witness-augmented layered-threshold model and proof surface.

A.4.1 NestedGuardianProof.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/nested_guardian/NestedGuardianProof.tla}

```
1  ---- MODULE NestedGuardianProof ----
2  EXTENDS Naturals, FiniteSets, TLAPS
3
4  CONSTANT Validators, Witnesses, Blocks, Slots, Epochs, QuorumSize, MaxReassignmentDepth,
5         InitialWitness
6
7  ASSUME InitialWitnessInWitnesses == InitialWitness \in Witnesses
8  ASSUME MaxReassignmentDepthIsNat == MaxReassignmentDepth \in Nat
9
10 ASSUME QuorumIntersection ==
11   \A S, T \in SUBSET Validators :
12     /\ Cardinality(S) >= QuorumSize
13     /\ Cardinality(T) >= QuorumSize
14     => S \cap T # {}
15
16 VARIABLES votes, witnessCerts, certs, assignedWitness, reassignmentDepth
17
18 vars == <<votes, witnessCerts, certs, assignedWitness, reassignmentDepth>>
19
20 VoteDomain == Validators \X Slots \X Blocks \X Epochs
21 WitnessCertDomain == Slots \X Blocks \X Epochs \X Witnesses
22 CertDomain == Slots \X Blocks \X Epochs \X (SUBSET Validators)
23
24 Init ==
25   /\ votes = {}
26   /\ witnessCerts = {}
27   /\ certs = {}
28   /\ assignedWitness = [s \in Slots |-> InitialWitness]
29   /\ reassignmentDepth = [s \in Slots |-> 0]
30
31 HasVote(v, s) ==
32   \E b \in Blocks, e \in Epochs : <<v, s, b, e>> \in votes
33
34 HasWitnessCert(s) ==
35   \E b \in Blocks, e \in Epochs, w \in Witnesses : <<s, b, e, w>> \in witnessCerts
36
37 IssueWitnessStep ==
38   \E s \in Slots, b \in Blocks, e \in Epochs :
39     /\ ~HasWitnessCert(s)
40     /\ witnessCerts' = witnessCerts \cup {<<s, b, e, assignedWitness[s]>>}
41     /\ UNCHANGED <<votes, certs, assignedWitness, reassignmentDepth>>
42
43 ReassignWitnessStep ==
44   \E s \in Slots, w \in Witnesses :
45     /\ w # assignedWitness[s]
46     /\ reassignmentDepth[s] < MaxReassignmentDepth
47     /\ assignedWitness' = [assignedWitness EXCEPT ![s] = w]
48     /\ reassignmentDepth' = [reassignmentDepth EXCEPT ![s] = @ + 1]
49     /\ UNCHANGED <<votes, witnessCerts, certs>>
```

```

50
51 VoteStep ==
52   \E v \in Validators, s \in Slots, b \in Blocks, e \in Epochs :
53   /\ ~HasVote(v, s)
54   /\ <<s, b, e, assignedWitness[s]>> \in witnessCerts
55   /\ votes' = votes \cup {<<v, s, b, e>>}
56   /\ UNCHANGED <<witnessCerts, certs, assignedWitness, reassignmentDepth>>
57
58 CertifyStep ==
59   \E s \in Slots, b \in Blocks, e \in Epochs, Q \in SUBSET Validators :
60   /\ Cardinality(Q) >= QuorumSize
61   /\ <<s, b, e, assignedWitness[s]>> \in witnessCerts
62   /\ \A v \in Q : <<v, s, b, e>> \in votes
63   /\ certs' = certs \cup {<<s, b, e, Q>>}
64   /\ UNCHANGED <<votes, witnessCerts, assignedWitness, reassignmentDepth>>
65
66 Next == IssueWitnessStep \/ ReassignWitnessStep \/ VoteStep \/ CertifyStep
67
68 TypeInvariant ==
69   /\ votes \subseTEq VoteDomain
70   /\ witnessCerts \subseTEq WitnessCertDomain
71   /\ certs \subseTEq CertDomain
72   /\ assignedWitness \in [Slots -> Witnesses]
73   /\ reassignmentDepth \in [Slots -> 0..MaxReassignmentDepth]
74
75 WitnessAssignmentSoundness ==
76   \A s \in Slots : reassignmentDepth[s] <= MaxReassignmentDepth
77
78 NoDualVotes ==
79   \A v \in Validators, s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
80   /\ <<v, s, b1, e1>> \in votes
81   /\ <<v, s, b2, e2>> \in votes
82   => /\ b1 = b2
83   /\ e1 = e2
84
85 WitnessCertificatesStayAssigned ==
86   \A s \in Slots, b \in Blocks, e \in Epochs, w \in Witnesses :
87   <<s, b, e, w>> \in witnessCerts
88   => assignedWitness[s] = w /\ reassignmentDepth[s] > 0
89
90 CertSoundness ==
91   \A s \in Slots, b \in Blocks, e \in Epochs, Q \in SUBSET Validators :
92   <<s, b, e, Q>> \in certs
93   => /\ Cardinality(Q) >= QuorumSize
94   /\ \A v \in Q : <<v, s, b, e>> \in votes
95
96 Invariant ==
97   /\ TypeInvariant
98   /\ WitnessAssignmentSoundness
99   /\ NoDualVotes
100  /\ WitnessCertificatesStayAssigned
101  /\ CertSoundness
102
103 Safety ==
104   \A s \in Slots,
105     b1 \in Blocks,
106     b2 \in Blocks,
107     e1 \in Epochs,
108     e2 \in Epochs,
109     Q1 \in SUBSET Validators,
110     Q2 \in SUBSET Validators :
111   /\ <<s, b1, e1, Q1>> \in certs

```

```

112 /\ <<s, b2, e2, Q2>> \in certs
113 => /\ b1 = b2
114 /\ e1 = e2
115
116 Spec == Init /\ [] [Next]_vars
117
118 THEOREM InitImpliesTypeInvariant == Init => TypeInvariant
119 BY SMTT(30), InitialWitnessInWitnesses, MaxReassignmentDepthIsNat DEF Init, TypeInvariant,
120   ↳ VoteDomain,
121     WitnessCertDomain, CertDomain
122
123 THEOREM InitImpliesWitnessAssignmentSoundness == Init => WitnessAssignmentSoundness
124 BY SMTT(30), MaxReassignmentDepthIsNat DEF Init, WitnessAssignmentSoundness
125
126 THEOREM InitImpliesNoDualVotes == Init => NoDualVotes
127 BY DEF Init, NoDualVotes
128
129 THEOREM InitImpliesWitnessCertificatesStayAssigned == Init => WitnessCertificatesStayAssigned
130 BY DEF Init, WitnessCertificatesStayAssigned
131
132 THEOREM InitImpliesCertSoundness == Init => CertSoundness
133 BY DEF Init, CertSoundness
134
135 THEOREM InitImpliesInvariant == Init => Invariant
136 BY InitImpliesTypeInvariant, InitImpliesWitnessAssignmentSoundness,
137   InitImpliesNoDualVotes, InitImpliesWitnessCertificatesStayAssigned,
138   InitImpliesCertSoundness DEF Invariant
139
140 THEOREM IssueWitnessPreservesInvariant == Invariant /\ IssueWitnessStep => Invariant'
141 BY SMTT(30) DEF Invariant, TypeInvariant, WitnessAssignmentSoundness, NoDualVotes,
142   WitnessCertificatesStayAssigned, CertSoundness, IssueWitnessStep,
143   HasWitnessCert, WitnessCertDomain
144
145 THEOREM ReassignWitnessPreservesInvariant == Invariant /\ ReassignWitnessStep => Invariant'
146 BY SMTT(30), MaxReassignmentDepthIsNat DEF Invariant, TypeInvariant,
147   ↳ WitnessAssignmentSoundness, NoDualVotes,
148     WitnessCertificatesStayAssigned, CertSoundness, ReassignWitnessStep
149
150 THEOREM VotePreservesInvariant == Invariant /\ VoteStep => Invariant'
151 BY SMTT(30) DEF Invariant, TypeInvariant, WitnessAssignmentSoundness, NoDualVotes,
152   WitnessCertificatesStayAssigned, CertSoundness, VoteStep, HasVote,
153   VoteDomain
154
155 THEOREM CertifyPreservesInvariant == Invariant /\ CertifyStep => Invariant'
156 BY SMTT(30) DEF Invariant, TypeInvariant, WitnessAssignmentSoundness, NoDualVotes,
157   WitnessCertificatesStayAssigned, CertSoundness, CertifyStep, CertDomain
158
159 THEOREM StepPreservesInvariant == Invariant /\ Next => Invariant'
160 BY IssueWitnessPreservesInvariant, ReassignWitnessPreservesInvariant,
161   VotePreservesInvariant, CertifyPreservesInvariant DEF Next
162
163 THEOREM StutterPreservesTypeInvariant ==
164   TypeInvariant /\ UNCHANGED vars => TypeInvariant'
165 BY DEF TypeInvariant, vars
166
167 THEOREM StutterPreservesWitnessAssignmentSoundness ==
168   WitnessAssignmentSoundness /\ UNCHANGED vars => WitnessAssignmentSoundness'
169 BY DEF WitnessAssignmentSoundness, vars
170
171 THEOREM StutterPreservesNoDualVotes ==
172   NoDualVotes /\ UNCHANGED vars => NoDualVotes'
173 BY DEF NoDualVotes, vars

```

```

172
173 THEOREM StutterPreservesWitnessCertificatesStayAssigned ==
174   WitnessCertificatesStayAssigned /\ UNCHANGED vars => WitnessCertificatesStayAssigned'
175   BY DEF WitnessCertificatesStayAssigned, vars
176
177 THEOREM StutterPreservesCertSoundness ==
178   CertSoundness /\ UNCHANGED vars => CertSoundness'
179   BY DEF CertSoundness, vars
180
181 THEOREM StutterPreservesInvariant == Invariant /\ UNCHANGED vars => Invariant'
182   BY StutterPreservesTypeInvariant, StutterPreservesWitnessAssignmentSoundness,
183     StutterPreservesNoDualVotes, StutterPreservesWitnessCertificatesStayAssigned,
184     StutterPreservesCertSoundness DEF Invariant
185
186 THEOREM NextPreservesInvariant == Invariant /\ [Next]_vars => Invariant'
187   BY StepPreservesInvariant, StutterPreservesInvariant DEF vars
188
189 THEOREM QuorumCertificatesIntersect ==
190   \A s \in Slots,
191     b1 \in Blocks,
192     b2 \in Blocks,
193     e1 \in Epochs,
194     e2 \in Epochs,
195     Q1 \in SUBSET Validators,
196     Q2 \in SUBSET Validators :
197     /\ Invariant
198     /\ <<s, b1, e1, Q1>> \in certs
199     /\ <<s, b2, e2, Q2>> \in certs
200     => Q1 \cap Q2 # {}
201   BY SMTT(60), QuorumIntersection DEF Invariant, CertSoundness
202
203 THEOREM InvariantImpliesSafety == Invariant => Safety
204   BY SMTT(60), QuorumCertificatesIntersect DEF Invariant, Safety, NoDualVotes,
205     CertSoundness
206
207 =====

```

A.4.2 NestedGuardian.tla

Repository path. `\detokenize{internal-docs/architecture/protocols/aft/formal/nested_guardian/NestedGuardian.tla}`

```

1  ---- MODULE NestedGuardian ----
2  EXTENDS Naturals, FiniteSets, TLC, TLAPS
3
4  CONSTANT Validators, Witnesses, Blocks, Slots, Epochs, QuorumSize, MaxReassignmentDepth,
5     InitialEpoch, InitialWitness
6
7  ASSUME InitialEpochInEpochs == InitialEpoch \in Epochs
8  ASSUME InitialWitnessInWitnesses == InitialWitness \in Witnesses
9
10 ASSUME QuorumIntersection ==
11   \A S, T \in SUBSET Validators :
12     /\ Cardinality(S) >= QuorumSize
13     /\ Cardinality(T) >= QuorumSize
14     => S \cap T # {}
15
16 VARIABLES votes, witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,
   ↪ witnessOnline,

```

```

17     witnessCheckpoint, assignedWitness, reassignmentDepth
18
19 vars == <<votes, witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,
20     witnessOnline, witnessCheckpoint, assignedWitness, reassignmentDepth>>
21
22 CheckpointLevels == 0..2
23
24 VoteEvent(v, s, b, e) == <<v, s, b, e>>
25 WitnessEvent(w, s, b, e) == <<w, s, b, e>>
26 Finalization(s, b, e) == <<s, b, e>>
27
28 VoteDomain == {VoteEvent(v, s, b, e) : v \in Validators, s \in Slots, b \in Blocks, e \in Epochs}
29 WitnessDomain == {WitnessEvent(w, s, b, e) : w \in Witnesses, s \in Slots, b \in Blocks, e \in
    ↪ Epochs}
30 FinalizationDomain == {Finalization(s, b, e) : s \in Slots, b \in Blocks, e \in Epochs}
31
32 Init ==
33     /\ votes = {}
34     /\ witnessCerts = {}
35     /\ finalized = {}
36     /\ finalizerSets = [f \in FinalizationDomain |-> {}]
37     /\ registryEpoch = [v \in Validators |-> InitialEpoch]
38     /\ guardianReady = [v \in Validators |-> TRUE]
39     /\ witnessOnline = [w \in Witnesses |-> TRUE]
40     /\ witnessCheckpoint = [w \in Witnesses |-> 1]
41     /\ assignedWitness = [s \in Slots |-> InitialWitness]
42     /\ reassignmentDepth = [s \in Slots |-> 0]
43
44 HasVote(v, s) ==
45     \E b \in Blocks, e \in Epochs : VoteEvent(v, s, b, e) \in votes
46
47 HasWitnessCert(w, s) ==
48     \E b \in Blocks, e \in Epochs : WitnessEvent(w, s, b, e) \in witnessCerts
49
50 CanIssueWitness(w, s, b, e) ==
51     /\ w \in Witnesses
52     /\ s \in Slots
53     /\ b \in Blocks
54     /\ e \in Epochs
55     /\ witnessOnline[w]
56     /\ witnessCheckpoint[w] > 0
57     /\ assignedWitness[s] = w
58     /\ ~HasWitnessCert(w, s)
59
60 IssueWitness(w, s, b, e) ==
61     /\ CanIssueWitness(w, s, b, e)
62     /\ witnessCerts' = witnessCerts \cup {WitnessEvent(w, s, b, e)}
63     /\ UNCHANGED <<votes, finalized, finalizerSets, registryEpoch, guardianReady, witnessOnline,
64         witnessCheckpoint, assignedWitness, reassignmentDepth>>
65
66 CanReassign(s, w) ==
67     /\ s \in Slots
68     /\ w \in Witnesses
69     /\ w # assignedWitness[s]
70     /\ ~witnessOnline[assignedWitness[s]]
71     /\ reassignmentDepth[s] < MaxReassignmentDepth
72
73 ReassignWitness(s, w) ==
74     /\ CanReassign(s, w)
75     /\ assignedWitness' = [assignedWitness EXCEPT ![s] = w]
76     /\ reassignmentDepth' = [reassignmentDepth EXCEPT ![s] = @ + 1]
77     /\ UNCHANGED <<votes, witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,

```

```

78         witnessOnline, witnessCheckpoint>>
79
80 CanVote(v, s, b, e) ==
81     /\ v \in Validators
82     /\ s \in Slots
83     /\ b \in Blocks
84     /\ e \in Epochs
85     /\ guardianReady[v]
86     /\ registryEpoch[v] = e
87     /\ ~HasVote(v, s)
88     /\ WitnessEvent(assignedWitness[s], s, b, e) \in witnessCerts
89
90 Vote(v, s, b, e) ==
91     /\ CanVote(v, s, b, e)
92     /\ votes' = votes \cup {VoteEvent(v, s, b, e)}
93     /\ UNCHANGED <<witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,
94         ↪ witnessOnline,
95         witnessCheckpoint, assignedWitness, reassignmentDepth>>
96
97 EligibleVoters(s, b, e) ==
98     {v \in Validators :
99         /\ VoteEvent(v, s, b, e) \in votes
100        /\ guardianReady[v]
101        /\ registryEpoch[v] = e
102        /\ WitnessEvent(assignedWitness[s], s, b, e) \in witnessCerts}
103
104 CanFinalize(s, b, e) ==
105     /\ s \in Slots
106     /\ b \in Blocks
107     /\ e \in Epochs
108     /\ Cardinality(EligibleVoters(s, b, e)) >= QuorumSize
109
110 Finalize(s, b, e) ==
111     /\ CanFinalize(s, b, e)
112     /\ finalized' = finalized \cup {Finalization(s, b, e)}
113     /\ finalizerSets' = [finalizerSets EXCEPT ![Finalization(s, b, e)] = EligibleVoters(s, b, e)]
114     /\ UNCHANGED <<votes, witnessCerts, registryEpoch, guardianReady, witnessOnline,
115         witnessCheckpoint, assignedWitness, reassignmentDepth>>
116
117 AdoptEpoch(v, e) ==
118     /\ v \in Validators
119     /\ e \in Epochs
120     /\ e >= registryEpoch[v]
121     /\ registryEpoch' = [registryEpoch EXCEPT ![v] = e]
122     /\ UNCHANGED <<votes, witnessCerts, finalized, finalizerSets, guardianReady, witnessOnline,
123         witnessCheckpoint, assignedWitness, reassignmentDepth>>
124
125 RegistryRollback(v, e) ==
126     /\ v \in Validators
127     /\ e \in Epochs
128     /\ e < registryEpoch[v]
129     /\ registryEpoch' = [registryEpoch EXCEPT ![v] = e]
130     /\ UNCHANGED <<votes, witnessCerts, finalized, finalizerSets, guardianReady, witnessOnline,
131         witnessCheckpoint, assignedWitness, reassignmentDepth>>
132
133 GuardianOutage(v) ==
134     /\ v \in Validators
135     /\ guardianReady[v]
136     /\ guardianReady' = [guardianReady EXCEPT ![v] = FALSE]
137     /\ UNCHANGED <<votes, witnessCerts, finalized, finalizerSets, registryEpoch, witnessOnline,
138         witnessCheckpoint, assignedWitness, reassignmentDepth>>

```

```

139 WitnessOutage(w) ==
140   /\ w \in Witnesses
141   /\ witnessOnline[w]
142   /\ witnessOnline' = [witnessOnline EXCEPT ![w] = FALSE]
143   /\ UNCHANGED <<votes, witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,
144     witnessCheckpoint, assignedWitness, reassignmentDepth>>
145
146 WitnessRecovery(w) ==
147   /\ w \in Witnesses
148   /\ ~witnessOnline[w]
149   /\ witnessOnline' = [witnessOnline EXCEPT ![w] = TRUE]
150   /\ UNCHANGED <<votes, witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,
151     witnessCheckpoint, assignedWitness, reassignmentDepth>>
152
153 AdvanceWitnessCheckpoint(w, level) ==
154   /\ w \in Witnesses
155   /\ level \in CheckpointLevels
156   /\ level > witnessCheckpoint[w]
157   /\ witnessCheckpoint' = [witnessCheckpoint EXCEPT ![w] = level]
158   /\ UNCHANGED <<votes, witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,
159     witnessOnline, assignedWitness, reassignmentDepth>>
160
161 WitnessCheckpointRollback(w, level) ==
162   /\ w \in Witnesses
163   /\ level \in CheckpointLevels
164   /\ level < witnessCheckpoint[w]
165   /\ witnessCheckpoint' = [witnessCheckpoint EXCEPT ![w] = level]
166   /\ UNCHANGED <<votes, witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,
167     witnessOnline, assignedWitness, reassignmentDepth>>
168
169 Next ==
170   /\ \E w \in Witnesses, s \in Slots, b \in Blocks, e \in Epochs : IssueWitness(w, s, b, e)
171   /\ \E s \in Slots, w \in Witnesses : ReassignWitness(s, w)
172   /\ \E v \in Validators, s \in Slots, b \in Blocks, e \in Epochs : Vote(v, s, b, e)
173   /\ \E s \in Slots, b \in Blocks, e \in Epochs : Finalize(s, b, e)
174   /\ \E v \in Validators, e \in Epochs : AdoptEpoch(v, e)
175   /\ \E v \in Validators, e \in Epochs : RegistryRollback(v, e)
176   /\ \E v \in Validators : GuardianOutage(v)
177   /\ \E w \in Witnesses : WitnessOutage(w)
178   /\ \E w \in Witnesses : WitnessRecovery(w)
179   /\ \E w \in Witnesses, level \in CheckpointLevels : AdvanceWitnessCheckpoint(w, level)
180   /\ \E w \in Witnesses, level \in CheckpointLevels : WitnessCheckpointRollback(w, level)
181
182 TypeInvariant ==
183   /\ votes \subseq VoteDomain
184   /\ witnessCerts \subseq WitnessDomain
185   /\ finalized \subseq FinalizationDomain
186   /\ finalizerSets \in [FinalizationDomain -> SUBSET Validators]
187   /\ registryEpoch \in [Validators -> Epochs]
188   /\ guardianReady \in [Validators -> BOOLEAN]
189   /\ witnessOnline \in [Witnesses -> BOOLEAN]
190   /\ witnessCheckpoint \in [Witnesses -> CheckpointLevels]
191   /\ assignedWitness \in [Slots -> Witnesses]
192   /\ reassignmentDepth \in [Slots -> 0..MaxReassignmentDepth]
193
194 WitnessAssignmentSoundness ==
195   \A s \in Slots :
196     reassignmentDepth[s] <= MaxReassignmentDepth
197
198 NoDualVotes ==
199   \A v \in Validators, s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
200     /\ VoteEvent(v, s, b1, e1) \in votes

```

```

201     /\ VoteEvent(v, s, b2, e2) \in votes
202     => /\ b1 = b2
203     /\ e1 = e2
204
205 WitnessCertificatesStayAssigned ==
206   \A w \in Witnesses, s \in Slots, b \in Blocks, e \in Epochs :
207     WitnessEvent(w, s, b, e) \in witnessCerts
208     => assignedWitness[s] = w /\ reassignmentDepth[s] > 0
209
210 FinalizationWitnessSoundness ==
211   \A s \in Slots, b \in Blocks, e \in Epochs :
212     Finalization(s, b, e) \in finalized
213     => /\ Cardinality(finalizerSets[Finalization(s, b, e)]) >= QuorumSize
214     /\ \A v \in finalizerSets[Finalization(s, b, e)] :
215       VoteEvent(v, s, b, e) \in votes
216
217 Invariant ==
218   /\ TypeInvariant
219   /\ WitnessAssignmentSoundness
220   /\ NoDualVotes
221   /\ WitnessCertificatesStayAssigned
222   /\ FinalizationWitnessSoundness
223
224 Safety ==
225   \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
226     /\ Finalization(s, b1, e1) \in finalized
227     /\ Finalization(s, b2, e2) \in finalized
228     => b1 = b2
229
230 Spec ==
231   Init /\ [] [Next]_vars
232
233 THEOREM OneInCheckpointLevels == 1 \in CheckpointLevels
234   BY SMT DEF CheckpointLevels
235
236 THEOREM InitImpliesInvariant == Init => Invariant
237 PROOF
238 <1>1. Init => TypeInvariant
239   BY SMT, InitialEpochInEpochs, InitialWitnessInWitnesses, OneInCheckpointLevels
240   DEF Init, TypeInvariant, VoteDomain, WitnessDomain, FinalizationDomain
241 <1>2. Init => WitnessAssignmentSoundness
242   BY DEF Init, WitnessAssignmentSoundness
243 <1>3. Init => NoDualVotes
244   BY DEF Init, NoDualVotes
245 <1>4. Init => WitnessCertificatesStayAssigned
246   BY DEF Init, WitnessCertificatesStayAssigned
247 <1>5. Init => FinalizationWitnessSoundness
248   BY DEF Init, FinalizationWitnessSoundness
249 <1>6. QED
250   BY <1>1, <1>2, <1>3, <1>4, <1>5 DEF Invariant
251
252 THEOREM StepPreservesInvariant == Invariant /\ Next => Invariant'
253   BY SMTT(30) DEF Invariant, TypeInvariant, WitnessAssignmentSoundness, NoDualVotes,
254     WitnessCertificatesStayAssigned, FinalizationWitnessSoundness, Next,
255     IssueWitness, ReassignWitness, Vote, Finalize, AdoptEpoch,
256     RegistryRollback, GuardianOutage, WitnessOutage, WitnessRecovery,
257     AdvanceWitnessCheckpoint, WitnessCheckpointRollback, CanIssueWitness,
258     CanVote, HasVote, CanFinalize, EligibleVoters, VoteEvent,
259     WitnessEvent, Finalization, VoteDomain, WitnessDomain, FinalizationDomain
260
261 THEOREM StutterPreservesInvariant == Invariant /\ UNCHANGED vars => Invariant'
262   BY SMT DEF Invariant, vars

```

```

263
264 THEOREM NextPreservesInvariant == Invariant /\ [Next]_vars => Invariant'
265   BY SMT, StepPreservesInvariant, StutterPreservesInvariant DEF vars
266
267 THEOREM FinalizedWitnessesIntersect ==
268   \A s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs :
269     /\ Invariant
270     /\ <<s, b1, e1>> \in finalized
271     /\ <<s, b2, e2>> \in finalized
272     => \E v \in Validators :
273       /\ <<v, s, b1, e1>> \in votes
274       /\ <<v, s, b2, e2>> \in votes
275 PROOF
276 <1>1. TAKE s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs
277 <1>2. ASSUME Invariant,
278       <<s, b1, e1>> \in finalized,
279       <<s, b2, e2>> \in finalized
280   PROVE \E v \in Validators :
281     /\ <<v, s, b1, e1>> \in votes
282     /\ <<v, s, b2, e2>> \in votes
283 <2>1. finalizerSets[<<s, b1, e1>>] \subseq Validators
284   BY SMT, <1>2 DEF Invariant, TypeInvariant, FinalizationDomain, Finalization
285 <2>2. finalizerSets[<<s, b2, e2>>] \subseq Validators
286   BY SMT, <1>2 DEF Invariant, TypeInvariant, FinalizationDomain, Finalization
287 <2>3. Cardinality(finalizerSets[<<s, b1, e1>>]) >= QuorumSize
288   BY SMT, <1>2 DEF Invariant, FinalizationWitnessSoundness
289 <2>4. Cardinality(finalizerSets[<<s, b2, e2>>]) >= QuorumSize
290   BY SMT, <1>2 DEF Invariant, FinalizationWitnessSoundness
291 <2>5. finalizerSets[<<s, b1, e1>>] \cap finalizerSets[<<s, b2, e2>>] # {}
292   BY <2>1, <2>2, <2>3, <2>4, QuorumIntersection
293 <2>6. PICK v \in finalizerSets[<<s, b1, e1>>] \cap finalizerSets[<<s, b2, e2>>] : TRUE
294   BY <2>5
295 <2>7. <<v, s, b1, e1>> \in votes
296   BY SMT, <1>2, <2>6 DEF Invariant, FinalizationWitnessSoundness
297 <2>8. <<v, s, b2, e2>> \in votes
298   BY SMT, <1>2, <2>6 DEF Invariant, FinalizationWitnessSoundness
299 <2>9. QED
300   BY SMT, <2>6, <2>7, <2>8
301 <1>3. QED
302   BY <1>2
303
304 THEOREM InvariantImpliesSafety == Invariant => Safety
305 PROOF
306 <1>1. TAKE s \in Slots, b1 \in Blocks, b2 \in Blocks, e1 \in Epochs, e2 \in Epochs
307 <1>2. ASSUME Invariant,
308       <<s, b1, e1>> \in finalized,
309       <<s, b2, e2>> \in finalized
310   PROVE b1 = b2
311 <2>1. \E v \in Validators :
312     /\ <<v, s, b1, e1>> \in votes
313     /\ <<v, s, b2, e2>> \in votes
314   BY SMT, FinalizedWitnessesIntersect, <1>2
315 <2>2. PICK v \in Validators :
316     /\ <<v, s, b1, e1>> \in votes
317     /\ <<v, s, b2, e2>> \in votes
318   BY <2>1
319 <2>3. QED
320   BY SMT, <1>2, <2>2 DEF Invariant, NoDualVotes
321 <1>3. QED
322   BY <1>2 DEF Safety
323
324 ====

```

A.5 CanonicalOrdering Companion Witnesses

Executable witness artifacts for omission dominance and recursive continuity.

A.5.1 CanonicalOrderingOmissionTrace.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/canonical_ordering/CanonicalOrderingOmissionTrace.tla}

```
1  ---- MODULE CanonicalOrderingOmissionTrace ----
2  EXTENDS CanonicalOrdering
3
4  TargetSlot == 1
5  TargetCert == {"tx1"}
6  TargetTx == "tx2"
7
8  WitnessState ==
9      /\ TargetSlot \in closedCutoffs
10     /\ bulletin[TargetSlot] = {"tx1", "tx2"}
11     /\ CertEvent(TargetSlot, TargetCert) \in candidateCerts
12     /\ OmissionEvent(TargetSlot, TargetCert, TargetTx) \in omissionProofs
13     /\ CertEvent(TargetSlot, TargetCert) \notin admittedCerts
14
15  NoOmissionDominanceWitness == ~WitnessState
16
17  =====
```

A.5.2 CanonicalOrderingOmissionTrace.cfg

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/canonical_ordering/CanonicalOrderingOmissionTrace.cfg}

```
1  SPECIFICATION Spec
2
3  CONSTANTS
4      Slots = {1}
5      Transactions = {"tx1", "tx2"}
6
7  INVARIANTS
8      TypeInvariant
9      AdmittedSoundness
10     OmissionSoundness
11     RecoveredSoundness
12     AdmittedUniqueness
13     Recoverability
14     OmissionDominates
15     NoOmissionDominanceWitness
```

A.5.3 CanonicalCollapseRecursiveContinuity.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/canonical_ordering/CanonicalCollapseRecursiveContinuity.tla}

```

1  ---- MODULE CanonicalCollapseRecursiveContinuity ----
2  EXTENDS Naturals, Sequences, TLC
3
4  CONSTANT MaxHeight
5
6  Slots == 1..MaxHeight
7  Genesis == 1
8  NonGenesisSlots == Slots \ {Genesis}
9  NullHash == 0
10
11  VARIABLES publishedCollapses, publishedProofs, publishedExtensionCertificates, admittedHeaders
12
13  vars ==
14    <<publishedCollapses, publishedProofs, publishedExtensionCertificates, admittedHeaders>>
15
16  RECURSIVE AccumulatorHash(_), ProofHash(_), ProofChain(_)
17
18  PayloadHash(s) == <<"payload", s>>
19
20  AccumulatorHash(s) ==
21    IF s = Genesis
22      THEN <<"accum", s, NullHash, PayloadHash(s)>>
23      ELSE <<"accum", s, AccumulatorHash(s - 1), PayloadHash(s)>>
24
25  Commitment(s) ==
26    [height |-> s,
27     continuityAccumulatorHash |-> AccumulatorHash(s),
28     resultingStateRootHash |-> <<"state", s>>]
29
30  CommitmentHash(s) == <<"commitment-hash", Commitment(s)>>
31
32  PreviousCommitmentHash(s) ==
33    IF s = Genesis THEN NullHash ELSE CommitmentHash(s - 1)
34
35  StatementHash(s) ==
36    <<"statement-hash", Commitment(s), PreviousCommitmentHash(s), PayloadHash(s)>>
37
38  PreviousProofHash(s) ==
39    IF s = Genesis THEN NullHash ELSE ProofHash(s - 1)
40
41  ProofBytes(s) ==
42    <<"hash-pcd-v1", StatementHash(s), PreviousProofHash(s)>>
43
44  ProofStep(s) ==
45    [commitment |-> Commitment(s),
46     previousCommitmentHash |-> PreviousCommitmentHash(s),
47     payloadHash |-> PayloadHash(s),
48     previousProofHash |-> PreviousProofHash(s),
49     proofBytes |-> ProofBytes(s)]
50
51  ProofHash(s) == <<"proof-hash", ProofStep(s)>>
52
53  ProofChain(s) ==
54    IF s = Genesis
55      THEN <<ProofStep(s)>>
56      ELSE Append(ProofChain(s - 1), ProofStep(s))
57
58  ExtensionCertificate(s) ==
59    [coveredHeight |-> s,
60     predecessorCommitment |-> Commitment(s - 1),
61     predecessorProofHash |-> ProofHash(s - 1)]

```

```

62
63 Last(seq) == seq[Len(seq)]
64
65 Init ==
66   /\ publishedCollapses = {}
67   /\ publishedProofs = {}
68   /\ publishedExtensionCertificates = {}
69   /\ admittedHeaders = {}
70
71 PublishCollapseStep ==
72   \E s \in Slots :
73     /\ s \notin publishedCollapses
74     /\ publishedCollapses' = publishedCollapses \cup {s}
75     /\ UNCHANGED <<publishedProofs, publishedExtensionCertificates, admittedHeaders>>
76
77 PublishRecursiveProofStep ==
78   \E s \in Slots :
79     /\ s \in publishedCollapses
80     /\ s \notin publishedProofs
81     /\ IF s = Genesis THEN TRUE ELSE s - 1 \in publishedProofs
82     /\ publishedProofs' = publishedProofs \cup {s}
83     /\ UNCHANGED <<publishedCollapses, publishedExtensionCertificates, admittedHeaders>>
84
85 PublishExtensionCertificateStep ==
86   \E s \in NonGenesisSlots :
87     /\ s \in publishedCollapses
88     /\ s - 1 \in publishedProofs
89     /\ s \notin publishedExtensionCertificates
90     /\ publishedExtensionCertificates' = publishedExtensionCertificates \cup {s}
91     /\ UNCHANGED <<publishedCollapses, publishedProofs, admittedHeaders>>
92
93 AdmitHeaderStep ==
94   \E s \in Slots :
95     /\ s \in publishedCollapses
96     /\ s \notin admittedHeaders
97     /\ IF s = Genesis THEN TRUE ELSE s \in publishedExtensionCertificates
98     /\ admittedHeaders' = admittedHeaders \cup {s}
99     /\ UNCHANGED <<publishedCollapses, publishedProofs, publishedExtensionCertificates>>
100
101 Next ==
102   \/ PublishCollapseStep
103   \/ PublishRecursiveProofStep
104   \/ PublishExtensionCertificateStep
105   \/ AdmitHeaderStep
106
107 TypeInvariant ==
108   /\ publishedCollapses \subseq Slots
109   /\ publishedProofs \subseq Slots
110   /\ publishedExtensionCertificates \subseq NonGenesisSlots
111   /\ admittedHeaders \subseq Slots
112
113 RecursiveProofSoundness ==
114   \A s \in publishedProofs :
115     /\ s \in publishedCollapses
116     /\ IF s = Genesis THEN TRUE ELSE s - 1 \in publishedProofs
117     /\ LET step == ProofStep(s) IN
118       /\ step.commitment = Commitment(s)
119       /\ step.previousCommitmentHash = PreviousCommitmentHash(s)
120       /\ step.payloadHash = PayloadHash(s)
121       /\ step.previousProofHash = PreviousProofHash(s)
122       /\ step.proofBytes = ProofBytes(s)
123       /\ step.commitment.continuityAccumulatorHash = AccumulatorHash(s)

```

```

124
125 RecursiveProofChainSoundness ==
126   \A s \in publishedProofs :
127     /\ Len(ProofChain(s)) = s
128     /\ Last(ProofChain(s)) = ProofStep(s)
129     /\ IF s = Genesis
130       THEN TRUE
131       ELSE /\ ProofChain(s)[Len(ProofChain(s)) - 1] = ProofStep(s - 1)
132
133 ExtensionCertificateSoundness ==
134   \A s \in publishedExtensionCertificates :
135     /\ s \in NonGenesisSlots
136     /\ s - 1 \in publishedProofs
137     /\ ExtensionCertificate(s).coveredHeight = s
138     /\ ExtensionCertificate(s).predecessorCommitment = Commitment(s - 1)
139     /\ ExtensionCertificate(s).predecessorProofHash = ProofHash(s - 1)
140
141 HeaderAdmissionSoundness ==
142   \A s \in admittedHeaders :
143     /\ s \in publishedCollapses
144     /\ IF s = Genesis
145       THEN TRUE
146       ELSE /\ s \in publishedExtensionCertificates
147         /\ s - 1 \in publishedProofs
148         /\ ExtensionCertificate(s).predecessorCommitment = Commitment(s - 1)
149         /\ ExtensionCertificate(s).predecessorProofHash = ProofHash(s - 1)
150
151 Invariant ==
152   /\ TypeInvariant
153   /\ RecursiveProofSoundness
154   /\ RecursiveProofChainSoundness
155   /\ ExtensionCertificateSoundness
156   /\ HeaderAdmissionSoundness
157
158 Spec == Init /\ [] [Next]_vars
159
160 =====

```

A.5.4 CanonicalCollapseRecursiveContinuity.cfg

Repository path. `\detokenize{internal-docs/architecture/protocols/aft/formal/canonical_ordering/CanonicalCollapseRecursiveContinuity.cfg}`

```

1 SPECIFICATION Spec
2
3 CONSTANTS
4   MaxHeight = 3
5
6 INVARIANTS
7   TypeInvariant
8   RecursiveProofSoundness
9   RecursiveProofChainSoundness
10  ExtensionCertificateSoundness
11  HeaderAdmissionSoundness

```

A.6 Recovery and Classical-Agreement Bridge

Recurring recovery kernel, finite reduction, totality bridge, and final classical-agreement collapse wrapper.

A.6.1 NestedGuardianRecoveryRecurringInductionCore.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/nested_guardian/NestedGuardianRecoveryRecurringInductionCore.tla}

```
1  ---- MODULE NestedGuardianRecoveryRecurringInductionCore ----
2  (*****
3  (* PREMISE-CONTRACT KERNEL (requalified 2026-08-17, AFT-CB P2.2, standing *)
4  (* rule 2 -- labels claim only what is checked). `RecoveryInductionBase == *)
5  (* <>RecoveryClosedPrefix(1)` and the step contracts below are PREMISES: *)
6  (* assumed contracts whose sufficiency to close the prefix is what this *)
7  (* module checks -- the premises themselves are NOT derived here from a *)
8  (* Byzantine network plus a dishonest-weight bound (the Q3 finding). The *)
9  (* boundary lane's successor does not inherit these premises: the *)
10 (* common-boundary cadence obligations are checked directly, under *)
11 (* explicit fairness, in common_boundary/BoundaryLiveness.tla *)
12 (* (model-checked at  $n \leq 4$ , stated as such -- never "proven"). *)
13 (*****
14 EXTENDS Naturals, FiniteSets, TLC
15
16 CONSTANT Validators, Witnesses, Blocks, Slots, Epochs, QuorumSize, MaxReassignmentDepth,
17         InitialEpoch, InitialWitness, TotalCycles,
18         TargetSlotOf(_), TargetBlockOf(_), StableEpochOf(_),
19         SmallCommitteesSlot, SmallRecoveryThreshold, LargeRecoveryThreshold,
20         SmallRecoveryCommittee, LargeRecoveryCommittee
21
22 ASSUME InitialEpoch \in Epochs
23 ASSUME InitialWitness \in Witnesses
24 ASSUME QuorumSize \in 1..Cardinality(Validators)
25 ASSUME TotalCycles \in Nat
26 ASSUME TotalCycles >= 1
27 ASSUME \A c \in 1..TotalCycles :
28     /\ TargetSlotOf(c) \in Slots
29     /\ TargetBlockOf(c) \in Blocks
30     /\ StableEpochOf(c) \in Epochs
31
32 VARIABLES votes, witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,
33     <-- witnessOnline,
34     witnessCheckpoint, assignedWitness, reassignmentDepth, currentCycle, phase, churnStage,
35     continuationBoundary, continuationAnchorPublished, continuationAnchorBoundary,
36     continuationPagePublished, continuationPageBoundary, continuationFetched,
37     shareReceipts, shareConflicts, windowClosed, missingShareClaims,
38     missingThresholdCertificates, recovered, recoveredSurfaces,
39     recoveryConflicts, aborted
40
41 Core ==
42     INSTANCE NestedGuardianRecoveryRecurringLivenessCore
43     WITH Validators <- Validators,
44         Witnesses <- Witnesses,
45         Blocks <- Blocks,
46         Slots <- Slots,
47         Epochs <- Epochs,
48         QuorumSize <- QuorumSize,
```

```

48     MaxReassignmentDepth <- MaxReassignmentDepth,
49     InitialEpoch <- InitialEpoch,
50     InitialWitness <- InitialWitness,
51     TotalCycles <- TotalCycles,
52     TargetSlotOf <- TargetSlotOf,
53     TargetBlockOf <- TargetBlockOf,
54     StableEpochOf <- StableEpochOf,
55     SmallCommitteeSlot <- SmallCommitteeSlot,
56     SmallRecoveryThreshold <- SmallRecoveryThreshold,
57     LargeRecoveryThreshold <- LargeRecoveryThreshold,
58     SmallRecoveryCommittee <- SmallRecoveryCommittee,
59     LargeRecoveryCommittee <- LargeRecoveryCommittee,
60     votes <- votes,
61     witnessCerts <- witnessCerts,
62     finalized <- finalized,
63     finalizerSets <- finalizerSets,
64     registryEpoch <- registryEpoch,
65     guardianReady <- guardianReady,
66     witnessOnline <- witnessOnline,
67     witnessCheckpoint <- witnessCheckpoint,
68     assignedWitness <- assignedWitness,
69     reassignmentDepth <- reassignmentDepth,
70     currentCycle <- currentCycle,
71     phase <- phase,
72     churnStage <- churnStage,
73     continuationBoundary <- continuationBoundary,
74     continuationAnchorPublished <- continuationAnchorPublished,
75     continuationAnchorBoundary <- continuationAnchorBoundary,
76     continuationPagePublished <- continuationPagePublished,
77     continuationPageBoundary <- continuationPageBoundary,
78     continuationFetched <- continuationFetched,
79     shareReceipts <- shareReceipts,
80     shareConflicts <- shareConflicts,
81     windowClosed <- windowClosed,
82     missingShareClaims <- missingShareClaims,
83     missingThresholdCertificates <- missingThresholdCertificates,
84     recovered <- recovered,
85     recoveredSurfaces <- recoveredSurfaces,
86     recoveryConflicts <- recoveryConflicts,
87     aborted <- aborted
88
89 vars == Core!vars
90 CycleRange == Core!CycleRange
91 PrefixAdvanceRange == Core!PrefixAdvanceRange
92 RecoveryClosedPrefix(c) == Core!RecoveryClosedPrefix(c)
93
94 PriorCycleRange(c) ==
95   IF c = 1 THEN {}
96   ELSE 1..(c - 1)
97
98 RecoveryInductionBase ==
99   <>RecoveryClosedPrefix(1)
100
101 RecoveryInductionStep(c) ==
102   /\ c \in PrefixAdvanceRange
103   /\ Core!RecoveryPrefixAdvanceContract(c)
104
105 RecoveryInductionPremisesUpTo(c) ==
106   /\ c \in CycleRange
107   /\ RecoveryInductionBase
108   /\ \A i \in PriorCycleRange(c) :
109     RecoveryInductionStep(i)

```

```

110
111 RecoveryRecurringInductionPremises ==
112   /\ RecoveryInductionBase
113   /\ \A i \in PrefixAdvanceRange :
114     RecoveryInductionStep(i)
115
116 RecoveryClosedPrefixInductionKernel(c) ==
117   /\ c \in CycleRange
118   /\ (RecoveryInductionPremisesUpTo(c)
119     => <>RecoveryClosedPrefix(c))
120
121 RecoveryRecurringInductionKernel ==
122   \A c \in CycleRange :
123     RecoveryClosedPrefixInductionKernel(c)
124
125 =====

```

A.6.2 NestedGuardianRecoveryRecurringProof.tla

Repository path. `\detokenize{internal-docs/architecture/protocols/aft/formal/nested_guardian/NestedGuardianRecoveryRecurringProof.tla}`

```

1  ---- MODULE NestedGuardianRecoveryRecurringProof ----
2  EXTENDS Naturals, FiniteSets, TLC, TLAPS
3
4  CONSTANT Validators, Witnesses, Blocks, Slots, Epochs, QuorumSize, MaxReassignmentDepth,
5     InitialEpoch, InitialWitness, TotalCycles,
6     TargetSlotOf(_), TargetBlockOf(_), StableEpochOf(_),
7     SmallCommitteesSlot, SmallRecoveryThreshold, LargeRecoveryThreshold,
8     SmallRecoveryCommittee, LargeRecoveryCommittee
9
10 ASSUME InitialEpochInEpochs == InitialEpoch \in Epochs
11 ASSUME InitialWitnessInWitnesses == InitialWitness \in Witnesses
12 ASSUME QuorumSizeInValidatorRange == QuorumSize \in 1..Cardinality(Validators)
13 ASSUME TotalCyclesIsNat == TotalCycles \in Nat
14 ASSUME TotalCyclesAtLeastOne == TotalCycles >= 1
15 ASSUME TargetCycleDomains ==
16   \A c \in 1..TotalCycles :
17     /\ TargetSlotOf(c) \in Slots
18     /\ TargetBlockOf(c) \in Blocks
19     /\ StableEpochOf(c) \in Epochs
20
21 VARIABLES votes, witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,
22   \leftrightarrow witnessOnline,
23     witnessCheckpoint, assignedWitness, reassignmentDepth, currentCycle, phase, churnStage,
24     continuationBoundary, continuationAnchorPublished, continuationAnchorBoundary,
25     continuationPagePublished, continuationPageBoundary, continuationFetched,
26     shareReceipts, shareConflicts, windowClosed, missingShareClaims,
27     missingThresholdCertificates, recovered, recoveredSurfaces,
28     recoveryConflicts, aborted
29
30 Core ==
31   INSTANCE NestedGuardianRecoveryRecurringInductionCore
32   WITH Validators <- Validators,
33     Witnesses <- Witnesses,
34     Blocks <- Blocks,
35     Slots <- Slots,
36     Epochs <- Epochs,
37     QuorumSize <- QuorumSize,

```

```

37     MaxReassignmentDepth <- MaxReassignmentDepth,
38     InitialEpoch <- InitialEpoch,
39     InitialWitness <- InitialWitness,
40     TotalCycles <- TotalCycles,
41     TargetSlotOf <- TargetSlotOf,
42     TargetBlockOf <- TargetBlockOf,
43     StableEpochOf <- StableEpochOf,
44     SmallCommitteeSlot <- SmallCommitteeSlot,
45     SmallRecoveryThreshold <- SmallRecoveryThreshold,
46     LargeRecoveryThreshold <- LargeRecoveryThreshold,
47     SmallRecoveryCommittee <- SmallRecoveryCommittee,
48     LargeRecoveryCommittee <- LargeRecoveryCommittee,
49     votes <- votes,
50     witnessCerts <- witnessCerts,
51     finalized <- finalized,
52     finalizerSets <- finalizerSets,
53     registryEpoch <- registryEpoch,
54     guardianReady <- guardianReady,
55     witnessOnline <- witnessOnline,
56     witnessCheckpoint <- witnessCheckpoint,
57     assignedWitness <- assignedWitness,
58     reassignmentDepth <- reassignmentDepth,
59     currentCycle <- currentCycle,
60     phase <- phase,
61     churnStage <- churnStage,
62     continuationBoundary <- continuationBoundary,
63     continuationAnchorPublished <- continuationAnchorPublished,
64     continuationAnchorBoundary <- continuationAnchorBoundary,
65     continuationPagePublished <- continuationPagePublished,
66     continuationPageBoundary <- continuationPageBoundary,
67     continuationFetched <- continuationFetched,
68     shareReceipts <- shareReceipts,
69     shareConflicts <- shareConflicts,
70     windowClosed <- windowClosed,
71     missingShareClaims <- missingShareClaims,
72     missingThresholdCertificates <- missingThresholdCertificates,
73     recovered <- recovered,
74     recoveredSurfaces <- recoveredSurfaces,
75     recoveryConflicts <- recoveryConflicts,
76     aborted <- aborted
77
78 RecoveryRecurringClosedPrefixes ==
79   \A c \in Core!CycleRange :
80     <>Core!RecoveryClosedPrefix(c)
81
82 RecoveryParametricRecurrenceArgument ==
83   /\ Core!RecoveryRecurringInductionPremises
84   /\ Core!RecoveryRecurringInductionKernel
85   => RecoveryRecurringClosedPrefixes
86
87 THEOREM PriorCycleRangeImpliesTotalCyclesNotOne ==
88   \A c \in Core!CycleRange :
89     \A i \in Core!PriorCycleRange(c) :
90       TotalCycles # 1
91   BY SMT, TotalCyclesIsNat, TotalCyclesAtLeastOne
92   DEF Core!CycleRange, Core!Core!CycleRange, Core!PriorCycleRange
93
94 THEOREM PriorCycleRangeImpliesPrefixAdvanceInterval ==
95   \A c \in Core!CycleRange :
96     \A i \in Core!PriorCycleRange(c) :
97       i \in 1..(TotalCycles - 1)
98   BY SMT, TotalCyclesIsNat, TotalCyclesAtLeastOne

```

```

99     DEF Core!CycleRange, Core!Core!CycleRange, Core!PriorCycleRange
100
101 THEOREM RecoveryInductionPremisesRestrict ==
102   \A c \in Core!CycleRange :
103     Core!RecoveryRecurringInductionPremises
104     => Core!RecoveryInductionPremisesUpTo(c)
105 PROOF
106   <1>1. TAKE c \in Core!CycleRange
107   <1>2. ASSUME Core!RecoveryRecurringInductionPremises
108     PROVE Core!RecoveryInductionPremisesUpTo(c)
109   <2>1. Core!RecoveryInductionBase
110     BY <1>2 DEF Core!RecoveryRecurringInductionPremises
111   <2>2. \A i \in Core!PriorCycleRange(c) :
112     Core!RecoveryInductionStep(i)
113   PROOF
114     <3>1. TAKE i \in Core!PriorCycleRange(c)
115     <3>2. i \in Core!PrefixAdvanceRange
116     PROOF
117       <4>1. TotalCycles # 1
118       BY PriorCycleRangeImpliesTotalCyclesNotOne, <1>1, <3>1
119       <4>2. i \in 1..(TotalCycles - 1)
120       BY PriorCycleRangeImpliesPrefixAdvanceInterval, <1>1, <3>1
121       <4>3. TotalCycles # 1
122       BY <4>1
123       <4>6. QED
124       BY <4>3, <4>2
125       DEF Core!PrefixAdvanceRange, Core!Core!PrefixAdvanceRange
126     <3>3. Core!RecoveryInductionStep(i)
127     BY <1>2, <3>2 DEF Core!RecoveryRecurringInductionPremises
128     <3>4. QED
129     BY <3>3
130   <2>3. QED
131   BY <1>1, <2>1, <2>2
132   DEF Core!RecoveryInductionPremisesUpTo
133   <1>3. QED
134   BY <1>2
135
136 THEOREM RecoveryInductionKernelRestrict ==
137   \A c \in Core!CycleRange :
138     Core!RecoveryRecurringInductionKernel
139     => Core!RecoveryClosedPrefixInductionKernel(c)
140   BY DEF Core!RecoveryRecurringInductionKernel,
141     Core!RecoveryClosedPrefixInductionKernel,
142     Core!CycleRange
143
144 THEOREM RecoveryRecurringClosedPrefixFromKernel ==
145   \A c \in Core!CycleRange :
146     /\ Core!RecoveryRecurringInductionPremises
147     /\ Core!RecoveryRecurringInductionKernel
148     => <>Core!RecoveryClosedPrefix(c)
149 PROOF
150   <1>1. TAKE c \in Core!CycleRange
151   <1>2. ASSUME Core!RecoveryRecurringInductionPremises,
152     Core!RecoveryRecurringInductionKernel
153   PROVE <>Core!RecoveryClosedPrefix(c)
154   <2>1. Core!RecoveryInductionPremisesUpTo(c)
155   BY RecoveryInductionPremisesRestrict, <1>1, <1>2
156   <2>2. Core!RecoveryClosedPrefixInductionKernel(c)
157   BY RecoveryInductionKernelRestrict, <1>1, <1>2
158   <2>3. QED
159   BY <2>1, <2>2
160   DEF Core!RecoveryClosedPrefixInductionKernel

```

```

161 <1>3. QED
162 BY <1>2
163
164 THEOREM RecoveryParametricRecurrenceArgumentTheorem ==
165   RecoveryParametricRecurrenceArgument
166   BY RecoveryRecurringClosedPrefixFromKernel
167     DEF RecoveryParametricRecurrenceArgument,
168       RecoveryRecurringClosedPrefixes
169
170 ====

```

A.6.3 NestedGuardianRecoveryClassicalAgreementReduction.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/nested_guardian/NestedGuardianRecoveryClassicalAgreementReduction.tla}

```

1  ---- MODULE NestedGuardianRecoveryClassicalAgreementReduction ----
2  EXTENDS Naturals, FiniteSets, TLC, TLAPS
3
4  CONSTANT Validators, Witnesses, Blocks, Slots, Epochs, QuorumSize, MaxReassignmentDepth,
5           InitialEpoch, InitialWitness, TotalCycles,
6           TargetSlotOf(_), TargetBlockOf(_), StableEpochOf(_),
7           SmallCommitteesSlot, SmallRecoveryThreshold, LargeRecoveryThreshold,
8           SmallRecoveryCommittee, LargeRecoveryCommittee
9
10 ASSUME InitialEpoch \in Epochs
11 ASSUME InitialWitness \in Witnesses
12 ASSUME QuorumSize \in 1..Cardinality(Validators)
13 ASSUME TotalCycles \in Nat
14 ASSUME TotalCycles >= 1
15 ASSUME \A c \in 1..TotalCycles :
16   /\ TargetSlotOf(c) \in Slots
17   /\ TargetBlockOf(c) \in Blocks
18   /\ StableEpochOf(c) \in Epochs
19
20 VARIABLES votes, witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,
21   ↪ witnessOnline,
22   witnessCheckpoint, assignedWitness, reassignmentDepth, currentCycle, phase, churnStage,
23   continuationBoundary, continuationAnchorPublished, continuationAnchorBoundary,
24   continuationPagePublished, continuationPageBoundary, continuationFetched,
25   shareReceipts, shareConflicts, windowClosed, missingShareClaims,
26   missingThresholdCertificates, recovered, recoveredSurfaces,
27   recoveryConflicts, aborted
28
29 Proof ==
30   INSTANCE NestedGuardianRecoveryRecurringProof
31   WITH Validators <- Validators,
32        Witnesses <- Witnesses,
33        Blocks <- Blocks,
34        Slots <- Slots,
35        Epochs <- Epochs,
36        QuorumSize <- QuorumSize,
37        MaxReassignmentDepth <- MaxReassignmentDepth,
38        InitialEpoch <- InitialEpoch,
39        InitialWitness <- InitialWitness,
40        TotalCycles <- TotalCycles,
41        TargetSlotOf <- TargetSlotOf,
42        TargetBlockOf <- TargetBlockOf,
43        StableEpochOf <- StableEpochOf,

```

```

43     SmallCommitteeSlot <- SmallCommitteeSlot,
44     SmallRecoveryThreshold <- SmallRecoveryThreshold,
45     LargeRecoveryThreshold <- LargeRecoveryThreshold,
46     SmallRecoveryCommittee <- SmallRecoveryCommittee,
47     LargeRecoveryCommittee <- LargeRecoveryCommittee,
48     votes <- votes,
49     witnessCerts <- witnessCerts,
50     finalized <- finalized,
51     finalizerSets <- finalizerSets,
52     registryEpoch <- registryEpoch,
53     guardianReady <- guardianReady,
54     witnessOnline <- witnessOnline,
55     witnessCheckpoint <- witnessCheckpoint,
56     assignedWitness <- assignedWitness,
57     reassignmentDepth <- reassignmentDepth,
58     currentCycle <- currentCycle,
59     phase <- phase,
60     churnStage <- churnStage,
61     continuationBoundary <- continuationBoundary,
62     continuationAnchorPublished <- continuationAnchorPublished,
63     continuationAnchorBoundary <- continuationAnchorBoundary,
64     continuationPagePublished <- continuationPagePublished,
65     continuationPageBoundary <- continuationPageBoundary,
66     continuationFetched <- continuationFetched,
67     shareReceipts <- shareReceipts,
68     shareConflicts <- shareConflicts,
69     windowClosed <- windowClosed,
70     missingShareClaims <- missingShareClaims,
71     missingThresholdCertificates <- missingThresholdCertificates,
72     recovered <- recovered,
73     recoveredSurfaces <- recoveredSurfaces,
74     recoveryConflicts <- recoveryConflicts,
75     aborted <- aborted
76
77 CycleRange == 1..TotalCycles
78
79 ClassicalAgreementDecisionObject(c) ==
80   [cycle |-> c,
81    slot |-> TargetSlotOf(c),
82    block |-> TargetBlockOf(c),
83    epoch |-> StableEpochOf(c)]
84
85 ClassicalAgreementPrefixObject(c) ==
86   [i \in 1..c |-> ClassicalAgreementDecisionObject(i)]
87
88 FiniteClassicalAgreementPrefixRealized(c) ==
89   /\ c \in CycleRange
90   /\ <>Proof!Core!RecoveryClosedPrefix(c)
91
92 FiniteClassicalAgreementLiveness ==
93   \A c \in CycleRange :
94     FiniteClassicalAgreementPrefixRealized(c)
95
96 ClassicalAgreementFiniteReduction ==
97   Proof!RecoveryRecurringClosedPrefixes
98   => FiniteClassicalAgreementLiveness
99
100 THEOREM ProofCycleRangeMatchesCycleRange ==
101   Proof!Core!CycleRange = CycleRange
102   BY DEF Proof!Core!CycleRange, Proof!Core!Core!CycleRange, CycleRange
103
104 THEOREM FiniteClassicalAgreementPrefixFromRecurringPrefixes ==

```

```

105 \A c \in CycleRange :
106   Proof!RecoveryRecurringClosedPrefixes
107   => FiniteClassicalAgreementPrefixRealized(c)
108 PROOF
109   <1>1. TAKE c \in CycleRange
110   <1>2. ASSUME Proof!RecoveryRecurringClosedPrefixes
111         PROVE FiniteClassicalAgreementPrefixRealized(c)
112         <2>1. c \in Proof!Core!CycleRange
113               BY ProofCycleRangeMatchesCycleRange, <1>1
114         <2>2. <>Proof!Core!RecoveryClosedPrefix(c)
115               BY <1>2, <2>1 DEF Proof!RecoveryRecurringClosedPrefixes
116         <2>3. QED
117               BY <1>1, <2>2 DEF FiniteClassicalAgreementPrefixRealized
118   <1>3. QED
119       BY <1>2
120
121 THEOREM ClassicalAgreementFiniteReductionTheorem ==
122   ClassicalAgreementFiniteReduction
123   BY FiniteClassicalAgreementPrefixFromRecurringPrefixes
124     DEF ClassicalAgreementFiniteReduction,
125         FiniteClassicalAgreementLiveness
126
127 =====

```

A.6.4 NestedGuardianRecoveryClassicalAgreementTotality.tla

Repository path. `\detokenize{internal-docs/architecture/protocols/aft/formal/nested_guardian/NestedGuardianRecoveryClassicalAgreementTotality.tla}`

```

1  ---- MODULE NestedGuardianRecoveryClassicalAgreementTotality ----
2  EXTENDS Naturals, FiniteSets, TLC, TLAPS
3
4  CONSTANT Validators, Witnesses, Blocks, Slots, Epochs, QuorumSize, MaxReassignmentDepth,
5             InitialEpoch, InitialWitness, TotalCycles,
6             TargetSlotOf(_), TargetBlockOf(_), StableEpochOf(_),
7             SmallCommitteesSlot, SmallRecoveryThreshold, LargeRecoveryThreshold,
8             SmallRecoveryCommittee, LargeRecoveryCommittee
9
10 ASSUME InitialEpochInEpochs == InitialEpoch \in Epochs
11 ASSUME InitialWitnessInWitnesses == InitialWitness \in Witnesses
12 ASSUME QuorumSizeInValidatorRange == QuorumSize \in 1..Cardinality(Validators)
13 ASSUME TotalCyclesIsNat == TotalCycles \in Nat
14 ASSUME TotalCyclesAtLeastOne == TotalCycles >= 1
15 ASSUME TargetCycleDomains ==
16   \A c \in 1..TotalCycles :
17     /\ TargetSlotOf(c) \in Slots
18     /\ TargetBlockOf(c) \in Blocks
19     /\ StableEpochOf(c) \in Epochs
20
21 VARIABLES votes, witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,
22   \witnessOnline,
23   witnessCheckpoint, assignedWitness, reassignmentDepth, currentCycle, phase, churnStage,
24   continuationBoundary, continuationAnchorPublished, continuationAnchorBoundary,
25   continuationPagePublished, continuationPageBoundary, continuationFetched,
26   shareReceipts, shareConflicts, windowClosed, missingShareClaims,
27   missingThresholdCertificates, recovered, recoveredSurfaces,
28   recoveryConflicts, aborted
29
30 Reduction ==

```

```

30  INSTANCE NestedGuardianRecoveryClassicalAgreementReduction
31  WITH Validators <- Validators,
32       Witnesses <- Witnesses,
33       Blocks <- Blocks,
34       Slots <- Slots,
35       Epochs <- Epochs,
36       QuorumSize <- QuorumSize,
37       MaxReassignmentDepth <- MaxReassignmentDepth,
38       InitialEpoch <- InitialEpoch,
39       InitialWitness <- InitialWitness,
40       TotalCycles <- TotalCycles,
41       TargetSlotOf <- TargetSlotOf,
42       TargetBlockOf <- TargetBlockOf,
43       StableEpochOf <- StableEpochOf,
44       SmallCommitteeSlot <- SmallCommitteeSlot,
45       SmallRecoveryThreshold <- SmallRecoveryThreshold,
46       LargeRecoveryThreshold <- LargeRecoveryThreshold,
47       SmallRecoveryCommittee <- SmallRecoveryCommittee,
48       LargeRecoveryCommittee <- LargeRecoveryCommittee,
49       votes <- votes,
50       witnessCerts <- witnessCerts,
51       finalized <- finalized,
52       finalizerSets <- finalizerSets,
53       registryEpoch <- registryEpoch,
54       guardianReady <- guardianReady,
55       witnessOnline <- witnessOnline,
56       witnessCheckpoint <- witnessCheckpoint,
57       assignedWitness <- assignedWitness,
58       reassignmentDepth <- reassignmentDepth,
59       currentCycle <- currentCycle,
60       phase <- phase,
61       churnStage <- churnStage,
62       continuationBoundary <- continuationBoundary,
63       continuationAnchorPublished <- continuationAnchorPublished,
64       continuationAnchorBoundary <- continuationAnchorBoundary,
65       continuationPagePublished <- continuationPagePublished,
66       continuationPageBoundary <- continuationPageBoundary,
67       continuationFetched <- continuationFetched,
68       shareReceipts <- shareReceipts,
69       shareConflicts <- shareConflicts,
70       windowClosed <- windowClosed,
71       missingShareClaims <- missingShareClaims,
72       missingThresholdCertificates <- missingThresholdCertificates,
73       recovered <- recovered,
74       recoveredSurfaces <- recoveredSurfaces,
75       recoveryConflicts <- recoveryConflicts,
76       aborted <- aborted
77
78  CycleRange == 1..TotalCycles
79
80  RecoveryRecurringClosedPrefixes == Reduction!Proof!RecoveryRecurringClosedPrefixes
81  RecoveryRecurringInductionPremises == Reduction!Proof!Core!RecoveryRecurringInductionPremises
82  RecoveryRecurringInductionKernel == Reduction!Proof!Core!RecoveryRecurringInductionKernel
83  RecoveryClosedPrefix(c) == Reduction!Proof!Core!RecoveryClosedPrefix(c)
84
85  ClassicalAgreementTotalHistoryObject ==
86    [i \in CycleRange |-> Reduction!ClassicalAgreementDecisionObject(i)]
87
88  ClassicalAgreementTotalHistoryExtendsFinitePrefixes ==
89    \A c \in CycleRange :
90      [i \in 1..c |-> ClassicalAgreementTotalHistoryObject[i]]
91      = Reduction!ClassicalAgreementPrefixObject(c)

```

```

92
93 TotalClassicalAgreementHistoryRealized ==
94   /\ <>RecoveryClosedPrefix(TotalCycles)
95   /\ ClassicalAgreementTotalHistoryExtendsFinitePrefixes
96
97 TotalClassicalAgreementReduction ==
98   Reduction!FiniteClassicalAgreementLiveness
99   => TotalClassicalAgreementHistoryRealized
100
101 TotalClassicalAgreementFromRecurringPrefixes ==
102   RecoveryRecurringClosedPrefixes
103   => TotalClassicalAgreementHistoryRealized
104
105 TotalClassicalAgreementFromRecurrenceKernel ==
106   /\ RecoveryRecurringInductionPremises
107   /\ RecoveryRecurringInductionKernel
108   => TotalClassicalAgreementHistoryRealized
109
110 THEOREM TotalHistoryEntryMatchesDecisionObject ==
111   \A c \in CycleRange :
112     \A i \in 1..c :
113       ClassicalAgreementTotalHistoryObject[i]
114       = Reduction!ClassicalAgreementDecisionObject(i)
115   BY SMT, TotalCyclesIsNat, TotalCyclesAtLeastOne
116   DEF CycleRange, ClassicalAgreementTotalHistoryObject
117
118 THEOREM ClassicalAgreementTotalHistoryExtendsFinitePrefixesTheorem ==
119   ClassicalAgreementTotalHistoryExtendsFinitePrefixes
120 PROOF
121   <1>1. \A c \in CycleRange :
122     [i \in 1..c |-> ClassicalAgreementTotalHistoryObject[i]]
123     = Reduction!ClassicalAgreementPrefixObject(c)
124   PROOF
125     <2>1. TAKE c \in CycleRange
126     <2>2. \A i \in 1..c :
127       ClassicalAgreementTotalHistoryObject[i]
128       = Reduction!ClassicalAgreementDecisionObject(i)
129     BY TotalHistoryEntryMatchesDecisionObject, <2>1
130     <2>3. QED
131     BY IsaWithSetExtensionality, <2>1, <2>2
132     DEF Reduction!ClassicalAgreementPrefixObject
133   <1>2. QED
134   BY <1>1 DEF ClassicalAgreementTotalHistoryExtendsFinitePrefixes
135
136 THEOREM TotalCyclesInCycleRange ==
137   TotalCycles \in CycleRange
138   BY SMT, TotalCyclesIsNat, TotalCyclesAtLeastOne DEF CycleRange
139
140 THEOREM TotalFinitePrefixFromFiniteLiveness ==
141   Reduction!FiniteClassicalAgreementLiveness
142   => Reduction!FiniteClassicalAgreementPrefixRealized(TotalCycles)
143 PROOF
144   <1>1. ASSUME Reduction!FiniteClassicalAgreementLiveness
145   PROVE Reduction!FiniteClassicalAgreementPrefixRealized(TotalCycles)
146   <2>1. TotalCycles \in Reduction!CycleRange
147   BY TotalCyclesInCycleRange DEF CycleRange, Reduction!CycleRange
148   <2>2. QED
149   BY <1>1, <2>1 DEF Reduction!FiniteClassicalAgreementLiveness
150   <1>2. QED
151   BY <1>1
152
153 THEOREM TotalRecoveryClosedPrefixFromFiniteLiveness ==

```

```

154 Reduction!FiniteClassicalAgreementLiveness
155   => <>RecoveryClosedPrefix(TotalCycles)
156 BY TotalFinitePrefixFromFiniteLiveness
157   DEF Reduction!FiniteClassicalAgreementPrefixRealized,
158     RecoveryClosedPrefix
159
160 THEOREM TotalClassicalAgreementReductionTheorem ==
161   TotalClassicalAgreementReduction
162 BY TotalRecoveryClosedPrefixFromFiniteLiveness,
163   ClassicalAgreementTotalHistoryExtendsFinitePrefixesTheorem
164   DEF TotalClassicalAgreementReduction,
165     TotalClassicalAgreementHistoryRealized
166
167 THEOREM ReductionLivenessFromRecurringPrefixes ==
168   RecoveryRecurringClosedPrefixes
169   => Reduction!FiniteClassicalAgreementLiveness
170 PROOF
171   <1>1. ASSUME RecoveryRecurringClosedPrefixes
172     PROVE Reduction!FiniteClassicalAgreementLiveness
173     <2>1.  $\forall c \in \text{Reduction!CycleRange} :$ 
174       Reduction!FiniteClassicalAgreementPrefixRealized(c)
175     PROOF
176       <3>1. TAKE  $c \in \text{Reduction!CycleRange}$ 
177       <3>2.  $c \in \text{Reduction!Proof!Core!CycleRange}$ 
178       BY <3>1
179         DEF Reduction!CycleRange,
180           Reduction!Proof!Core!CycleRange,
181           Reduction!Proof!Core!Core!CycleRange
182       <3>3. Reduction!Proof!RecoveryRecurringClosedPrefixes
183       BY <1>1 DEF RecoveryRecurringClosedPrefixes
184       <3>4. <>Reduction!Proof!Core!RecoveryClosedPrefix(c)
185       BY <3>2, <3>3 DEF Reduction!Proof!RecoveryRecurringClosedPrefixes
186       <3>5. QED
187       BY <3>1, <3>4 DEF Reduction!FiniteClassicalAgreementPrefixRealized
188     <2>2. QED
189     BY <2>1 DEF Reduction!FiniteClassicalAgreementLiveness
190   <1>2. QED
191   BY <1>1
192
193 THEOREM TotalClassicalAgreementFromRecurringPrefixesTheorem ==
194   TotalClassicalAgreementFromRecurringPrefixes
195 PROOF
196   <1>1. ASSUME RecoveryRecurringClosedPrefixes
197     PROVE TotalClassicalAgreementHistoryRealized
198     <2>1. Reduction!FiniteClassicalAgreementLiveness
199     BY ReductionLivenessFromRecurringPrefixes, <1>1
200     <2>2. Reduction!FiniteClassicalAgreementLiveness
201     => TotalClassicalAgreementHistoryRealized
202     BY TotalClassicalAgreementReductionTheorem
203     DEF TotalClassicalAgreementReduction
204     <2>3. QED
205     BY <2>1, <2>2
206   <1>2. QED
207   BY <1>1 DEF TotalClassicalAgreementFromRecurringPrefixes
208
209 THEOREM RecoveryRecurringClosedPrefixesFromRecurrenceKernel ==
210   /\ RecoveryRecurringInductionPremises
211   /\ RecoveryRecurringInductionKernel
212   => RecoveryRecurringClosedPrefixes
213 PROOF
214   <1>1. ASSUME RecoveryRecurringInductionPremises,
215     RecoveryRecurringInductionKernel

```

```

216     PROVE RecoveryRecurringClosedPrefixes
217 <2>1. \A c \in Reduction!Proof!Core!CycleRange :
218     <>Reduction!Proof!Core!RecoveryClosedPrefix(c)
219     PROOF
220     <3>1. TAKE c \in Reduction!Proof!Core!CycleRange
221     <3>2. Reduction!Proof!Core!RecoveryInductionPremisesUpTo(c)
222         PROOF
223         <4>1. Reduction!Proof!Core!RecoveryInductionBase
224             BY <1>1
225             DEF RecoveryRecurringInductionPremises,
226                 Reduction!Proof!Core!RecoveryRecurringInductionPremises
227         <4>2. \A i \in Reduction!Proof!Core!PriorCycleRange(c) :
228             Reduction!Proof!Core!RecoveryInductionStep(i)
229             PROOF
230             <5>1. TAKE i \in Reduction!Proof!Core!PriorCycleRange(c)
231             <5>2. TotalCycles # 1
232                 BY SMT, TotalCyclesIsNat, TotalCyclesAtLeastOne, <3>1, <5>1
233                 DEF Reduction!Proof!Core!CycleRange,
234                     Reduction!Proof!Core!Core!CycleRange,
235                     Reduction!Proof!Core!PriorCycleRange
236             <5>3. i \in 1..(TotalCycles - 1)
237                 BY SMT, TotalCyclesIsNat, TotalCyclesAtLeastOne, <3>1, <5>1
238                 DEF Reduction!Proof!Core!CycleRange,
239                     Reduction!Proof!Core!Core!CycleRange,
240                     Reduction!Proof!Core!PriorCycleRange
241             <5>4. i \in Reduction!Proof!Core!PrefixAdvanceRange
242                 BY <5>2, <5>3
243                 DEF Reduction!Proof!Core!PrefixAdvanceRange,
244                     Reduction!Proof!Core!Core!PrefixAdvanceRange
245             <5>5. QED
246                 BY <1>1, <5>4
247                 DEF RecoveryRecurringInductionPremises,
248                     Reduction!Proof!Core!RecoveryRecurringInductionPremises
249             <4>3. QED
250                 BY <3>1, <4>1, <4>2
251                 DEF Reduction!Proof!Core!RecoveryInductionPremisesUpTo
252         <3>3. Reduction!Proof!Core!RecoveryClosedPrefixInductionKernel(c)
253             BY <1>1, <3>1
254             DEF RecoveryRecurringInductionKernel,
255                 Reduction!Proof!Core!RecoveryRecurringInductionKernel
256         <3>4. QED
257             BY <3>2, <3>3
258             DEF Reduction!Proof!Core!RecoveryClosedPrefixInductionKernel
259     <2>2. QED
260         BY <2>1 DEF RecoveryRecurringClosedPrefixes,
261             Reduction!Proof!RecoveryRecurringClosedPrefixes
262 <1>2. QED
263     BY <1>1
264     DEF RecoveryRecurringClosedPrefixes,
265         RecoveryRecurringInductionPremises,
266         RecoveryRecurringInductionKernel,
267         Reduction!Proof!RecoveryRecurringClosedPrefixes
268
269 THEOREM TotalClassicalAgreementFromRecurrenceKernelTheorem ==
270     TotalClassicalAgreementFromRecurrenceKernel
271     PROOF
272     <1>1. ASSUME RecoveryRecurringInductionPremises,
273         RecoveryRecurringInductionKernel
274     PROVE TotalClassicalAgreementHistoryRealized
275     <2>1. RecoveryRecurringClosedPrefixes
276         BY RecoveryRecurringClosedPrefixesFromRecurrenceKernel, <1>1
277     <2>2. TotalClassicalAgreementFromRecurringPrefixes

```

```

278         BY TotalClassicalAgreementFromRecurringPrefixesTheorem
279     <2>3. QED
280     BY <2>1, <2>2 DEF TotalClassicalAgreementFromRecurringPrefixes
281 <1>2. QED
282     BY <1>1 DEF TotalClassicalAgreementFromRecurrenceKernel
283
284 =====

```

A.6.5 NestedGuardianRecoveryClassicalAgreementCollapse.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/nested_guardian/NestedGuardianRecoveryClassicalAgreementCollapse.tla}

```

1  ---- MODULE NestedGuardianRecoveryClassicalAgreementCollapse ----
2  EXTENDS Naturals, FiniteSets, TLC, TLAPS
3
4  CONSTANT Validators, Witnesses, Blocks, Slots, Epochs, QuorumSize, MaxReassignmentDepth,
5         InitialEpoch, InitialWitness, TotalCycles,
6         TargetSlotOf(_), TargetBlockOf(_), StableEpochOf(_),
7         SmallCommitteeSlot, SmallRecoveryThreshold, LargeRecoveryThreshold,
8         SmallRecoveryCommittee, LargeRecoveryCommittee
9
10 ASSUME InitialEpochInEpochs == InitialEpoch \in Epochs
11 ASSUME InitialWitnessInWitnesses == InitialWitness \in Witnesses
12 ASSUME QuorumSizeInValidatorRange == QuorumSize \in 1..Cardinality(Validators)
13 ASSUME TotalCyclesIsNat == TotalCycles \in Nat
14 ASSUME TotalCyclesAtLeastOne == TotalCycles >= 1
15 ASSUME TargetCycleDomains ==
16     \A c \in 1..TotalCycles :
17         /\ TargetSlotOf(c) \in Slots
18         /\ TargetBlockOf(c) \in Blocks
19         /\ StableEpochOf(c) \in Epochs
20
21 VARIABLES votes, witnessCerts, finalized, finalizerSets, registryEpoch, guardianReady,
22     ↪ witnessOnline,
23     witnessCheckpoint, assignedWitness, reassignmentDepth, currentCycle, phase, churnStage,
24     continuationBoundary, continuationAnchorPublished, continuationAnchorBoundary,
25     continuationPagePublished, continuationPageBoundary, continuationFetched,
26     shareReceipts, shareConflicts, windowClosed, missingShareClaims,
27     missingThresholdCertificates, recovered, recoveredSurfaces,
28     recoveryConflicts, aborted
29
30 Totality ==
31     INSTANCE NestedGuardianRecoveryClassicalAgreementTotality
32     WITH Validators <- Validators,
33         Witnesses <- Witnesses,
34         Blocks <- Blocks,
35         Slots <- Slots,
36         Epochs <- Epochs,
37         QuorumSize <- QuorumSize,
38         MaxReassignmentDepth <- MaxReassignmentDepth,
39         InitialEpoch <- InitialEpoch,
40         InitialWitness <- InitialWitness,
41         TotalCycles <- TotalCycles,
42         TargetSlotOf <- TargetSlotOf,
43         TargetBlockOf <- TargetBlockOf,
44         StableEpochOf <- StableEpochOf,
45         SmallCommitteeSlot <- SmallCommitteeSlot,
46         SmallRecoveryThreshold <- SmallRecoveryThreshold,

```

```

46     LargeRecoveryThreshold <- LargeRecoveryThreshold,
47     SmallRecoveryCommittee <- SmallRecoveryCommittee,
48     LargeRecoveryCommittee <- LargeRecoveryCommittee,
49     votes <- votes,
50     witnessCerts <- witnessCerts,
51     finalized <- finalized,
52     finalizerSets <- finalizerSets,
53     registryEpoch <- registryEpoch,
54     guardianReady <- guardianReady,
55     witnessOnline <- witnessOnline,
56     witnessCheckpoint <- witnessCheckpoint,
57     assignedWitness <- assignedWitness,
58     reassignmentDepth <- reassignmentDepth,
59     currentCycle <- currentCycle,
60     phase <- phase,
61     churnStage <- churnStage,
62     continuationBoundary <- continuationBoundary,
63     continuationAnchorPublished <- continuationAnchorPublished,
64     continuationAnchorBoundary <- continuationAnchorBoundary,
65     continuationPagePublished <- continuationPagePublished,
66     continuationPageBoundary <- continuationPageBoundary,
67     continuationFetched <- continuationFetched,
68     shareReceipts <- shareReceipts,
69     shareConflicts <- shareConflicts,
70     windowClosed <- windowClosed,
71     missingShareClaims <- missingShareClaims,
72     missingThresholdCertificates <- missingThresholdCertificates,
73     recovered <- recovered,
74     recoveredSurfaces <- recoveredSurfaces,
75     recoveryConflicts <- recoveryConflicts,
76     aborted <- aborted
77
78 CycleRange == Totality!CycleRange
79
80 ClassicalAgreementDecision(c) ==
81   [cycle |-> c,
82    slot |-> TargetSlotOf(c),
83    block |-> TargetBlockOf(c),
84    epoch |-> StableEpochOf(c)]
85
86 ClassicalAgreementHistory(history) ==
87   \A i \in CycleRange :
88     history[i] = ClassicalAgreementDecision(i)
89
90 OrdinaryClassicalAgreementHistory(history) ==
91   /\ ClassicalAgreementHistory(history)
92   /\ <>Totality!RecoveryClosedPrefix(TotalCycles)
93
94 UnconditionalClassicalAgreementSentence ==
95   OrdinaryClassicalAgreementHistory(
96     Totality!ClassicalAgreementTotalHistoryObject
97   )
98
99 THEOREM TotalHistoryMatchesCollapsedClassicalHistory ==
100   ClassicalAgreementHistory(Totality!ClassicalAgreementTotalHistoryObject)
101 PROOF
102   <1>1. \A i \in CycleRange :
103     Totality!ClassicalAgreementTotalHistoryObject[i]
104       = ClassicalAgreementDecision(i)
105   PROOF
106     <2>1. TAKE i \in CycleRange
107     <2>2. QED

```

```

108         BY <2>1
109         DEF ClassicalAgreementDecision,
110             CycleRange,
111             Totality!ClassicalAgreementTotalHistoryObject,
112             Totality!Reduction!ClassicalAgreementDecisionObject
113     <1>2. QED
114     BY <1>1 DEF ClassicalAgreementHistory
115
116 THEOREM TotalityCyclePrefixEntriesStayInCycleRange ==
117     \A c \in Totality!CycleRange :
118     \A i \in 1..c :
119         i \in Totality!CycleRange
120 BY SMT, TotalCyclesIsNat, TotalCyclesAtLeastOne
121     DEF Totality!CycleRange
122
123 THEOREM TotalityReductionCorePriorCycleRangeImpliesPrefixAdvanceRange ==
124     \A c \in Totality!Reduction!Proof!Core!CycleRange :
125     \A i \in Totality!Reduction!Proof!Core!PriorCycleRange(c) :
126         i \in Totality!Reduction!Proof!Core!PrefixAdvanceRange
127 BY SMT, TotalCyclesIsNat, TotalCyclesAtLeastOne
128     DEF Totality!Reduction!Proof!Core!CycleRange,
129         Totality!Reduction!Proof!Core!Core!CycleRange,
130         Totality!Reduction!Proof!Core!PriorCycleRange,
131         Totality!Reduction!Proof!Core!PrefixAdvanceRange,
132         Totality!Reduction!Proof!Core!Core!PrefixAdvanceRange
133
134 THEOREM LocalReductionLivenessFromRecurringPrefixesTheorem ==
135     Totality!RecoveryRecurringClosedPrefixes
136     => Totality!Reduction!FiniteClassicalAgreementLiveness
137 PROOF
138     <1>1. ASSUME Totality!RecoveryRecurringClosedPrefixes
139     PROVE Totality!Reduction!FiniteClassicalAgreementLiveness
140     <2>1. \A c \in Totality!Reduction!CycleRange :
141         Totality!Reduction!FiniteClassicalAgreementPrefixRealized(c)
142     PROOF
143         <3>1. TAKE c \in Totality!Reduction!CycleRange
144         <3>2. c \in Totality!Reduction!Proof!Core!CycleRange
145         BY <3>1
146             DEF Totality!Reduction!CycleRange,
147                 Totality!Reduction!Proof!Core!CycleRange,
148                 Totality!Reduction!Proof!Core!Core!CycleRange
149         <3>3. <>Totality!Reduction!Proof!Core!RecoveryClosedPrefix(c)
150         BY <1>1, <3>2
151             DEF Totality!RecoveryRecurringClosedPrefixes,
152                 Totality!Reduction!Proof!RecoveryRecurringClosedPrefixes
153         <3>4. QED
154         BY <3>1, <3>3
155             DEF Totality!Reduction!FiniteClassicalAgreementPrefixRealized
156     <2>2. QED
157     BY <2>1 DEF Totality!Reduction!FiniteClassicalAgreementLiveness
158 <1>2. QED
159 BY <1>1
160
161 THEOREM LocalTotalClassicalAgreementHistoryFromFiniteLivenessTheorem ==
162     Totality!Reduction!FiniteClassicalAgreementLiveness
163     => Totality!TotalClassicalAgreementHistoryRealized
164 PROOF
165     <1>1. ASSUME Totality!Reduction!FiniteClassicalAgreementLiveness
166     PROVE Totality!TotalClassicalAgreementHistoryRealized
167     <2>1. TotalCycles \in Totality!Reduction!CycleRange
168     BY SMT, TotalCyclesIsNat, TotalCyclesAtLeastOne
169     DEF Totality!Reduction!CycleRange

```

```

170 <2>2. Totality!Reduction!FiniteClassicalAgreementPrefixRealized(TotalCycles)
171 BY <1>1, <2>1 DEF Totality!Reduction!FiniteClassicalAgreementLiveness
172 <2>3. <>Totality!RecoveryClosedPrefix(TotalCycles)
173 BY <2>2
174 DEF Totality!Reduction!FiniteClassicalAgreementPrefixRealized,
175 Totality!RecoveryClosedPrefix
176 <2>4. Totality!ClassicalAgreementTotalHistoryExtendsFinitePrefixes
177 PROOF
178 <3>1. \A c \in Totality!CycleRange :
179 [i \in 1..c |-> Totality!ClassicalAgreementTotalHistoryObject[i]]
180 = Totality!Reduction!ClassicalAgreementPrefixObject(c)
181 PROOF
182 <4>1. TAKE c \in Totality!CycleRange
183 <4>2. \A i \in 1..c :
184 Totality!ClassicalAgreementTotalHistoryObject[i]
185 = Totality!Reduction!ClassicalAgreementDecisionObject(i)
186 PROOF
187 <5>1. TAKE i \in 1..c
188 <5>2. c \in 1..TotalCycles
189 BY <4>1 DEF Totality!CycleRange
190 <5>3. i \in Totality!CycleRange
191 BY TotalityCyclePrefixEntriesStayInCycleRange, <4>1, <5>1
192 <5>4. QED
193 BY <5>3
194 DEF Totality!ClassicalAgreementTotalHistoryObject
195 <4>3. QED
196 BY IsaWithSetExtensionality, <4>1, <4>2
197 DEF Totality!Reduction!ClassicalAgreementPrefixObject
198 <3>2. QED
199 BY <3>1 DEF Totality!ClassicalAgreementTotalHistoryExtendsFinitePrefixes
200 <2>5. QED
201 BY <2>3, <2>4 DEF Totality!TotalClassicalAgreementHistoryRealized
202 <1>2. QED
203 BY <1>1
204
205 THEOREM LocalRecurringPrefixesFromRecurrenceKernelTheorem ==
206 /\ Totality!RecoveryRecurringInductionPremises
207 /\ Totality!RecoveryRecurringInductionKernel
208 => Totality!RecoveryRecurringClosedPrefixes
209 PROOF
210 <1>1. ASSUME Totality!RecoveryRecurringInductionPremises,
211 Totality!RecoveryRecurringInductionKernel
212 PROVE Totality!RecoveryRecurringClosedPrefixes
213 <2>1. \A c \in Totality!Reduction!Proof!Core!CycleRange :
214 <>Totality!Reduction!Proof!Core!RecoveryClosedPrefix(c)
215 PROOF
216 <3>1. TAKE c \in Totality!Reduction!Proof!Core!CycleRange
217 <3>2. /\ Totality!Reduction!Proof!Core!RecoveryRecurringInductionPremises
218 /\ Totality!Reduction!Proof!Core!RecoveryRecurringInductionKernel
219 BY <1>1
220 DEF Totality!RecoveryRecurringInductionPremises,
221 Totality!RecoveryRecurringInductionKernel
222 <3>3. Totality!Reduction!Proof!Core!RecoveryInductionPremisesUpTo(c)
223 PROOF
224 <4>1. Totality!Reduction!Proof!Core!RecoveryInductionBase
225 BY <3>2
226 DEF Totality!Reduction!Proof!Core!RecoveryRecurringInductionPremises
227 <4>2. \A i \in Totality!Reduction!Proof!Core!PriorCycleRange(c) :
228 Totality!Reduction!Proof!Core!RecoveryInductionStep(i)
229 PROOF
230 <5>1. TAKE i \in Totality!Reduction!Proof!Core!PriorCycleRange(c)
231 <5>2. i \in Totality!Reduction!Proof!Core!PrefixAdvanceRange

```

```

232         BY TotalityReductionCorePriorCycleRangeImpliesPrefixAdvanceRange,
233         <3>1, <5>1
234     <5>3. QED
235     BY <3>2, <5>2
236     DEF Totality!Reduction!Proof!Core!RecoveryRecurringInductionPremises
237 <4>3. QED
238     BY <3>1, <4>1, <4>2
239     DEF Totality!Reduction!Proof!Core!RecoveryInductionPremisesUpTo
240 <3>4. Totality!Reduction!Proof!Core!RecoveryClosedPrefixInductionKernel(c)
241     BY <3>1, <3>2
242     DEF Totality!Reduction!Proof!Core!RecoveryRecurringInductionKernel
243 <3>5. <>Totality!Reduction!Proof!Core!RecoveryClosedPrefix(c)
244     BY <3>3, <3>4
245     DEF Totality!Reduction!Proof!Core!RecoveryClosedPrefixInductionKernel
246 <3>6. (/ Totality!Reduction!Proof!Core!RecoveryRecurringInductionPremises
247     /\ Totality!Reduction!Proof!Core!RecoveryRecurringInductionKernel)
248     => <>Totality!Reduction!Proof!Core!RecoveryClosedPrefix(c)
249     BY <3>5
250 <3>7. QED
251     BY <3>5
252 <2>2. QED
253     BY <2>1
254     DEF Totality!RecoveryRecurringClosedPrefixes,
255     Totality!Reduction!Proof!RecoveryRecurringClosedPrefixes
256 <1>2. QED
257     BY <1>1
258
259 THEOREM UnconditionalClassicalAgreementFromTotalHistoryTheorem ==
260     Totality!TotalClassicalAgreementHistoryRealized
261     => UnconditionalClassicalAgreementSentence
262 PROOF
263 <1>1. ASSUME Totality!TotalClassicalAgreementHistoryRealized
264     PROVE UnconditionalClassicalAgreementSentence
265 <2>1. ClassicalAgreementHistory(Totality!ClassicalAgreementTotalHistoryObject)
266     BY TotalHistoryMatchesCollapsedClassicalHistory
267 <2>2. <>Totality!RecoveryClosedPrefix(TotalCycles)
268     BY <1>1 DEF Totality!TotalClassicalAgreementHistoryRealized
269 <2>3. QED
270     BY <2>1, <2>2
271     DEF UnconditionalClassicalAgreementSentence,
272     OrdinaryClassicalAgreementHistory
273 <1>2. QED
274     BY <1>1
275
276 THEOREM UnconditionalClassicalAgreementFromRecurringPrefixesTheorem ==
277     Totality!RecoveryRecurringClosedPrefixes
278     => UnconditionalClassicalAgreementSentence
279 PROOF
280 <1>1. ASSUME Totality!RecoveryRecurringClosedPrefixes
281     PROVE UnconditionalClassicalAgreementSentence
282 <2>1. Totality!Reduction!FiniteClassicalAgreementLiveness
283     BY LocalReductionLivenessFromRecurringPrefixesTheorem, <1>1
284 <2>2. Totality!TotalClassicalAgreementHistoryRealized
285     BY LocalTotalClassicalAgreementHistoryFromFiniteLivenessTheorem, <2>1
286 <2>3. QED
287     BY UnconditionalClassicalAgreementFromTotalHistoryTheorem, <2>2
288 <1>2. QED
289     BY <1>1
290
291 THEOREM UnconditionalClassicalAgreementFromRecurrenceKernelTheorem ==
292     /\ Totality!RecoveryRecurringInductionPremises
293     /\ Totality!RecoveryRecurringInductionKernel

```

```

294   => UnconditionalClassicalAgreementSentence
295 PROOF
296   <1>1. ASSUME Totality!RecoveryRecurringInductionPremises,
297           Totality!RecoveryRecurringInductionKernel
298           PROVE UnconditionalClassicalAgreementSentence
299   <2>1. Totality!RecoveryRecurringClosedPrefixes
300       BY LocalRecurringPrefixesFromRecurrenceKernelTheorem, <1>1
301   <2>2. Totality!Reduction!FiniteClassicalAgreementLiveness
302       BY LocalReductionLivenessFromRecurringPrefixesTheorem, <2>1
303   <2>3. Totality!TotalClassicalAgreementHistoryRealized
304       BY LocalTotalClassicalAgreementHistoryFromFiniteLivenessTheorem, <2>2
305   <2>4. QED
306       BY UnconditionalClassicalAgreementFromTotalHistoryTheorem, <2>3
307   <1>3. QED
308       BY <1>1
309
310   ====

```

A.7 Common Boundary (AFT-CB)

The mechanized kernels of the common-boundary theorem corpus (Section 14): the Boundary Ring uniqueness kernel and its TLAPS discharge, the liveness and two-tier separation model, the membership-transition and custody-obligation kernels with their proofs, the wire-level forensic-accountability kernel, and the honestly narrowed succession clock (its full safety claim withdrawn; the kernel header states the narrowing).

A.7.1 BoundaryRing.tla

Repository path. `\detokenize{internal-docs/architecture/protocols/aft/formal/common_boundary/BoundaryRing.tla}`

```
1  ---- MODULE BoundaryRing ----
2  (*****
3  (* AFT-CB P2.1 -- the Boundary Ring action-level uniqueness kernel.          *)
4  (*                                                                           *)
5  (* Models one signer configuration (Ring) forming Unanimous Boundary        *)
6  (* Closes per slot. The adversary schedules all delivery (no synchrony      *)
7  (* appears anywhere) and emits arbitrary well-signed messages for corrupt   *)
8  (* members -- the ONLY exclusion is forging an honest member's signature  *)
9  (* (A1), modeled by HonestFinalAck being the sole producer of honest ack    *)
10 (* records. Honest final-acks are guarded by a durable single-entry         *)
11 (* journal (A3). Uniqueness is DERIVED from the inductive invariant under   *)
12 (* the Minimal Honesty Axiom (A2) -- no admitted-equals-canonical          *)
13 (* definition exists in this module (the Q2 defect class this kernel        *)
14 (* exists to kill).                                                         *)
15 (*                                                                           *)
16 (* Spec source: specs/common_boundary.md §§0-2, §11-§12; theorem T1 in      *)
17 (* specs/common_boundary_theorems.md. The TLAPS discharge lives in         *)
18 (* BoundaryRingProof.tla; the TLC configuration explores n = 3 with         *)
19 (* exactly one honest member (the MHA corner).                             *)
20 (*****)
21 EXTENDS Naturals, FiniteSets
22
23 CONSTANTS
24   Ring,           \* the signer configuration C_v
25   HonestMembers,  \* the honest subset (MHA: nonempty -- assumed in the proof)
26   Slots,          \* seal slots s
27   Roots,          \* candidate boundary roots
28   Artifacts,      \* submittable artifacts (COLLECT/RECONCILE structure)
29   NoRoot          \* journal sentinel: "no final-ack recorded for this slot"
30
31 ASSUME ConstantAssumptions ==
32   /\ HonestMembers \subseq Ring
33   /\ NoRoot \notin Roots
34
35 VARIABLES
36   delivered, \* subset of Artifacts \X Ring: what the adversary delivered where
37   declared,  \* [Ring -> [Slots -> SUBSET Artifacts]]: declared sets
38   journal,   \* [HonestMembers -> [Slots -> Roots \cup {NoRoot}]]: A3 journal
39   acks,      \* subset of Ring \X Slots \X Roots: final-ack shares in existence
40   closes,    \* subset of Slots \X Roots: assembled UBCs
41
42 vars == <<delivered, declared, journal, acks, closes>>
43
```

```

44 TypeOK ==
45   /\ delivered \subsepeq Artifacts \X Ring
46   /\ declared \in [Ring -> [Slots -> SUBSET Artifacts]]
47   /\ journal \in [HonestMembers -> [Slots -> Roots \cup {NoRoot}]]
48   /\ acks \subsepeq Ring \X Slots \X Roots
49   /\ closes \subsepeq Slots \X Roots
50
51 Init ==
52   /\ delivered = {}
53   /\ declared = [m \in Ring |-> [s \in Slots |-> {}]]
54   /\ journal = [m \in HonestMembers |-> [s \in Slots |-> NoRoot]]
55   /\ acks = {}
56   /\ closes = {}
57
58 (*****
59 (* COLLECT: the adversary schedules delivery of any artifact to any      *)
60 (* member, in any order, at any time.  No fairness, no bound -- this is  *)
61 (* where asynchrony lives.                                              *)
62 (*****
63 Deliver(a, m) ==
64   /\ delivered' = delivered \cup {<<a, m>>}
65   /\ UNCHANGED <<declared, journal, acks, closes>>
66
67 (*****
68 (* RECONCILE/DECLARE: an honest member's declared set for a slot is drawn *)
69 (* from what it actually received (spec §3.1: cutoff is local -- the    *)
70 (* declared set is whatever was delivered when the member declares).    *)
71 (* Corrupt members' declared sets are set arbitrarily by the adversary. *)
72 (*****
73 HonestDeclare(m, s, A) ==
74   /\ m \in HonestMembers
75   /\ A \subsepeq {a \in Artifacts : <<a, m>> \in delivered}
76   /\ declared' = [declared EXCEPT ![m] = [declared[m] EXCEPT ![s] = A]]
77   /\ UNCHANGED <<delivered, journal, acks, closes>>
78
79 ByzantineDeclare(m, s, A) ==
80   /\ m \in Ring \ HonestMembers
81   /\ A \subsepeq Artifacts
82   /\ declared' = [declared EXCEPT ![m] = [declared[m] EXCEPT ![s] = A]]
83   /\ UNCHANGED <<delivered, journal, acks, closes>>
84
85 (*****
86 (* FINAL-ACK, honest: journal-guarded (A3).  The journal entry is written *)
87 (* in the same atomic step that lets the share exist -- spec §2.1's      *)
88 (* "journal before the share leaves the process".  The guard              *)
89 (* journal[m][s] = NoRoot is the single-final-ack discipline: once a root *)
90 (* is journaled for (m, s), no second honest share for that slot can ever *)
91 (* be produced (recovery replays bytes; it never re-decides -- §2.2).    *)
92 (*****
93 HonestFinalAck(m, s, r) ==
94   /\ m \in HonestMembers
95   /\ r \in Roots
96   /\ s \in Slots
97   /\ journal[m][s] = NoRoot
98   /\ journal' = [journal EXCEPT ![m] = [journal[m] EXCEPT ![s] = r]]
99   /\ acks' = acks \cup {<<m, s, r>>}
100   /\ UNCHANGED <<delivered, declared, closes>>
101
102 (*****
103 (* The unconstrained Byzantine action: a corrupt member emits ANY      *)
104 (* well-formed signed message, for any slot, any root, any number of    *)
105 (* times, on any schedule.  Forging an honest signature is the only      *)

```

```

106 (* exclusion (A1): m ranges over corrupt members only. No journal binds *)
107 (* corrupt members. *)
108 (*****
109 ByzantineEmit(m, s, r) ==
110   /\ m \in Ring \ HonestMembers
111   /\ s \in Slots
112   /\ r \in Roots
113   /\ acks' = acks \cup {<<m, s, r>>}
114   /\ UNCHANGED <<delivered, declared, journal, closes>>
115
116 (*****
117 (* CLOSE: anyone (the adversary included) may assemble a UBC from shares *)
118 (* that exist -- n-of-n: EVERY ring member's share over this exact *)
119 (* (slot, root). There is no smaller quorum anywhere in this module (the *)
120 (* (n-1)-close mutation drill targets exactly this conjunct). *)
121 (*****
122 CloseAct(s, r) ==
123   /\ s \in Slots
124   /\ r \in Roots
125   /\ \A m \in Ring : <<m, s, r>> \in acks
126   /\ closes' = closes \cup {<<s, r>>}
127   /\ UNCHANGED <<delivered, declared, journal, acks>>
128
129 Next ==
130   \/ \E a \in Artifacts, m \in Ring : Deliver(a, m)
131   \/ \E m \in Ring, s \in Slots, A \in SUBSET Artifacts :
132     HonestDeclare(m, s, A) \/ ByzantineDeclare(m, s, A)
133   \/ \E m \in Ring, s \in Slots, r \in Roots :
134     HonestFinalAck(m, s, r) \/ ByzantineEmit(m, s, r)
135   \/ \E s \in Slots, r \in Roots : CloseAct(s, r)
136
137 Spec == Init /\ [] [Next]_vars
138
139 (*****
140 (* The derived property. Uniqueness is NOT assumed anywhere above: no *)
141 (* definition equates "admitted" with "canonical"; closes is an ordinary *)
142 (* set any adversary schedule can try to populate twice. *)
143 (*****
144 Uniqueness ==
145   \A s \in Slots, r1 \in Roots, r2 \in Roots :
146     (<<s, r1>> \in closes /\ <<s, r2>> \in closes) => r1 = r2
147
148 (*****
149 (* The inductive invariant (discharged in BoundaryRingProof.tla): *)
150 (* an honest share always matches the journal entry (so an honest member *)
151 (* has at most one share per slot), and every close carries every *)
152 (* member's share. Uniqueness follows under MHA. *)
153 (*****
154 Inv ==
155   /\ TypeOK
156   /\ \A m \in HonestMembers, s \in Slots, r \in Roots :
157     (<<m, s, r>> \in acks) => journal[m][s] = r
158   /\ \A s \in Slots, r \in Roots :
159     (<<s, r>> \in closes) => \A m \in Ring : <<m, s, r>> \in acks
160
161 ====

```

A.7.2 BoundaryRingProof.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/common_boundary/BoundaryRingProof.tla}

```

1  ---- MODULE BoundaryRingProof ----
2  (*****
3  (* AFT-CB P2.1 -- TLAPS discharge of Boundary Ring uniqueness (T1).          *)
4  (*                                                                           *)
5  (* Structure: Init => Inv; Inv /\ [Next]_vars => Inv'; Inv /\ MHA =>          *)
6  (* Uniqueness; composed to THEOREM Spec => []Uniqueness under the MHA      *)
7  (* constant assumption. Uniqueness is derived, never defined into          *)
8  (* existence.                                                                *)
9  (*****
10 EXTENDS BoundaryRing, TLAPS
11
12 ASSUME MHA == HonestMembers # {}
13
14 LEMMA InvInit == Init => Inv
15   BY ConstantAssumptions DEF Init, Inv, TypeOK
16
17 LEMMA InvNext == Inv /\ [Next]_vars => Inv'
18 <1>1. SUFFICES ASSUME Inv, [Next]_vars PROVE Inv'
19   OBVIOUS
20 <1>2. CASE UNCHANGED vars
21   BY <1>1, <1>2 DEF Inv, TypeOK, vars
22 <1>3. ASSUME NEW a \in Artifacts, NEW m \in Ring, Deliver(a, m)
23   PROVE Inv'
24   BY <1>1, <1>3 DEF Inv, TypeOK, Deliver
25 <1>4. ASSUME NEW m \in Ring, NEW s \in Slots, NEW A \in SUBSET Artifacts,
26   HonestDeclare(m, s, A)
27   PROVE Inv'
28   BY <1>1, <1>4 DEF Inv, TypeOK, HonestDeclare
29 <1>5. ASSUME NEW m \in Ring, NEW s \in Slots, NEW A \in SUBSET Artifacts,
30   ByzantineDeclare(m, s, A)
31   PROVE Inv'
32   BY <1>1, <1>5 DEF Inv, TypeOK, ByzantineDeclare
33 <1>6. ASSUME NEW m \in Ring, NEW s \in Slots, NEW r \in Roots,
34   HonestFinalAck(m, s, r)
35   PROVE Inv'
36 <2>1. m \in HonestMembers /\ journal[m][s] = NoRoot
37   BY <1>6 DEF HonestFinalAck
38 <2>2. TypeOK'
39   BY <1>1, <1>6, <2>1 DEF Inv, TypeOK, HonestFinalAck
40 <2>3. \A mm \in HonestMembers, ss \in Slots, rr \in Roots :
41   (<<mm, ss, rr>> \in acks') => journal'[mm][ss] = rr
42 <3>1. SUFFICES ASSUME NEW mm \in HonestMembers, NEW ss \in Slots,
43   NEW rr \in Roots, <<mm, ss, rr>> \in acks'
44   PROVE journal'[mm][ss] = rr
45   OBVIOUS
46 <3>2. acks' = acks \cup {<<m, s, r>>}
47   BY <1>6 DEF HonestFinalAck
48 <3>3. CASE <<mm, ss, rr>> = <<m, s, r>>
49   <4>1. mm = m /\ ss = s /\ rr = r
50   BY <3>3
51   <4>2. journal'[m][s] = r
52   BY <1>1, <1>6, <2>1 DEF Inv, TypeOK, HonestFinalAck
53   <4> QED BY <4>1, <4>2
54 <3>4. CASE <<mm, ss, rr>> \in acks
55   <4>1. journal[mm][ss] = rr

```

```

56      BY <1>1, <3>4 DEF Inv
57      <4>2. CASE mm = m /\ ss = s
58      <5>1. journal[m][s] = rr
59      BY <4>1, <4>2
60      <5>2. rr \in Roots /\ NoRoot \notin Roots
61      BY <3>1, ConstantAssumptions
62      <5>3. FALSE
63      BY <2>1, <5>1, <5>2
64      <5> QED BY <5>3
65      <4>3. CASE ~(mm = m /\ ss = s)
66      <5>1. journal'[mm][ss] = journal[mm][ss]
67      BY <1>1, <1>6, <2>1, <4>3 DEF Inv, TypeOK, HonestFinalAck
68      <5> QED BY <4>1, <5>1
69      <4> QED BY <4>2, <4>3
70      <3> QED BY <3>1, <3>2, <3>3, <3>4
71      <2>4. \A ss \in Slots, rr \in Roots :
72      (<<ss, rr>> \in closes') => \A mm \in Ring : <<mm, ss, rr>> \in acks'
73      BY <1>1, <1>6 DEF Inv, HonestFinalAck
74      <2> QED BY <2>2, <2>3, <2>4 DEF Inv
75      <1>7. ASSUME NEW m \in Ring, NEW s \in Slots, NEW r \in Roots,
76      ByzantineEmit(m, s, r)
77      PROVE Inv'
78      <2>1. m \notin HonestMembers
79      BY <1>7 DEF ByzantineEmit
80      <2>2. TypeOK'
81      BY <1>1, <1>7 DEF Inv, TypeOK, ByzantineEmit
82      <2>3. \A mm \in HonestMembers, ss \in Slots, rr \in Roots :
83      (<<mm, ss, rr>> \in acks') => journal'[mm][ss] = rr
84      BY <1>1, <1>7, <2>1 DEF Inv, ByzantineEmit
85      <2>4. \A ss \in Slots, rr \in Roots :
86      (<<ss, rr>> \in closes') => \A mm \in Ring : <<mm, ss, rr>> \in acks'
87      BY <1>1, <1>7 DEF Inv, ByzantineEmit
88      <2> QED BY <2>2, <2>3, <2>4 DEF Inv
89      <1>8. ASSUME NEW s \in Slots, NEW r \in Roots, CloseAct(s, r)
90      PROVE Inv'
91      <2>1. TypeOK'
92      BY <1>1, <1>8 DEF Inv, TypeOK, CloseAct
93      <2>2. \A mm \in HonestMembers, ss \in Slots, rr \in Roots :
94      (<<mm, ss, rr>> \in acks') => journal'[mm][ss] = rr
95      BY <1>1, <1>8 DEF Inv, CloseAct
96      <2>3. \A ss \in Slots, rr \in Roots :
97      (<<ss, rr>> \in closes') => \A mm \in Ring : <<mm, ss, rr>> \in acks'
98      <3>1. SUFFICES ASSUME NEW ss \in Slots, NEW rr \in Roots,
99      <<ss, rr>> \in closes'
100      PROVE \A mm \in Ring : <<mm, ss, rr>> \in acks'
101      OBVIOUS
102      <3>2. closes' = closes \cup {<<s, r>>} /\ acks' = acks
103      BY <1>8 DEF CloseAct
104      <3>3. CASE <<ss, rr>> \in closes
105      BY <1>1, <3>2, <3>3 DEF Inv
106      <3>4. CASE <<ss, rr>> = <<s, r>>
107      BY <1>8, <3>2, <3>4 DEF CloseAct
108      <3> QED BY <3>1, <3>2, <3>3, <3>4
109      <2> QED BY <2>1, <2>2, <2>3 DEF Inv
110      <1>9. QED
111      BY <1>1, <1>2, <1>3, <1>4, <1>5, <1>6, <1>7, <1>8 DEF Next, vars
112
113      LEMMA InvUniqueness == Inv => Uniqueness
114      <1>1. SUFFICES ASSUME Inv, NEW s \in Slots, NEW r1 \in Roots, NEW r2 \in Roots,
115      <<s, r1>> \in closes, <<s, r2>> \in closes
116      PROVE r1 = r2
117      BY DEF Uniqueness

```

```

118 <1>2. PICK h \in HonestMembers : TRUE
119     BY MHA
120 <1>3. h \in Ring
121     BY ConstantAssumptions
122 <1>4. <<h, s, r1>> \in acks /\ <<h, s, r2>> \in acks
123     BY <1>1, <1>3 DEF Inv
124 <1>5. journal[h][s] = r1 /\ journal[h][s] = r2
125     BY <1>1, <1>2, <1>4 DEF Inv
126 <1>6. QED
127     BY <1>5
128
129 THEOREM BoundaryUniqueness == Spec => []Uniqueness
130 <1>1. Init => Inv
131     BY InvInit
132 <1>2. Inv /\ [Next]_vars => Inv'
133     BY InvNext
134 <1>3. Spec => []Inv
135     BY <1>1, <1>2, PTL DEF Spec
136 <1>4. Inv => Uniqueness
137     BY InvUniqueness
138 <1>5. QED
139     BY <1>3, <1>4, PTL
140
141 ====

```

A.7.3 BoundaryLiveness.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/common_boundary/BoundaryLiveness.tla}

```

1  ---- MODULE BoundaryLiveness ----
2  (*****
3  (* AFT-CB P2.2 -- SEAL CADENCE, not chain liveness (T4b), and the two-tier *)
4  (* separation property. *)
5  (* *)
6  (* Claim scope (standing rule 2, stated exactly): the cadence and *)
7  (* separation results below are model-checked at n≤4 -- never "proven". *)
8  (* Live-tier liveness (T4a) is the engine's own obligation, tracked at *)
9  (* R10 and deliberately NOT claimed here: LiveBlock below is a progress *)
10 (* ABSTRACTION that reads no ring variable, so the separation property is *)
11 (* visible by construction and checked by TLC. *)
12 (* *)
13 (* A5 is modeled as FAIRNESS: weak fairness on responsive members' acks *)
14 (* and on seal assembly is exactly the post-GST eventual-delivery *)
15 (* abstraction (T4b's Assumes line carries A5; this module says where it *)
16 (* lives). A2/A3's safety content is BoundaryRing's business (P2.1) -- *)
17 (* this module checks progress, with honest-shaped members only. *)
18 (* *)
19 (* The action set includes the assurance-preserving handover (§9): an *)
20 (* unresponsive member can be replaced by a responsive standby when *)
21 (* HandoverEnabled -- modeled as an available transition; its safety terms *)
22 (* (old-ring unanimity + new-ring acceptance) are P2.3's obligation, not *)
23 (* re-proved here. NO EJECTION ACTION EXISTS AT THIS LAYER: weighted *)
24 (* ejection is live-tier-only (spec §5), and the strong ring changes only *)
25 (* by handover here. *)
26 (* *)
27 (* Mutation drill (leg-mandated): one member permanently silent with *)
28 (* handover disabled ⇒ SealCadenceReached goes RED while *)
29 (* LiveTierProgressReached stays GREEN -- both directions of the tier *)

```

```

30 (* split, demonstrated by the same counterexample run. *)
31 (*****)
32 EXTENDS Naturals, FiniteSets
33
34 CONSTANTS
35   Members,          \* the initial ring configuration
36   Standbys,         \* standby pool for handover (disjoint from Members)
37   Responsive,       \* the responsive subset of Members \cup Standbys
38   MaxSeals,         \* cadence target the property demands be reached
39   MaxBlocks,        \* live-tier progress target
40   HandoverEnabled   \* BOOLEAN: is the §9 handover transition available?
41
42 ASSUME LivenessConstants ==
43   /\ Standbys \cap Members = {}
44   /\ Responsive \subseq (Members \cup Standbys)
45   /\ MaxSeals \in Nat \ {0}
46   /\ MaxBlocks \in Nat \ {0}
47   /\ HandoverEnabled \in BOOLEAN
48
49 VARIABLES
50   ring,             \* current configuration (changes only by handover)
51   acked,            \* members of ring that final-acked the CURRENT slot
52   sealCount,        \* seals closed so far (the cadence measure)
53   liveBlocks,       \* live-tier progress abstraction (reads no ring state)
54   version           \* configuration version (increments on handover)
55
56 vars == <<ring, acked, sealCount, liveBlocks, version>>
57
58 TypeInvariant ==
59   /\ ring \subseq (Members \cup Standbys)
60   /\ acked \subseq ring
61   /\ sealCount \in 0..MaxSeals
62   /\ liveBlocks \in 0..MaxBlocks
63   /\ version \in Nat
64
65 Init ==
66   /\ ring = Members
67   /\ acked = {}
68   /\ sealCount = 0
69   /\ liveBlocks = 0
70   /\ version = 0
71
72 (* A responsive ring member final-acks the current slot. Unresponsive *)
73 (* members simply never take this action -- silence is the absence of a *)
74 (* step, not a message (spec §10's discipline holds even in the liveness *)
75 (* model). *)
76 Ack(m) ==
77   /\ m \in ring
78   /\ m \in Responsive
79   /\ m \notin acked
80   /\ sealCount < MaxSeals
81   /\ acked' = acked \cup {m}
82   /\ UNCHANGED <<ring, sealCount, liveBlocks, version>>
83
84 (* The n-of-n close: EVERY current ring member has acked. Unanimity is *)
85 (* the whole point -- one silent member freezes this action forever (L2's *)
86 (* forced trade, conceded by T4b). *)
87 SealClose ==
88   /\ acked = ring
89   /\ ring # {}
90   /\ sealCount < MaxSeals
91   /\ sealCount' = sealCount + 1

```

```

92  /\ acked' = {}
93  /\ UNCHANGED <<ring, liveBlocks, version>>
94
95  (* The live tier produces blocks reading NO ring variable: the guard and *)
96  (* effect mention only liveBlocks. A stalled ring cannot disable this *)
97  (* action -- that is the separation property, structurally. *)
98  LiveBlock ==
99  /\ liveBlocks < MaxBlocks
100  /\ liveBlocks' = liveBlocks + 1
101  /\ UNCHANGED <<ring, acked, sealCount, version>>
102
103  (* Assurance-preserving handover (§9), abstracted to its liveness role: *)
104  (* an unresponsive member is replaced by a responsive standby and the *)
105  (* configuration version advances. Enabled only when the deployment *)
106  (* provides the ceremony (HandoverEnabled) -- the mutation drill disables *)
107  (* it to prove the cadence property fails for the right reason. *)
108  Handover(m, sb) ==
109  /\ HandoverEnabled
110  /\ m \in ring
111  /\ m \notin Responsive
112  /\ sb \in Standbys
113  /\ sb \in Responsive
114  /\ sb \notin ring
115  /\ ring' = (ring \ {m}) \cup {sb}
116  /\ acked' = acked \ {m}
117  /\ version' = version + 1
118  /\ UNCHANGED <<sealCount, liveBlocks>>
119
120  (* Terminal self-loop so a finished run is quiescence, not deadlock. *)
121  Terminating ==
122  /\ sealCount = MaxSeals
123  /\ liveBlocks = MaxBlocks
124  /\ UNCHANGED vars
125
126  SomeAck == \E m \in Members \cup Standbys : Ack(m)
127  SomeHandover == \E m \in Members, sb \in Standbys : Handover(m, sb)
128
129  Next ==
130  \/ SomeAck
131  \/ SealClose
132  \/ LiveBlock
133  \/ SomeHandover
134  \/ Terminating
135
136  (* Fairness IS the A5 abstraction: post-GST, enabled progress steps *)
137  (* eventually happen. Weak fairness per responsive participant's ack, *)
138  (* plus seal assembly, live production, and the handover ceremony when *)
139  (* available. *)
140  LivenessSpec ==
141  /\ Init
142  /\ [] [Next]_vars
143  /\ \A m \in Members \cup Standbys : WF_vars(Ack(m))
144  /\ WF_vars(SealClose)
145  /\ WF_vars(LiveBlock)
146  /\ WF_vars(SomeHandover)
147
148  (* The cadence property (T4b shape): seals actually reach the target -- *)
149  (* positive seals, not merely eventual aborts. *)
150  SealCadenceReached == <>(sealCount = MaxSeals)
151
152  (* The separation property: the live tier reaches its target regardless *)
153  (* of ring state -- checked GREEN in every configuration including the *)

```

```

154 (* mutation drill where cadence is RED. *)
155 LiveTierProgressReached == <>(liveBlocks = MaxBlocks)
156
157 ====

```

A.7.4 MembershipTransition.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/common_boundary/MembershipTransition.tla}

```

1  ---- MODULE MembershipTransition ----
2  (*****)
3  (* AFT-CB P2.3 -- membership transitions, the no-silence action set, and *)
4  (* typed lineage roots. *)
5  (* *)
6  (* Event-driven, versioned configurations: a transition record is closed *)
7  (* by the CURRENT configuration's unanimous transition-acks (old-ring *)
8  (* unanimity) plus every successor member's acceptance (T5c') -- the only *)
9  (* strong-ring transition action in this module. There is NO calendar *)
10 (* epoch anywhere: nothing advances by time. *)
11 (* *)
12 (* Silence records EXIST in this model -- the adversary mints attested *)
13 (* non-response records freely -- and can never justify a transition: the *)
14 (* TransitionsJustified invariant states that every closed transition is *)
15 (* fully signature-justified (acks + acceptances), which is the spec-level *)
16 (* no-silence-derived-action assertion the leg demands (spec §10.2). The *)
17 (* mutation drill adds a silence-justified closure action and watches this *)
18 (* invariant go red. *)
19 (* *)
20 (* Re-genesis is a TYPED ROOT: every transition record carries exactly one *)
21 (* lineage id, roots mint fresh lineage ids, and a well-formed history is *)
22 (* single-lineage BY TYPE -- a root-crossing history is distinguishable by *)
23 (* construction (spec §14.4). Uniqueness (T5a) quantifies WITHIN one *)
24 (* lineage, from a common parent configuration, under per-configuration *)
25 (* MHA stated as an explicit theorem condition. *)
26 (* *)
27 (* Bootstrap: the deployed A6 freshness anchor is an EXPLICIT CONSTANT *)
28 (* (DeployedAnchor), and the freshness theorem in the proof module is *)
29 (* conditional on it IN THE STATEMENT -- never in prose. *)
30 (*****)
31 EXTENDS Naturals, FiniteSets
32
33 CONSTANTS
34   AllMembers,      \* the member universe
35   HonestMembers,   \* honest subset (per-config MHA is a theorem condition)
36   Lineages,        \* lineage identifiers (roots mint them)
37   GenesisLineage,  \* the anchored genesis lineage
38   Versions,        \* transition version numbers (event-driven, no epochs)
39   NoSucc,          \* journal sentinel
40   Anchors,         \* the A6 anchor menu (explicit constants)
41   DeployedAnchor,  \* the deployed freshness anchor, or NoAnchor
42   NoAnchor
43
44 Configs == (SUBSET AllMembers) \ {}
45
46 ASSUME MembershipConstants ==
47   /\ HonestMembers \subseteqq AllMembers
48   /\ GenesisLineage \in Lineages
49   /\ NoAnchor \notin Anchors

```

```

50 /\ DeployedAnchor \in Anchors \cup {NoAnchor}
51 \* NoSucc \notin Configs is additionally assumed in the PROOF module only:
52 \* TLC cannot membership-test a model value against this set of sets, and
53 \* TLC's own model-value semantics already guarantee the disequality.
54
55 VARIABLES
56   tjournal,      \* [HonestMembers -> [Lineages -> [Versions -> Configs \cup {NoSucc}]]]
57   tacks,         \* HONEST old-ring transition final-acks: <<m, l, v, succ>>
58   taccepts,      \* HONEST new-ring seat acceptances: <<m, l, v, succ>>
59   transitions,   \* closed records: <<l, v, parent, succ>>
60   roots,         \* minted lineage roots: <<newLineage, priorLineage>>
61   silenceRecords \* attested non-response records: <<attester, subject>>
62
63 vars == <<tjournal, tacks, taccepts, transitions, roots, silenceRecords>>
64
65 (*****
66 (* The unconstrained adversary, modeled STRUCTURALLY: a corrupt member's *)
67 (* signature over ANY transition content is always available (A1 excludes *)
68 (* only forging honest signatures), so Byzantine acks and acceptances are *)
69 (* not state -- they are simply TRUE. Only honest signatures are state, *)
70 (* because only honest members refuse to sign twice (the journal). This *)
71 (* keeps the adversary maximal while keeping the state space enumerable. *)
72 (*****
73 AckedBy(m, l, v, succ) ==
74   IF m \in HonestMembers THEN <<m, l, v, succ>> \in tacks ELSE TRUE
75
76 AcceptedBy(m, l, v, succ) ==
77   IF m \in HonestMembers THEN <<m, l, v, succ>> \in taccepts ELSE TRUE
78
79 TypeOK ==
80   /\ tjournal \in [HonestMembers -> [Lineages -> [Versions -> Configs \cup {NoSucc}]]]
81   /\ tacks \subseq HonestMembers \X Lineages \X Versions \X Configs
82   /\ taccepts \subseq HonestMembers \X Lineages \X Versions \X Configs
83   /\ transitions \subseq Lineages \X Versions \X Configs \X Configs
84   /\ roots \subseq Lineages \X Lineages
85   /\ silenceRecords \subseq AllMembers \X AllMembers
86
87 Init ==
88   /\ tjournal = [m \in HonestMembers |> [l \in Lineages |> [v \in Versions |> NoSucc]]]
89   /\ tacks = {}
90   /\ taccepts = {}
91   /\ transitions = {}
92   /\ roots = {}
93   /\ silenceRecords = {}
94
95 (* Old-ring transition final-ack, journal-guarded per (lineage, version): *)
96 (* the A3 discipline at the transition layer -- an honest member acks at *)
97 (* most one successor per version of a lineage, ever. *)
98 HonestTransitionAck(m, l, v, succ) ==
99   /\ m \in HonestMembers
100   /\ l \in Lineages /\ v \in Versions /\ succ \in Configs
101   /\ tjournal[m][l][v] = NoSucc
102   /\ tjournal' = [tjournal EXCEPT ![m] =
103     [tjournal[m] EXCEPT ![l] = [tjournal[m][l] EXCEPT ![v] = succ]]]
104   /\ tacks' = tacks \cup {<<m, l, v, succ>>}
105   /\ UNCHANGED <<taccepts, transitions, roots, silenceRecords>>
106
107 (* Honest new-ring acceptance of a seat in a successor configuration the *)
108 (* member belongs to. Acceptance is not the safety bottleneck (the old *)
109 (* ring's unanimity is), so no journal binds it; corrupt members' *)
110 (* acceptances are always available (AcceptedBy). *)
111 Accept(m, l, v, succ) ==

```

```

112 /\ m \in HonestMembers
113 /\ l \in Lineages /\ v \in Versions /\ succ \in Configs
114 /\ m \in succ
115 /\ taccepts' = taccepts \cup {<<m, l, v, succ>>}
116 /\ UNCHANGED <<tjournal, tacks, transitions, roots, silenceRecords>>
117
118 (* THE ONLY strong-ring transition action: assurance-preserving handover -- *)
119 (* old-ring unanimity (every parent member's ack, with corrupt signatures *)
120 (* always available) AND new-ring acceptance (every successor member's). *)
121 (* No other action in this module writes `transitions`; in particular no *)
122 (* action consumes silenceRecords. *)
123 CloseTransition(l, v, parent, succ) ==
124 /\ l \in Lineages /\ v \in Versions
125 /\ parent \in Configs /\ succ \in Configs
126 /\ \A m \in parent : AckedBy(m, l, v, succ)
127 /\ \A m \in succ : AcceptedBy(m, l, v, succ)
128 /\ transitions' = transitions \cup {<<l, v, parent, succ>>}
129 /\ UNCHANGED <<tjournal, tacks, taccepts, roots, silenceRecords>>
130
131 (* Anchored re-genesis: a typed lineage ROOT. It mints a FRESH lineage id *)
132 (* -- never the genesis lineage, never an id already rooted -- and records *)
133 (* the prior lineage it administratively succeeds. Nothing about it is a *)
134 (* transition: it writes `roots`, not `transitions`, and the lineage seam *)
135 (* is visible in the type itself. *)
136 MintRoot(newL, priorL) ==
137 /\ newL \in Lineages /\ priorL \in Lineages
138 /\ newL # GenesisLineage
139 /\ newL # priorL
140 /\ \A p \in Lineages : <<newL, p>> \notin roots
141 /\ roots' = roots \cup {<<newL, priorL>>}
142 /\ UNCHANGED <<tjournal, tacks, taccepts, transitions, silenceRecords>>
143
144 (* Attested non-response records: ANYONE may mint one about ANY member, in *)
145 (* any quantity -- and the state machine gives them no power: no guard of *)
146 (* any transition-writing action mentions this variable. *)
147 EmitSilenceRecord(attester, subject) ==
148 /\ attester \in AllMembers /\ subject \in AllMembers
149 /\ silenceRecords' = silenceRecords \cup {<<attester, subject>>}
150 /\ UNCHANGED <<tjournal, tacks, taccepts, transitions, roots>>
151
152 Next ==
153 /\ \E m \in AllMembers, l \in Lineages, v \in Versions, succ \in Configs :
154 HonestTransitionAck(m, l, v, succ) /\ Accept(m, l, v, succ)
155 /\ \E l \in Lineages, v \in Versions, parent \in Configs, succ \in Configs :
156 CloseTransition(l, v, parent, succ)
157 /\ \E newL \in Lineages, priorL \in Lineages : MintRoot(newL, priorL)
158 /\ \E a \in AllMembers, s \in AllMembers : EmitSilenceRecord(a, s)
159
160 Spec == Init /\ [] [Next]_vars
161
162 (*****
163 (* The spec-level no-silence assertion: every closed transition is FULLY *)
164 (* justified by signatures -- old-ring unanimity and new-ring acceptance -- *)
165 (* while silence records float freely in the state with no effect. The *)
166 (* silence-mutation drill adds a closure action justified by *)
167 (* silenceRecords instead, and this invariant goes red. *)
168 (*****
169 TransitionsJustified ==
170 /\ l \in Lineages, v \in Versions, parent \in Configs, succ \in Configs :
171 (<<l, v, parent, succ>> \in transitions) =>
172 /\ \A m \in parent : AckedBy(m, l, v, succ)
173 /\ \A m \in succ : AcceptedBy(m, l, v, succ)

```

```

174
175 (* T5a at the transition layer: within ONE lineage, from a common parent *)
176 (* configuration containing at least one honest member, the closed *)
177 (* successor at any version is unique. Divergence convicts the FULL *)
178 (* parent configuration (every member double-acked) -- checked by TLC and *)
179 (* proven in MembershipTransitionProof. *)
180 LineageUniqueness ==
181   \A l \in Lineages, v \in Versions, parent \in Configs,
182     s1 \in Configs, s2 \in Configs :
183     /\ <<l, v, parent, s1>> \in transitions
184     /\ <<l, v, parent, s2>> \in transitions
185     /\ parent \cap HonestMembers # {} => s1 = s2
186
187 (* Root-crossing distinguishability (spec §14.4): every record carries its *)
188 (* lineage id in the first component, so a verifier's well-formedness rule *)
189 (* -- a single history holds records of ONE lineage -- makes a root-crossing *)
190 (* history distinguishable by construction. WellFormedHistory is the *)
191 (* verifier's rule; CrossLineageUnclaimed states what the model does NOT *)
192 (* claim: uniqueness never quantifies across lineages, and the drill that *)
193 (* mutates WellFormedHistory to ignore lineage ids watches a cross-lineage *)
194 (* "uniqueness" invariant collapse immediately (two lineages may close *)
195 (* different successors at the same version, legitimately). *)
196 WellFormedHistory(H) ==
197   \E l \in Lineages : \A r \in H : r[1] = l
198
199 CrossLineageUniquenessWouldBeFalse ==
200   \A H \in SUBSET transitions :
201     WellFormedHistory(H) =>
202       \A r1 \in H, r2 \in H, v \in Versions :
203         (r1[2] = v /\ r2[2] = v /\ r1[3] = r2[3]
204          /\ r1[3] \cap HonestMembers # {}) => r1[4] = r2[4]
205
206 Inv ==
207   /\ TypeOK
208   /\ \A m \in HonestMembers, l \in Lineages, v \in Versions, succ \in Configs :
209     (<<m, l, v, succ>> \in tacks) => tjournal[m][l][v] = succ
210   /\ TransitionsJustified
211
212 ====

```

A.7.5 MembershipTransitionProof.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/common_boundary/MembershipTransitionProof.tla}

```

1 ---- MODULE MembershipTransitionProof ----
2 (*****
3 (* AFT-CB P2.3 -- TLAPS discharge: T5a lineage uniqueness under *)
4 (* per-configuration MHA (stated as a theorem CONDITION, not a global *)
5 (* assumption), the signature-justification invariant (the no-silence *)
6 (* assertion), and the A6-conditional bootstrap-freshness statement -- *)
7 (* conditionality in the theorem, never in prose. *)
8 (*****
9 EXTENDS MembershipTransition, TLAPS
10
11 (* Assumed here rather than in the model module: TLC cannot evaluate this *)
12 (* membership test over a model value, while its model-value semantics *)
13 (* already make it true; tlpm consumes it for the journal-conflict step. *)
14 ASSUME NoSuccOutsideConfigs == NoSucc \notin Configs

```

```

15
16 LEMMA MInvInit == Init => Inv
17   BY MembershipConstants DEF Init, Inv, TypeOK, TransitionsJustified
18
19 LEMMA MInvNext == Inv /\ [Next]_vars => Inv'
20 <1>1. SUFFICES ASSUME Inv, [Next]_vars PROVE Inv'
21   OBVIOUS
22 <1>2. CASE UNCHANGED vars
23   BY <1>1, <1>2 DEF Inv, TypeOK, TransitionsJustified, AckedBy, AcceptedBy,
24     vars
25 <1>3. ASSUME NEW m \in AllMembers, NEW l \in Lineages, NEW v \in Versions,
26     NEW succ \in Configs, HonestTransitionAck(m, l, v, succ)
27   PROVE Inv'
28   <2>1. m \in HonestMembers /\ tjournal[m][l][v] = NoSucc
29     BY <1>3 DEF HonestTransitionAck
30   <2>2. TypeOK'
31     BY <1>1, <1>3, <2>1 DEF Inv, TypeOK, HonestTransitionAck
32   <2>3. \A mm \in HonestMembers, ll \in Lineages, vv \in Versions,
33     ss \in Configs :
34     (<<mm, ll, vv, ss>> \in tacks') => tjournal'[mm][ll][vv] = ss
35   <3>1. SUFFICES ASSUME NEW mm \in HonestMembers, NEW ll \in Lineages,
36     NEW vv \in Versions, NEW ss \in Configs,
37     <<mm, ll, vv, ss>> \in tacks'
38     PROVE tjournal'[mm][ll][vv] = ss
39   OBVIOUS
40   <3>2. tacks' = tacks \cup {<<m, l, v, succ>>}
41     BY <1>3 DEF HonestTransitionAck
42   <3>3. CASE <<mm, ll, vv, ss>> = <<m, l, v, succ>>
43     <4>1. mm = m /\ ll = l /\ vv = v /\ ss = succ
44     BY <3>3
45     <4>2. tjournal'[m][l][v] = succ
46     BY <1>1, <1>3, <2>1 DEF Inv, TypeOK, HonestTransitionAck
47     <4> QED BY <4>1, <4>2
48   <3>4. CASE <<mm, ll, vv, ss>> \in tacks
49     <4>1. tjournal'[mm][ll][vv] = ss
50     BY <1>1, <3>4 DEF Inv
51     <4>2. CASE mm = m /\ ll = l /\ vv = v
52     <5>1. tjournal'[m][l][v] = ss
53     BY <4>1, <4>2
54     <5>2. tjournal'[m][l][v] = NoSucc
55     BY <2>1
56     <5>3. ss # NoSucc
57     BY <3>1, NoSuccOutsideConfigs
58     <5> QED BY <5>1, <5>2, <5>3
59     <4>3. CASE ~(mm = m /\ ll = l /\ vv = v)
60     <5>1. tjournal'[mm][ll][vv] = tjournal'[mm][ll][vv]
61     BY <1>1, <1>3, <2>1, <4>3 DEF Inv, TypeOK, HonestTransitionAck
62     <5> QED BY <4>1, <5>1
63     <4> QED BY <4>2, <4>3
64   <3> QED BY <3>1, <3>2, <3>3, <3>4
65 <2>4. TransitionsJustified'
66   <3>1. SUFFICES ASSUME NEW ll \in Lineages, NEW vv \in Versions,
67     NEW pp \in Configs, NEW ss \in Configs,
68     <<ll, vv, pp, ss>> \in transitions'
69     PROVE /\ \A mm \in pp : AckedBy(mm, ll, vv, ss)'
70     /\ \A mm \in ss : AcceptedBy(mm, ll, vv, ss)'
71   BY DEF TransitionsJustified
72   <3>2. transitions' = transitions /\ tacks' = tacks \cup {<<m, l, v, succ>>}
73     /\ taccepts' = taccepts
74   BY <1>3 DEF HonestTransitionAck
75   <3>3. /\ \A mm \in pp : AckedBy(mm, ll, vv, ss)
76     /\ \A mm \in ss : AcceptedBy(mm, ll, vv, ss)

```

```

77     BY <1>1, <3>1, <3>2 DEF Inv, TransitionsJustified
78     <3> QED BY <3>1, <3>2, <3>3 DEF AckedBy, AcceptedBy
79     <2> QED BY <2>2, <2>3, <2>4 DEF Inv
80 <1>5. ASSUME NEW m \in AllMembers, NEW l \in Lineages, NEW v \in Versions,
81     NEW succ \in Configs, Accept(m, l, v, succ)
82     PROVE Inv'
83     <2>1. TypeOK'
84     BY <1>1, <1>5 DEF Inv, TypeOK, Accept
85     <2>2. \A mm \in HonestMembers, ll \in Lineages, vv \in Versions,
86         ss \in Configs :
87         (<<mm, ll, vv, ss>> \in tacks') => tjournal'[mm][ll][vv] = ss
88     BY <1>1, <1>5 DEF Inv, Accept
89     <2>3. TransitionsJustified'
90     <3>1. SUFFICES ASSUME NEW ll \in Lineages, NEW vv \in Versions,
91         NEW pp \in Configs, NEW ss \in Configs,
92         <<ll, vv, pp, ss>> \in transitions'
93         PROVE /\ \A mm \in pp : AckedBy(mm, ll, vv, ss)'
94         /\ \A mm \in ss : AcceptedBy(mm, ll, vv, ss)'
95     BY DEF TransitionsJustified
96     <3>2. transitions' = transitions /\ tacks' = tacks
97         /\ taccepts' = taccepts \cup {<m, l, v, succ>}
98     BY <1>5 DEF Accept
99     <3>3. /\ \A mm \in pp : AckedBy(mm, ll, vv, ss)
100        /\ \A mm \in ss : AcceptedBy(mm, ll, vv, ss)
101     BY <1>1, <3>1, <3>2 DEF Inv, TransitionsJustified
102     <3> QED BY <3>1, <3>2, <3>3 DEF AckedBy, AcceptedBy
103     <2> QED BY <2>1, <2>2, <2>3 DEF Inv
104 <1>6. ASSUME NEW l \in Lineages, NEW v \in Versions, NEW parent \in Configs,
105     NEW succ \in Configs, CloseTransition(l, v, parent, succ)
106     PROVE Inv'
107     <2>1. TypeOK'
108     BY <1>1, <1>6 DEF Inv, TypeOK, CloseTransition
109     <2>2. \A mm \in HonestMembers, ll \in Lineages, vv \in Versions,
110         ss \in Configs :
111         (<<mm, ll, vv, ss>> \in tacks') => tjournal'[mm][ll][vv] = ss
112     BY <1>1, <1>6 DEF Inv, TransitionsJustified, CloseTransition
113     <2>3. TransitionsJustified'
114     BY <1>1, <1>6 DEF Inv, TransitionsJustified, CloseTransition,
115         AckedBy, AcceptedBy
116     <2> QED BY <2>1, <2>2, <2>3 DEF Inv
117 <1>7. ASSUME NEW newL \in Lineages, NEW priorL \in Lineages,
118     MintRoot(newL, priorL)
119     PROVE Inv'
120     BY <1>1, <1>7 DEF Inv, TypeOK, TransitionsJustified, AckedBy, AcceptedBy,
121         MintRoot
122 <1>8. ASSUME NEW a \in AllMembers, NEW s \in AllMembers,
123     EmitSilenceRecord(a, s)
124     PROVE Inv'
125     BY <1>1, <1>8 DEF Inv, TypeOK, TransitionsJustified, AckedBy, AcceptedBy,
126         EmitSilenceRecord
127 <1>9. QED
128     BY <1>1, <1>2, <1>3, <1>5, <1>6, <1>7, <1>8 DEF Next, vars
129
130 (*****
131 (* T5a: within one lineage, from a common parent configuration that *)
132 (* contains at least one honest member (per-configuration MHA as an *)
133 (* explicit CONDITION of the statement), closed successors are unique. *)
134 (*****
135 LEMMA InvLineageUniqueness == Inv => LineageUniqueness
136 <1>1. SUFFICES ASSUME Inv, NEW l \in Lineages, NEW v \in Versions,
137     NEW parent \in Configs,
138     NEW s1 \in Configs, NEW s2 \in Configs,

```

```

139         <<l, v, parent, s1>> \in transitions,
140         <<l, v, parent, s2>> \in transitions,
141         parent \cap HonestMembers # {}
142     PROVE s1 = s2
143     BY DEF LineageUniqueness
144 <1>2. PICK h \in parent \cap HonestMembers : TRUE
145     BY <1>1
146 <1>3. h \in parent /\ h \in HonestMembers
147     BY <1>2
148 <1>4. <<h, l, v, s1>> \in tacks /\ <<h, l, v, s2>> \in tacks
149     BY <1>1, <1>3 DEF Inv, TransitionsJustified, AckedBy
150 <1>5. tjournal[h][l][v] = s1 /\ tjournal[h][l][v] = s2
151     BY <1>1, <1>3, <1>4 DEF Inv
152 <1>6. QED
153     BY <1>5
154
155 THEOREM MembershipSafety == Spec => [] (LineageUniqueness /\ TransitionsJustified)
156 <1>1. Init => Inv
157     BY MInvInit
158 <1>2. Inv /\ [Next]_vars => Inv'
159     BY MInvNext
160 <1>3. Spec => [] Inv
161     BY <1>1, <1>2, PTL DEF Spec
162 <1>4. Inv => (LineageUniqueness /\ TransitionsJustified)
163     BY InvLineageUniqueness DEF Inv
164 <1>5. QED
165     BY <1>3, <1>4, PTL
166
167 (*****
168 (* Bootstrap freshness, A6-conditional IN THE STATEMENT: with a deployed *)
169 (* anchor, a verifier that partitions records by lineage id (well-formed *)
170 (* histories) obtains per-lineage uniqueness wherever the parent satisfies *)
171 (* MHA -- the anchor's job (selecting the live lineage among the *)
172 (* partitions) is exactly the deployed mechanism's own assumption, which *)
173 (* is why the implication's hypothesis names it. With NO anchor deployed *)
174 (* the theorem asserts nothing -- out-of-model, stated structurally. *)
175 (*****
176 THEOREM BootstrapFreshness ==
177     (DeployedAnchor \in Anchors) =>
178         (Spec => [] (\A l \in Lineages, v \in Versions, parent \in Configs,
179             s1 \in Configs, s2 \in Configs :
180                 /\ <<l, v, parent, s1>> \in transitions
181                 /\ <<l, v, parent, s2>> \in transitions
182                 /\ parent \cap HonestMembers # {}) => s1 = s2))
183 <1>1. SUFFICES ASSUME DeployedAnchor \in Anchors PROVE
184     Spec => [] LineageUniqueness
185     BY DEF LineageUniqueness
186 <1>2. Spec => [] (LineageUniqueness /\ TransitionsJustified)
187     BY MembershipSafety
188 <1>3. QED
189     BY <1>2, PTL
190
191 ====

```

A.7.6 CustodyObligation.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/common_boundary/CustodyObligation.tla}

```

1  ---- MODULE CustodyObligation ----
2  (*****
3  (* AFT-CB P2.4 -- validate-and-hold as a signing obligation (T3), the *)
4  (* retention/succession discipline, and the OPTIONAL audit lane. *)
5  (* *)
6  (* The obligation (spec §4): an honest member final-acks a boundary only *)
7  (* having OBTAINED the full byte set, and the ack commits it to RETAIN and *)
8  (* SERVE those bytes -- so a close implies close-time replication at every *)
9  (* compliant signer, and one honest signer (A2, stated as a theorem *)
10 (* condition) yields conditional availability deterministically. *)
11 (* *)
12 (* Pairwise audits are OPTIONAL enforcement actions producing slashing *)
13 (* evidence that NOTHING consumes as a close precondition: the audit *)
14 (* variable is written by the audit action alone and read by no guard of *)
15 (* any close-path action -- mirroring P2.3's silence-record discipline. *)
16 (* The zero-audit reachability witness (a trace that closes a slot with *)
17 (* auditEvidence = {}) is produced by TLC via the NoCloseEver violation *)
18 (* run recorded in the leg ledger. *)
19 (* *)
20 (* Succession (spec §6): custody releases only RECONSTRUCT-BEFORE-RELEASE *)
21 (* -- an honest holder may drop a slot's bytes (and its bond may release) *)
22 (* only once a successor member has reconstructed full custody of them. *)
23 (* *)
24 (* Adversary: structural (the P2.3 pattern) -- corrupt members' acks are *)
25 (* always available and their storage is arbitrary (they may sign without *)
26 (* holding and delete at will, which is why availability quantifies over *)
27 (* honest holders only). Deletion by the other n-1 is harmless BY *)
28 (* THEOREM, not by hope. *)
29 (*****
30 EXTENDS Naturals, FiniteSets
31
32 CONSTANTS
33   Ring,          \* the signer configuration
34   HonestMembers, \* honest subset (MHA enters theorems as a condition)
35   Slots,
36   Artifacts,     \* the byte universe; a boundary is a nonempty subset
37   NoBoundary     \* journal sentinel
38
39 ASSUME CustodyConstants ==
40   /\ HonestMembers \subseteqq Ring
41
42 Boundaries == (SUBSET Artifacts) \ {{}}
43
44 VARIABLES
45   delivered,     \* Artifacts \X Ring: adversary-scheduled delivery
46   holds,         \* [HonestMembers -> SUBSET Artifacts]: bytes actually held
47   journal,       \* [HonestMembers -> [Slots -> Boundaries \cup {NoBoundary}]]
48   acks,          \* honest final-acks <<m, s, B>>
49   retained,      \* [HonestMembers -> [Slots -> SUBSET Artifacts]]: serving custody
50   closes,        \* closed slots <<s, B>>
51   reconstructed, \* succession receipts <<successor, s>>: full custody rebuilt
52   bondReleased,  \* [HonestMembers -> BOOLEAN]
53   auditEvidence  \* audit-lane records <<auditor, subject, s>> -- never load-bearing
54
55 vars == <<delivered, holds, journal, acks, retained, closes, reconstructed,
56         bondReleased, auditEvidence>>
57
58 AckedBy(m, s, B) ==
59   IF m \in HonestMembers THEN <<m, s, B>> \in acks ELSE TRUE
60
61 TypeOK ==

```

```

62 /\ delivered \subseq Artifacts \X Ring
63 /\ holds \in [HonestMembers -> SUBSET Artifacts]
64 /\ journal \in [HonestMembers -> [Slots -> Boundaries \cup {NoBoundary}]]
65 /\ acks \subseq HonestMembers \X Slots \X Boundaries
66 /\ retained \in [HonestMembers -> [Slots -> SUBSET Artifacts]]
67 /\ closes \subseq Slots \X Boundaries
68 /\ reconstructed \subseq HonestMembers \X Slots
69 /\ bondReleased \in [HonestMembers -> BOOLEAN]
70 /\ auditEvidence \subseq Ring \X Ring \X Slots
71
72 Init ==
73 /\ delivered = {}
74 /\ holds = [m \in HonestMembers |> {}]
75 /\ journal = [m \in HonestMembers |> [s \in Slots |> NoBoundary]]
76 /\ acks = {}
77 /\ retained = [m \in HonestMembers |> [s \in Slots |> {}]]
78 /\ closes = {}
79 /\ reconstructed = {}
80 /\ bondReleased = [m \in HonestMembers |> FALSE]
81 /\ auditEvidence = {}
82
83 Deliver(a, m) ==
84 /\ delivered' = delivered \cup {<a, m>}
85 /\ UNCHANGED <<holds, journal, acks, retained, closes, reconstructed,
86 bondReleased, auditEvidence>>
87
88 (* An honest member obtains bytes it was actually delivered. *)
89 Fetch(m, a) ==
90 /\ m \in HonestMembers
91 /\ <a, m> \in delivered
92 /\ holds' = [holds EXCEPT ![m] = holds[m] \cup {a}]
93 /\ UNCHANGED <<delivered, journal, acks, retained, closes, reconstructed,
94 bondReleased, auditEvidence>>
95
96 (* VALIDATE-AND-HOLD: the guard B \subseq holds[m] is the obligation -- *)
97 (* the member has OBTAINED every byte of the boundary it signs -- and the *)
98 (* effect commits it to retain/serve (retained[m][s] = B). The journal *)
99 (* keeps single-final-ack (A3). The mutation drill removes the holds *)
100 (* guard and the T3-shaped invariant goes red. *)
101 HonestFinalAck(m, s, B) ==
102 /\ m \in HonestMembers
103 /\ ~bondReleased[m] \* a departed member signs nothing new
104 /\ s \in Slots /\ B \in Boundaries
105 /\ B \subseq holds[m]
106 /\ journal[m][s] = NoBoundary
107 /\ journal' = [journal EXCEPT ![m] = [journal[m] EXCEPT ![s] = B]]
108 /\ acks' = acks \cup {<m, s, B>}
109 /\ retained' = [retained EXCEPT ![m] = [retained[m] EXCEPT ![s] = B]]
110 /\ UNCHANGED <<delivered, holds, closes, reconstructed, bondReleased,
111 auditEvidence>>
112
113 (* n-of-n close: honest acks are state; corrupt signatures always *)
114 (* available (they may sign never having held a byte -- that is the fault *)
115 (* model, and why the theorem leans on the honest signer only). *)
116 CloseSlot(s, B) ==
117 /\ s \in Slots /\ B \in Boundaries
118 /\ \A m \in Ring : AckedBy(m, s, B)
119 /\ closes' = closes \cup {<s, B>}
120 /\ UNCHANGED <<delivered, holds, journal, acks, retained, reconstructed,
121 bondReleased, auditEvidence>>
122
123 (* Succession: a successor honest member reconstructs FULL custody of a *)

```

```

124 (* slot's closed boundary from bytes it actually holds. Reconstruction *)
125 (* ADDS custody (union) -- it never shrinks anyone's serving set; custody *)
126 (* drops ride bond release, outside this kernel's mutation surface. *)
127 Reconstruct(succ, s, B) ==
128   /\ succ \in HonestMembers
129   /\ ~bondReleased[succ]   \* a departed member takes on no new custody
130   /\ <<s, B>> \in closes
131   /\ B \subseteqq holds[succ]
132   /\ retained' = [retained EXCEPT ![succ] =
133     [retained[succ] EXCEPT ![s] = retained[succ][s] \cup B]]
134   /\ reconstructed' = reconstructed \cup {<<succ, s>>}
135   /\ UNCHANGED <<delivered, holds, journal, acks, closes, bondReleased,
136     auditEvidence>>
137
138 (* RECONSTRUCT-BEFORE-RELEASE: an honest member's bond releases only when *)
139 (* every slot it retains custody for has a DIFFERENT reconstructed *)
140 (* successor. (Custody drop rides bond release; the invariant below is *)
141 (* what the mutation targets.) *)
142 ReleaseBond(m) ==
143   /\ m \in HonestMembers
144   /\ \A s \in Slots :
145     (retained[m][s] # {}) =>
146       \E succ \in HonestMembers : succ # m /\ <<succ, s>> \in reconstructed
147   /\ bondReleased' = [bondReleased EXCEPT ![m] = TRUE]
148   /\ UNCHANGED <<delivered, holds, journal, acks, retained, closes,
149     reconstructed, auditEvidence>>
150
151 (* The OPTIONAL audit lane: an auditor challenges a subject over a slot *)
152 (* and a record is minted. No close-path guard reads auditEvidence -- *)
153 (* audits produce slashing evidence, never availability. *)
154 Audit(auditor, subject, s) ==
155   /\ auditor \in Ring /\ subject \in Ring /\ s \in Slots
156   /\ auditEvidence' = auditEvidence \cup {<<auditor, subject, s>>}
157   /\ UNCHANGED <<delivered, holds, journal, acks, retained, closes,
158     reconstructed, bondReleased>>
159
160 Next ==
161   \/ \E a \in Artifacts, m \in Ring : Deliver(a, m)
162   \/ \E m \in HonestMembers, a \in Artifacts : Fetch(m, a)
163   \/ \E m \in HonestMembers, s \in Slots, B \in Boundaries :
164     HonestFinalAck(m, s, B)
165   \/ \E s \in Slots, B \in Boundaries : CloseSlot(s, B)
166   \/ \E succ \in HonestMembers, s \in Slots, B \in Boundaries :
167     Reconstruct(succ, s, B)
168   \/ \E m \in HonestMembers : ReleaseBond(m)
169   \/ \E auditor \in Ring, subject \in Ring, s \in Slots :
170     Audit(auditor, subject, s)
171
172 Spec == Init /\ [] [Next]_vars
173
174 (*****
175 (* The T3-shaped invariant: every honest ack carries full custody -- the *)
176 (* acknowledged boundary is retained by the acker (or a reconstructed successor *)
177 (* stands in for it once released; in this kernel retained never shrinks, *)
178 (* so the direct form holds). From it: a closed slot with one honest ring *)
179 (* member is AVAILABLE -- some honest member retains every byte. *)
180 (*****)
181 CustodyInvariant ==
182   \A m \in HonestMembers, s \in Slots, B \in Boundaries :
183     (<<m, s, B>> \in acks) => B \subseteqq retained[m][s]
184
185 ConditionalAvailability ==

```

```

186   \A s \in Slots, B \in Boundaries :
187     (<<s, B>> \in closes /\ Ring \cap HonestMembers # {}) =>
188       \E m \in HonestMembers : B \subseq retained[m][s]
189
190 ReleaseDiscipline ==
191   \A m \in HonestMembers :
192     bondReleased[m] =>
193       \A s \in Slots :
194         (retained[m][s] # {}) =>
195           \E succ \in HonestMembers : succ # m /\ <<succ, s>> \in reconstructed
196
197 HonestAckJournal ==
198   \A m \in HonestMembers, s \in Slots, B \in Boundaries :
199     (<<m, s, B>> \in acks) => journal[m][s] = B
200
201 (* The T3 truth the validate-and-hold guard protects: every byte a member *)
202 (* CLAIMS custody of (retained) is a byte it actually HOLDS and can serve. *)
203 (* The leg's mutation drill removes the guard and THIS invariant goes red *)
204 (* -- a member would then claim custody of bytes it never obtained. *)
205 RetainedBacked ==
206   \A m \in HonestMembers, s \in Slots : retained[m][s] \subseq holds[m]
207
208 Inv ==
209   /\ TypeOK
210   /\ HonestAckJournal
211   /\ RetainedBacked
212   /\ CustodyInvariant
213   /\ \A s \in Slots, B \in Boundaries :
214     (<<s, B>> \in closes) => \A m \in Ring : AckedBy(m, s, B)
215   /\ ReleaseDiscipline
216
217
218 (* WITNESS-ONLY definition for the zero-audit reachability run: TLC *)
219 (* violating this invariant yields a shortest trace that closes a slot -- *)
220 (* and a shortest trace contains no Audit step, which IS the required *)
221 (* witness that audits are not load-bearing for a close. *)
222 NoCloseEver == closes = {}
223
224 ====

```

A.7.7 CustodyObligationProof.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/common_boundary/CustodyObligationProof.tla}

```

1 ---- MODULE CustodyObligationProof ----
2 (*****
3 (* AFT-CB P2.4 -- TLAPS discharge: the custody invariant is inductive, and *)
4 (* conditional availability (T3's shape) follows with MHA as an explicit *)
5 (* theorem condition. No proof step mentions auditEvidence: the audit *)
6 (* lane is not load-bearing, by the shape of the proof itself. *)
7 (*****
8 EXTENDS CustodyObligation, TLAPS
9
10 ASSUME NoBoundaryOutsideBoundaries == NoBoundary \notin Boundaries
11
12 LEMMA CInvInit == Init => Inv
13   BY CustodyConstants DEF Init, Inv, TypeOK, HonestAckJournal,
14     RetainedBacked, CustodyInvariant, ReleaseDiscipline, AckedBy

```

```

15
16 LEMMA CInvNext == Inv /\ [Next]_vars => Inv'
17 <1>1. SUFFICES ASSUME Inv, [Next]_vars PROVE Inv'
18 OBVIOUS
19 <1>2. CASE UNCHANGED vars
20   BY <1>1, <1>2 DEF Inv, TypeOK, HonestAckJournal, RetainedBacked,
21     CustodyInvariant, ReleaseDiscipline, AckedBy, vars
22 <1>3. ASSUME NEW a \in Artifacts, NEW m \in Ring, Deliver(a, m)
23   PROVE Inv'
24   BY <1>1, <1>3 DEF Inv, TypeOK, HonestAckJournal, RetainedBacked,
25     CustodyInvariant, ReleaseDiscipline, AckedBy, Deliver
26 <1>4. ASSUME NEW m \in HonestMembers, NEW a \in Artifacts, Fetch(m, a)
27   PROVE Inv'
28   <2>1. TypeOK'
29     BY <1>1, <1>4, CustodyConstants DEF Inv, TypeOK, Fetch
30   <2>2. HonestAckJournal'
31     BY <1>1, <1>4 DEF Inv, HonestAckJournal, Fetch
32   <2>3. CustodyInvariant'
33     BY <1>1, <1>4 DEF Inv, CustodyInvariant, Fetch
34   <2>4. ((\A s2 \in Slots, B2 \in Boundaries :
35     (<<s2, B2>> \in closes') => \A mm \in Ring : AckedBy(mm, s2, B2)')
36     BY <1>1, <1>4 DEF Inv, AckedBy, Fetch
37   <2>5. ReleaseDiscipline'
38     BY <1>1, <1>4 DEF Inv, ReleaseDiscipline, Fetch
39   <2>6. RetainedBacked'
40     <3>1. SUFFICES ASSUME NEW mm \in HonestMembers, NEW ss \in Slots
41       PROVE retained'[mm][ss] \subseteq holds'[mm]
42     BY DEF RetainedBacked
43   <3>2. retained' = retained
44     BY <1>4 DEF Fetch
45   <3>3. holds[mm] \subseteq holds'[mm]
46     BY <1>1, <1>4, CustodyConstants DEF Inv, TypeOK, Fetch
47   <3>4. retained[mm][ss] \subseteq holds[mm]
48     BY <1>1, <3>1 DEF Inv, RetainedBacked
49   <3> QED BY <3>1, <3>2, <3>3, <3>4
50   <2> QED BY <2>1, <2>2, <2>3, <2>4, <2>5, <2>6 DEF Inv
51 <1>5. ASSUME NEW m \in HonestMembers, NEW s \in Slots, NEW B \in Boundaries,
52   HonestFinalAck(m, s, B)
53   PROVE Inv'
54   <2>1. B \subseteq holds[m] /\ journal[m][s] = NoBoundary
55     BY <1>5 DEF HonestFinalAck
56   <2>2. TypeOK'
57     BY <1>1, <1>5, <2>1, CustodyConstants
58     DEF Inv, TypeOK, HonestFinalAck
59   <2>3. HonestAckJournal'
60     <3>1. SUFFICES ASSUME NEW mm \in HonestMembers, NEW ss \in Slots,
61       NEW BB \in Boundaries, <<mm, ss, BB>> \in acks'
62     PROVE journal'[mm][ss] = BB
63     BY DEF HonestAckJournal
64   <3>2. acks' = acks \cup {<<m, s, B>>}
65     BY <1>5 DEF HonestFinalAck
66   <3>3. CASE <<mm, ss, BB>> = <<m, s, B>>
67     <4>1. journal'[m][s] = B
68       BY <1>1, <1>5, <2>1 DEF Inv, TypeOK, HonestFinalAck
69     <4> QED BY <3>3, <4>1
70   <3>4. CASE <<mm, ss, BB>> \in acks
71     <4>1. journal[mm][ss] = BB
72       BY <1>1, <3>4 DEF Inv, HonestAckJournal
73     <4>2. CASE mm = m /\ ss = s
74       <5>1. journal[m][s] = BB
75       BY <4>1, <4>2
76     <5>2. BB # NoBoundary

```

```

77      BY <3>1, NoBoundaryOutsideBoundaries
78      <5> QED BY <2>1, <5>1, <5>2
79      <4>3. CASE ~(mm = m /\ ss = s)
80      <5>1. journal'[mm][ss] = journal[mm][ss]
81      BY <1>1, <1>5, <2>1, <4>3 DEF Inv, TypeOK, HonestFinalAck
82      <5> QED BY <4>1, <5>1
83      <4> QED BY <4>2, <4>3
84      <3> QED BY <3>1, <3>2, <3>3, <3>4
85      <2>4. CustodyInvariant'
86      <3>1. SUFFICES ASSUME NEW mm \in HonestMembers, NEW ss \in Slots,
87      NEW BB \in Boundaries, <<mm, ss, BB>> \in acks'
88      PROVE BB \subseteq retained'[mm][ss]
89      BY DEF CustodyInvariant
90      <3>2. acks' = acks \cup {<<m, s, B>>}
91      BY <1>5 DEF HonestFinalAck
92      <3>3. CASE <<mm, ss, BB>> = <<m, s, B>>
93      <4>1. retained'[m][s] = B
94      BY <1>1, <1>5, CustodyConstants DEF Inv, TypeOK, HonestFinalAck
95      <4> QED BY <3>3, <4>1
96      <3>4. CASE <<mm, ss, BB>> \in acks
97      <4>1. BB \subseteq retained[mm][ss]
98      BY <1>1, <3>4 DEF Inv, CustodyInvariant
99      <4>2. CASE mm = m /\ ss = s
100     <5>1. journal[m][s] = BB
101     BY <1>1, <3>4, <4>2 DEF Inv, HonestAckJournal
102     <5>2. BB # NoBoundary
103     BY <3>1, NoBoundaryOutsideBoundaries
104     <5> QED BY <2>1, <5>1, <5>2
105     <4>3. CASE ~(mm = m /\ ss = s)
106     <5>1. retained'[mm][ss] = retained[mm][ss]
107     BY <1>1, <1>5, CustodyConstants, <4>3
108     DEF Inv, TypeOK, HonestFinalAck
109     <5> QED BY <4>1, <5>1
110     <4> QED BY <4>2, <4>3
111     <3> QED BY <3>1, <3>2, <3>3, <3>4
112     <2>5. (\A s2 \in Slots, B2 \in Boundaries :
113     (<<s2, B2>> \in closes') => \A mm \in Ring : AckedBy(mm, s2, B2)')
114     BY <1>1, <1>5 DEF Inv, AckedBy, HonestFinalAck
115     <2>6. ReleaseDiscipline'
116     BY <1>1, <1>5, CustodyConstants
117     DEF Inv, TypeOK, ReleaseDiscipline, HonestFinalAck
118     <2>7. RetainedBacked'
119     BY <1>1, <1>5, <2>1, CustodyConstants
120     DEF Inv, TypeOK, RetainedBacked, HonestFinalAck
121     <2> QED BY <2>2, <2>3, <2>4, <2>5, <2>6, <2>7 DEF Inv
122     <1>6. ASSUME NEW s \in Slots, NEW B \in Boundaries, CloseSlot(s, B)
123     PROVE Inv'
124     <2>1. TypeOK'
125     BY <1>1, <1>6 DEF Inv, TypeOK, CloseSlot
126     <2>2. HonestAckJournal'
127     BY <1>1, <1>6 DEF Inv, HonestAckJournal, CloseSlot
128     <2>3. CustodyInvariant'
129     BY <1>1, <1>6 DEF Inv, CustodyInvariant, CloseSlot
130     <2>4. (\A s2 \in Slots, B2 \in Boundaries :
131     (<<s2, B2>> \in closes') => \A mm \in Ring : AckedBy(mm, s2, B2)')
132     <3>1. SUFFICES ASSUME NEW s2 \in Slots, NEW B2 \in Boundaries,
133     <<s2, B2>> \in closes', NEW mm \in Ring
134     PROVE AckedBy(mm, s2, B2)'
135     OBVIOUS
136     <3>2. closes' = closes \cup {<<s, B>>} /\ acks' = acks
137     BY <1>6 DEF CloseSlot
138     <3>3. CASE <<s2, B2>> \in closes

```

```

139     BY <1>1, <3>2, <3>3 DEF Inv, AckedBy
140     <3>4. CASE <<s2, B2>> = <<s, B>>
141     BY <1>6, <3>2, <3>4 DEF CloseSlot, AckedBy
142     <3> QED BY <3>1, <3>2, <3>3, <3>4
143 <2>5. ReleaseDiscipline'
144     BY <1>1, <1>6 DEF Inv, ReleaseDiscipline, CloseSlot
145 <2>6. RetainedBacked'
146     BY <1>1, <1>6 DEF Inv, RetainedBacked, CloseSlot
147 <2> QED BY <2>1, <2>2, <2>3, <2>4, <2>5, <2>6 DEF Inv
148 <1>7. ASSUME NEW succ \in HonestMembers, NEW s \in Slots, NEW B \in Boundaries,
149     Reconstruct(succ, s, B)
150     PROVE Inv'
151 <2>1. TypeOK'
152     BY <1>1, <1>7, CustodyConstants DEF Inv, TypeOK, Reconstruct
153 <2>2. CustodyInvariant'
154     <3>1. SUFFICES ASSUME NEW mm \in HonestMembers, NEW ss \in Slots,
155         NEW BB \in Boundaries, <<mm, ss, BB>> \in acks'
156         PROVE BB \subseq retained'[mm][ss]
157     BY DEF CustodyInvariant
158 <3>2. acks' = acks
159     BY <1>7 DEF Reconstruct
160 <3>3. BB \subseq retained'[mm][ss]
161     BY <1>1, <3>1, <3>2 DEF Inv, CustodyInvariant
162 <3>4. retained'[mm][ss] \subseq retained'[mm][ss]
163     (* Reconstruction only ever UNIONS custody in, so every member's *)
164     (* serving set is monotone under this action. *)
165     BY <1>1, <1>7, CustodyConstants DEF Inv, TypeOK, Reconstruct
166 <3> QED BY <3>3, <3>4
167 <2>3. HonestAckJournal'
168     BY <1>1, <1>7 DEF Inv, HonestAckJournal, Reconstruct
169 <2>4. (\A s2 \in Slots, B2 \in Boundaries :
170     (<<s2, B2>> \in closes') => \A mm \in Ring : AckedBy(mm, s2, B2)')
171     BY <1>1, <1>7 DEF Inv, AckedBy, Reconstruct
172 <2>5. ReleaseDiscipline'
173     BY <1>1, <1>7, CustodyConstants
174     DEF Inv, TypeOK, ReleaseDiscipline, Reconstruct
175 <2>6. RetainedBacked'
176     BY <1>1, <1>7, CustodyConstants
177     DEF Inv, TypeOK, RetainedBacked, Reconstruct
178 <2> QED BY <2>1, <2>2, <2>3, <2>4, <2>5, <2>6 DEF Inv
179 <1>8. ASSUME NEW m \in HonestMembers, ReleaseBond(m)
180     PROVE Inv'
181 <2>1. TypeOK'
182     BY <1>1, <1>8, CustodyConstants DEF Inv, TypeOK, ReleaseBond
183 <2>2. HonestAckJournal'
184     BY <1>1, <1>8 DEF Inv, HonestAckJournal, ReleaseBond
185 <2>3. CustodyInvariant'
186     BY <1>1, <1>8 DEF Inv, CustodyInvariant, ReleaseBond
187 <2>4. (\A s2 \in Slots, B2 \in Boundaries :
188     (<<s2, B2>> \in closes') => \A mm \in Ring : AckedBy(mm, s2, B2)')
189     BY <1>1, <1>8 DEF Inv, AckedBy, ReleaseBond
190 <2>5. ReleaseDiscipline'
191     <3>1. SUFFICES ASSUME NEW mm \in HonestMembers, bondReleased'[mm],
192         NEW ss \in Slots, retained'[mm][ss] # {}
193         PROVE \E succ \in HonestMembers :
194             succ # mm /\ <<succ, ss>> \in reconstructed'
195     BY DEF ReleaseDiscipline
196 <3>2. /\ bondReleased' = [bondReleased EXCEPT ![m] = TRUE]
197     /\ retained' = retained /\ reconstructed' = reconstructed
198     BY <1>8 DEF ReleaseBond
199 <3>3. CASE mm = m
200     BY <1>1, <1>8, <3>1, <3>2, <3>3, CustodyConstants

```

```

201     DEF Inv, TypeOK, ReleaseBond
202     <3>4. CASE mm # m
203     <4>1. bondReleased[mm]
204     BY <1>1, <3>1, <3>2, <3>4, CustodyConstants DEF Inv, TypeOK
205     <4> QED BY <1>1, <3>1, <3>2, <4>1 DEF Inv, ReleaseDiscipline
206     <3> QED BY <3>1, <3>3, <3>4
207     <2>6. RetainedBacked'
208     BY <1>1, <1>8 DEF Inv, RetainedBacked, ReleaseBond
209     <2> QED BY <2>1, <2>2, <2>3, <2>4, <2>5, <2>6 DEF Inv
210     <1>9. ASSUME NEW auditor \in Ring, NEW subject \in Ring, NEW s \in Slots,
211     Audit(auditor, subject, s)
212     PROVE Inv'
213     BY <1>1, <1>9 DEF Inv, TypeOK, HonestAckJournal, RetainedBacked,
214     CustodyInvariant, ReleaseDiscipline, AckedBy, Audit
215     <1>10. QED
216     BY <1>1, <1>2, <1>3, <1>4, <1>5, <1>6, <1>7, <1>8, <1>9 DEF Next, vars
217
218     LEMMA InvAvailability == Inv => ConditionalAvailability
219     <1>1. SUFFICES ASSUME Inv, NEW s \in Slots, NEW B \in Boundaries,
220     <<s, B>> \in closes, Ring \cap HonestMembers # {}
221     PROVE \E m \in HonestMembers : B \subseq retained[m][s]
222     BY DEF ConditionalAvailability
223     <1>2. PICK h \in Ring \cap HonestMembers : TRUE
224     BY <1>1
225     <1>3. AckedBy(h, s, B)
226     BY <1>1, <1>2 DEF Inv
227     <1>4. <<h, s, B>> \in acks
228     BY <1>2, <1>3 DEF AckedBy
229     <1>5. B \subseq retained[h][s]
230     BY <1>1, <1>2, <1>4 DEF Inv, CustodyInvariant
231     <1>6. QED
232     BY <1>2, <1>5
233
234     THEOREM CustodySafety ==
235     Spec => [] (ConditionalAvailability /\ ReleaseDiscipline)
236     <1>1. Init => Inv
237     BY CInvInit
238     <1>2. Inv /\ [Next]_vars => Inv'
239     BY CInvNext
240     <1>3. Spec => [] Inv
241     BY <1>1, <1>2, PTL DEF Spec
242     <1>4. Inv => (ConditionalAvailability /\ ReleaseDiscipline)
243     BY InvAvailability DEF Inv
244     <1>5. QED
245     BY <1>3, <1>4, PTL
246
247     ====

```

A.7.8 ForensicAccountability.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/common_boundary/ForensicAccountability.tla}

```

1  ---- MODULE ForensicAccountability ----
2  (*****
3  (* AFT-CB P2.6 -- forensic accountability (T7), proven against the      *)
4  (* WIRE-LEVEL certificate format.                                       *)
5  (*                                                                       *)
6  (* The wire format is part of the theorem (spec §12): a seal certificate *)

```

```

7  (* is a bitmap multisig -- a record carrying, for EVERY ring member, that *)
8  (* member's individually verifiable signature share over the exact *)
9  (* domain-separated tuple. Assembly is impossible without all n shares, *)
10 (* and decomposition back into per-member shares is total: any holder of *)
11 (* two conflicting certificates extracts, for every member, a pair of *)
12 (* conflicting shares -- a self-contained conviction (T7), covering every *)
13 (* participant necessary for the violation (all n; the L9 maximum). *)
14 (* *)
15 (* The mutation drill swaps the encoding for an attribution-destroying *)
16 (* OPAQUE AGGREGATE -- a certificate that proves "the ring signed" while *)
17 (* decomposing into nobody -- and the attributability invariant goes red: *)
18 (* exactly the trade the spec's §12.4 forbids on any seal path. *)
19 (* *)
20 (* The staged double-seal gate runs at the ALL-BYZANTINE configuration *)
21 (* (HonestMembers = {}): both conflicting certificates assemble (nothing *)
22 (* stops an all-corrupt ring from double-sealing -- that is above the *)
23 (* fault threshold), and TLC's DoubleSealReached violation trace IS the *)
24 (* extraction witness: the full necessary-participant set, recorded in *)
25 (* the leg ledger. *)
26 (* *)
27 (* Adversary: structural (P2.3's pattern). Corrupt members' shares over *)
28 (* any tuple are always available; honest shares exist only via the *)
29 (* journal-guarded honest ack (A3), which is why conflicting certificates *)
30 (* under MHA are impossible (P2.1's theorem, restated here over the wire *)
31 (* structure as CertUniqueness). *)
32 (*****
33 EXTENDS Naturals, FiniteSets
34
35 CONSTANTS
36   Ring,
37   HonestMembers,
38   Slots,
39   Roots,
40   NoRoot
41
42 ASSUME ForensicConstants ==
43   /\ HonestMembers \subseq Ring
44
45 (* A wire-level share: member m's signature over the domain-separated *)
46 (* tuple (slot, root). A certificate is bitmap-complete: it embeds the *)
47 (* share of EVERY member over ITS OWN (slot, root) -- the record structure *)
48 (* IS the format; there is nothing else on the wire. *)
49 Share(m, s, r) == <<m, s, r>>
50
51 Cert(s, r) == [slot |-> s, root |-> r,
52               shares |-> {Share(m, s, r) : m \in Ring}]
53
54 VARIABLES
55   journal,  \* [HonestMembers -> [Slots -> Roots \cup {NoRoot}]] (A3)
56   hshares,  \* honest shares in existence
57   certs    \* assembled certificates (wire records)
58
59 vars == <<journal, hshares, certs>>
60
61 ShareAvailable(m, s, r) ==
62   IF m \in HonestMembers THEN Share(m, s, r) \in hshares ELSE TRUE
63
64 TypeOK ==
65   /\ journal \in [HonestMembers -> [Slots -> Roots \cup {NoRoot}]]
66   /\ hshares \subseq HonestMembers \X Slots \X Roots
67   /\ certs \subseq [slot : Slots, root : Roots,
68                     shares : SUBSET (Ring \X Slots \X Roots)]

```

```

69
70 Init ==
71   /\ journal = [m \in HonestMembers |-> [s \in Slots |-> NoRoot]]
72   /\ hshares = {}
73   /\ certs = {}
74
75 HonestSign(m, s, r) ==
76   /\ m \in HonestMembers
77   /\ s \in Slots /\ r \in Roots
78   /\ journal[m][s] = NoRoot
79   /\ journal' = [journal EXCEPT ![m] = [journal[m] EXCEPT ![s] = r]]
80   /\ hshares' = hshares \cup {Share(m, s, r)}
81   /\ UNCHANGED certs
82
83 (* Assembly: anyone may assemble the bitmap-complete certificate once      *)
84 (* every member's share over the exact tuple is available (corrupt         *)
85 (* members' shares always are -- A1 excludes only forging honest ones).    *)
86 Assemble(s, r) ==
87   /\ s \in Slots /\ r \in Roots
88   /\ \A m \in Ring : ShareAvailable(m, s, r)
89   /\ certs' = certs \cup {Cert(s, r)}
90   /\ UNCHANGED <<journal, hshares>>
91
92 Next ==
93   /\ \E m \in Ring, s \in Slots, r \in Roots : HonestSign(m, s, r)
94   /\ \E s \in Slots, r \in Roots : Assemble(s, r)
95
96 Spec == Init /\ [] [Next]_vars
97
98 (*****
99 (* THE EXTRACTION PROCEDURE (spec §12.5), specified over the wire records: *)
100 (* given two certificates for one slot with different roots, the          *)
101 (* extracted set is every member exhibiting a share in BOTH -- a pair of  *)
102 (* signatures by one key over conflicting tuples, each pair a             *)
103 (* self-contained conviction.                                             *)
104 (*****
105 ExtractedSet(c1, c2) ==
106   {m \in Ring : /\ Share(m, c1.slot, c1.root) \in c1.shares
107                 /\ Share(m, c2.slot, c2.root) \in c2.shares}
108
109 Conflicting(c1, c2) ==
110   /\ c1.slot = c2.slot
111   /\ c1.root # c2.root
112
113 (* T7 at the wire level: ANY two conflicting certificates decompose into  *)
114 (* conflicting share pairs for EVERY ring member -- the full necessary-    *)
115 (* participant set (n-of-n makes each member necessary; L9 makes all-n    *)
116 (* the maximum any protocol can attribute).                               *)
117 AttributionComplete ==
118   \A c1 \in certs, c2 \in certs :
119     Conflicting(c1, c2) => ExtractedSet(c1, c2) = Ring
120
121 (* P2.1's uniqueness, restated over the wire structure: under MHA (as a   *)
122 (* condition), conflicting certificates cannot both assemble.             *)
123 CertUniqueness ==
124   \A c1 \in certs, c2 \in certs :
125     (Conflicting(c1, c2)) => HonestMembers = {}
126
127 (* Reachability witness target for the all-Byzantine staged double-seal:  *)
128 (* TLC violating this invariant produces the trace in which both          *)
129 (* conflicting certificates assemble and the extraction yields all n.      *)
130 DoubleSealReached ==

```

```

131 ~\E c1 \in certs, c2 \in certs : Conflicting(c1, c2)
132
133 Inv ==
134 /\ TypeOK
135 /\ \A m \in HonestMembers, s \in Slots, r \in Roots :
136   (Share(m, s, r) \in hshares) => journal[m][s] = r
137 /\ \A c \in certs :
138   /\ c.shares = {Share(m, c.slot, c.root) : m \in Ring}
139   /\ \A m \in HonestMembers : Share(m, c.slot, c.root) \in hshares
140
141 ====

```

A.7.9 ForensicAccountabilityProof.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/common_boundary/ForensicAccountabilityProof.tla}

```

1  ---- MODULE ForensicAccountabilityProof ----
2  (*****
3  (* AFT-CB P2.6 -- TLAPS discharge of T7 over the wire format: attribution *)
4  (* completeness is a consequence of the bitmap-complete record structure, *)
5  (* and certificate uniqueness under MHA restates P2.1's theorem over the *)
6  (* wire. The extraction procedure is ExtractedSet, and the theorem says *)
7  (* it returns ALL of Ring on any conflicting pair. *)
8  (*****)
9  EXTENDS ForensicAccountability, TLAPS
10
11 ASSUME NoRootOutsideRootsF == NoRoot \notin Roots
12
13 LEMMA FInvInit == Init => Inv
14   BY ForensicConstants DEF Init, Inv, TypeOK
15
16 LEMMA FInvNext == Inv /\ [Next]_vars => Inv'
17 <1>1. SUFFICES ASSUME Inv, [Next]_vars PROVE Inv'
18   OBVIOUS
19 <1>2. CASE UNCHANGED vars
20   BY <1>1, <1>2 DEF Inv, TypeOK, vars
21 <1>3. ASSUME NEW m \in Ring, NEW s \in Slots, NEW r \in Roots,
22       HonestSign(m, s, r)
23   PROVE Inv'
24   <2>1. m \in HonestMembers /\ journal[m][s] = NoRoot
25   BY <1>3 DEF HonestSign
26   <2>2. TypeOK'
27   BY <1>1, <1>3, <2>1, ForensicConstants DEF Inv, TypeOK, HonestSign, Share
28   <2>3. \A mm \in HonestMembers, ss \in Slots, rr \in Roots :
29       (Share(mm, ss, rr) \in hshares') => journal'[mm][ss] = rr
30   <3>1. SUFFICES ASSUME NEW mm \in HonestMembers, NEW ss \in Slots,
31       NEW rr \in Roots, Share(mm, ss, rr) \in hshares'
32   PROVE journal'[mm][ss] = rr
33   OBVIOUS
34   <3>2. hshares' = hshares \cup {Share(m, s, r)}
35   BY <1>3 DEF HonestSign
36   <3>3. CASE Share(mm, ss, rr) = Share(m, s, r)
37   <4>1. mm = m /\ ss = s /\ rr = r
38   BY <3>3 DEF Share
39   <4>2. journal'[m][s] = r
40   BY <1>1, <1>3, <2>1 DEF Inv, TypeOK, HonestSign
41   <4> QED BY <4>1, <4>2
42   <3>4. CASE Share(mm, ss, rr) \in hshares

```

```

43      <4>1. journal[mm][ss] = rr
44      BY <1>1, <3>4 DEF Inv
45      <4>2. CASE mm = m /\ ss = s
46      <5>1. journal[m][s] = rr
47      BY <4>1, <4>2
48      <5>2. rr # NoRoot
49      BY <3>1, NoRootOutsideRootsF
50      <5> QED BY <2>1, <5>1, <5>2
51      <4>3. CASE ~(mm = m /\ ss = s)
52      <5>1. journal'[mm][ss] = journal[mm][ss]
53      BY <1>1, <1>3, <2>1, <4>3 DEF Inv, TypeOK, HonestSign
54      <5> QED BY <4>1, <5>1
55      <4> QED BY <4>2, <4>3
56      <3> QED BY <3>1, <3>2, <3>3, <3>4
57      <2>4. \A c \in certs' :
58      /\ c.shares = {Share(mm, c.slot, c.root) : mm \in Ring}
59      /\ \A mm \in HonestMembers : Share(mm, c.slot, c.root) \in hshares'
60      <3>1. certs' = certs
61      BY <1>3 DEF HonestSign
62      <3>2. hshares \subsesteq hshares'
63      BY <1>3 DEF HonestSign
64      <3> QED BY <1>1, <3>1, <3>2 DEF Inv
65      <2> QED BY <2>2, <2>3, <2>4 DEF Inv
66      <1>4. ASSUME NEW s \in Slots, NEW r \in Roots, Assemble(s, r)
67      PROVE Inv'
68      <2>1. TypeOK'
69      <3>1. Cert(s, r) \in [slot : Slots, root : Roots,
70      shares : SUBSET (Ring \X Slots \X Roots)]
71      BY <1>4 DEF Cert, Share
72      <3> QED BY <1>1, <1>4, <3>1 DEF Inv, TypeOK, Assemble
73      <2>2. \A mm \in HonestMembers, ss \in Slots, rr \in Roots :
74      (Share(mm, ss, rr) \in hshares') => journal'[mm][ss] = rr
75      BY <1>1, <1>4 DEF Inv, Assemble
76      <2>3. \A c \in certs' :
77      /\ c.shares = {Share(mm, c.slot, c.root) : mm \in Ring}
78      /\ \A mm \in HonestMembers : Share(mm, c.slot, c.root) \in hshares'
79      <3>1. SUFFICES ASSUME NEW c \in certs'
80      PROVE /\ c.shares = {Share(mm, c.slot, c.root) : mm \in Ring}
81      /\ \A mm \in HonestMembers :
82      Share(mm, c.slot, c.root) \in hshares'
83      OBVIOUS
84      <3>2. certs' = certs \cup {Cert(s, r)} /\ hshares' = hshares
85      BY <1>4 DEF Assemble
86      <3>3. CASE c \in certs
87      BY <1>1, <3>2, <3>3 DEF Inv
88      <3>4. CASE c = Cert(s, r)
89      <4>1. c.shares = {Share(mm, s, r) : mm \in Ring}
90      /\ c.slot = s /\ c.root = r
91      BY <3>4 DEF Cert
92      <4>2. \A mm \in HonestMembers : Share(mm, s, r) \in hshares
93      <5>1. \A mm \in Ring : ShareAvailable(mm, s, r)
94      BY <1>4 DEF Assemble
95      <5> QED BY <5>1, ForensicConstants DEF ShareAvailable
96      <4> QED BY <3>2, <4>1, <4>2
97      <3> QED BY <3>1, <3>2, <3>3, <3>4
98      <2> QED BY <2>1, <2>2, <2>3 DEF Inv
99      <1>5. QED
100     BY <1>1, <1>2, <1>3, <1>4 DEF Next, vars
101
102     (*****
103     (* T7: any two conflicting wire certificates decompose into conflicting *)
104     (* share pairs for EVERY ring member -- the extraction procedure returns *)

```

```

105 (* the full necessary-participant set. Purely structural: the record *)
106 (* format guarantees it, which is exactly why the format is part of the *)
107 (* theorem and an opaque aggregate destroys it (the mutation drill). *)
108 (*****)
109 LEMMA InvAttribution == Inv => AttributionComplete
110 <1>1. SUFFICES ASSUME Inv, NEW c1 \in certs, NEW c2 \in certs,
111       Conflicting(c1, c2)
112       PROVE ExtractedSet(c1, c2) = Ring
113       BY DEF AttributionComplete
114 <1>2. c1.shares = {Share(mm, c1.slot, c1.root) : mm \in Ring}
115       /\ c2.shares = {Share(mm, c2.slot, c2.root) : mm \in Ring}
116       BY <1>1 DEF Inv
117 <1>3. \A m \in Ring : /\ Share(m, c1.slot, c1.root) \in c1.shares
118       /\ Share(m, c2.slot, c2.root) \in c2.shares
119       BY <1>2
120 <1>4. QED
121       BY <1>3 DEF ExtractedSet
122
123 (*****)
124 (* Certificate uniqueness under MHA, restated over the wire: a conflicting *)
125 (* pair requires an honest member's shares over two roots at one slot, *)
126 (* which the journal forbids -- so conflict implies zero honest members. *)
127 (*****)
128 LEMMA InvCertUniqueness == Inv => CertUniqueness
129 <1>1. SUFFICES ASSUME Inv, NEW c1 \in certs, NEW c2 \in certs,
130       Conflicting(c1, c2)
131       PROVE HonestMembers = {}
132       BY DEF CertUniqueness
133 <1>2. SUFFICES ASSUME NEW h \in HonestMembers PROVE FALSE
134       OBVIOUS
135 <1>3. Share(h, c1.slot, c1.root) \in hshares
136       /\ Share(h, c2.slot, c2.root) \in hshares
137       BY <1>1, <1>2 DEF Inv
138 <1>4. journal[h][c1.slot] = c1.root /\ journal[h][c2.slot] = c2.root
139       <2>1. c1.slot \in Slots /\ c1.root \in Roots
140             /\ c2.slot \in Slots /\ c2.root \in Roots
141       BY <1>1 DEF Inv, TypeOK
142       <2> QED BY <1>1, <1>2, <1>3, <2>1 DEF Inv
143 <1>5. c1.slot = c2.slot /\ c1.root # c2.root
144       BY <1>1 DEF Conflicting
145 <1>6. QED
146       BY <1>4, <1>5
147
148 THEOREM ForensicSafety ==
149   Spec => [] (AttributionComplete /\ CertUniqueness)
150 <1>1. Init => Inv
151       BY FInvInit
152 <1>2. Inv /\ [Next]_vars => Inv'
153       BY FInvNext
154 <1>3. Spec => [] Inv
155       BY <1>1, <1>2, PTL DEF Spec
156 <1>4. Inv => (AttributionComplete /\ CertUniqueness)
157       BY InvAttribution, InvCertUniqueness
158 <1>5. QED
159       BY <1>3, <1>4, PTL
160
161 =====

```

A.7.10 SuccessionClock.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/common_boundary/SuccessionClock.tla}

```

1  ---- MODULE SuccessionClock ----
2  (*****
3  (* AFT-CB P2.7 -- the §16 v3 succession kernel. HONEST LABEL AFTER REVIEW *)
4  (* ROUND 5: this module discharges a STRICTLY NARROWED statement, not §16 *)
5  (* (R5-F3): the Byzantine standby member has no modeled behavior, *)
6  (* obs_commit is computed as the honest union (never adversary-chosen), *)
7  (* lastSealTick never advances, and the MHA-corner config cannot hold two *)
8  (* conflicting old seals. The proof is internally valid and RETAINED as *)
9  (* the record of what v3's refusal mechanism does prove; it is NOT a *)
10 (* discharge of §16, whose v3 was refuted (see the spec banner and *)
11 (* p2-7-respec-review.md). A faithful re-mechanization is a v4 *)
12 (* precondition. *)
13 (* *)
14 (* v3's design (spec §16, RESPECIFIED -- pending its own review): the *)
15 (* standby S_v adjudicates under its OWN per-configuration MHA. A rival *)
16 (* old-ring seal BINDS iff it is in the fallback seal's UBC-committed *)
17 (* observation set; an honest standby member REFUSES to sign a fallback *)
18 (* seal that omits an observed object or covers a slot where it observed *)
19 (* a rival seal at all -- preemption by refusal, before any certificate *)
20 (* exists. Unobserved rivals are VOID by the old ring's own *)
21 (* formation-time signature. No bulletin, no stamps, no delivery bound *)
22 (* on any medium exists in this design. *)
23 (* *)
24 (* Time: ticks advance monotonically (A9 gives objectivity; the chain *)
25 (* condition " $\geq T_{halt}$  seal-free ticks" is read off the single-valued *)
26 (* seal-seeded chain, modeled as lastSealTick). The adversary schedules *)
27 (* everything, including when submissions become observations (A10 -- an *)
28 (* object submitted to the standby reaches  $\geq 1$  honest member within *)
29 (* G_deliv -- is the FAIRNESS the old ring's protection rides; the safety *)
30 (* theorem below never consumes it, which is the point: safety holds *)
31 (* even when A10 fails, at the old ring's liveness cost). *)
32 (* *)
33 (* Single contested slot; roots are the contested content. *)
34 (*****
35 EXTENDS Naturals, FiniteSets
36
37 CONSTANTS
38   OldRing,
39   HonestOld,      \* honest subset of the old ring
40   Standby,
41   HonestStandby,  \* honest subset of the standby (A2(S_v) enters theorems)
42   Roots,
43   NoRoot,
44   T_halt,         \* activation threshold (ticks since last seal)
45   G,              \* release wait after activation
46   MaxTick
47
48 ASSUME SuccessionConstants ==
49   /\ HonestOld \subseteqq OldRing
50   /\ HonestStandby \subseteqq Standby
51   /\ T_halt \in Nat \ {0}
52   /\ G \in Nat
53   /\ MaxTick \in Nat
54
55 VARIABLES
56   tick,          \* the objective clock (monotone; A9)

```

```

57 lastSealTick,    \* tick of the last old-ring seal folded into the chain
58 policySigned,    \* BOOLEAN: formation-time unanimous succession policy
59 oldJournal,      \* [HonestOld -> Roots \cup {NoRoot}]: single-final-ack (A3)
60 oldSeals,        \* assembled old-ring seals for the contested slot: roots
61 submitted,       \* old seals submitted to the standby set: roots
62 observedBy,      \* [HonestStandby -> SUBSET Roots]: rival seals observed
63 activation,       \* 0 = none, else the activation tick T_a
64 fallbackSeals,   \* [root : Roots, obs : SUBSET Roots]: obs = obs_commit
65 released         \* roots whose fallback irreversible effect released
66
67 vars == <<tick, lastSealTick, policySigned, oldJournal, oldSeals, submitted,
68         observedBy, activation, fallbackSeals, released>>
69
70 OldAckedBy(m, r) ==
71   IF m \in HonestOld THEN oldJournal[m] = r ELSE TRUE
72
73 TypeOK ==
74   /\ tick \in 0..MaxTick
75   /\ lastSealTick \in 0..MaxTick
76   /\ policySigned \in BOOLEAN
77   /\ oldJournal \in [HonestOld -> Roots \cup {NoRoot}]
78   /\ oldSeals \subsepeq Roots
79   /\ submitted \subsepeq Roots
80   /\ observedBy \in [HonestStandby -> SUBSET Roots]
81   /\ activation \in 0..MaxTick
82   /\ fallbackSeals \subsepeq [root : Roots, obs : SUBSET Roots]
83   /\ released \subsepeq Roots
84
85 Init ==
86   /\ tick = 0
87   /\ lastSealTick = 0
88   /\ policySigned = TRUE    \* formation-time unanimity (the drill flips it)
89   /\ oldJournal = [m \in HonestOld |-> NoRoot]
90   /\ oldSeals = {}
91   /\ submitted = {}
92   /\ observedBy = [h \in HonestStandby |-> {}]
93   /\ activation = 0
94   /\ fallbackSeals = {}
95   /\ released = {}
96
97 Tick ==
98   /\ tick < MaxTick
99   /\ tick' = tick + 1
100  /\ UNCHANGED <<lastSealTick, policySigned, oldJournal, oldSeals, submitted,
101    observedBy, activation, fallbackSeals, released>>
102
103 (* Old-ring final-ack (honest, journal-guarded) and structural Byzantine *)
104 (* availability let old seals assemble; the standby race is what this *)
105 (* module studies, so seal assembly is one action. *)
106 OldAck(m, r) ==
107   /\ m \in HonestOld
108   /\ r \in Roots
109   /\ oldJournal[m] = NoRoot
110   /\ oldJournal' = [oldJournal EXCEPT ![m] = r]
111   /\ UNCHANGED <<tick, lastSealTick, policySigned, oldSeals, submitted,
112    observedBy, activation, fallbackSeals, released>>
113
114 AssembleOldSeal(r) ==
115   /\ r \in Roots
116   /\ \A m \in OldRing : OldAckedBy(m, r)
117   /\ oldSeals' = oldSeals \cup {r}
118   /\ UNCHANGED <<tick, lastSealTick, policySigned, oldJournal, submitted,

```

```

119         observedBy, activation, fallbackSeals, released>>
120
121 (* Submission of a rival old seal toward the standby set, and its *)
122 (* adversary-scheduled arrival at an honest standby member's observation. *)
123 (* A10 (reach within G_deliv) is deliberately NOT enforced here: safety *)
124 (* must hold on every schedule, including ones where A10 fails. *)
125 Submit(r) ==
126     /\ r \in oldSeals
127     /\ submitted' = submitted \cup {r}
128     /\ UNCHANGED <<tick, lastSealTick, policySigned, oldJournal, oldSeals,
129         observedBy, activation, fallbackSeals, released>>
130
131 Observe(h, r) ==
132     /\ h \in HonestStandby
133     /\ r \in submitted
134     /\ observedBy' = [observedBy EXCEPT ![h] = observedBy[h] \cup {r}]
135     /\ UNCHANGED <<tick, lastSealTick, policySigned, oldJournal, oldSeals,
136         submitted, activation, fallbackSeals, released>>
137
138 (* Activation: valid only through the formation-time policy AND the *)
139 (* objective chain condition --  $\geq T_{\text{halt}}$  seal-free ticks (A9; the chain is *)
140 (* single-valued because only seals fold into it). No silence-derived *)
141 (* input exists anywhere in this action. *)
142 Activate ==
143     /\ policySigned
144     /\ activation = 0
145     /\ tick - lastSealTick  $\geq T_{\text{halt}}$ 
146     /\ activation' = tick
147     /\ UNCHANGED <<tick, lastSealTick, policySigned, oldJournal, oldSeals,
148         submitted, observedBy, fallbackSeals, released>>
149
150 (* The observation-committed fallback seal. The honest-refusal guard is *)
151 (* the v3 mechanism: every honest standby member's observations must be *)
152 (* inside obs_commit, and NO honest member may have observed ANY rival -- *)
153 (* otherwise no certificate can exist (A2(S_v) makes the honest share *)
154 (* necessary). obs_commit is exactly the union of honest observations at *)
155 (* signing time. *)
156 AssembleFallback(r) ==
157     /\ activation > 0
158     /\ r \in Roots
159     /\ \A h \in HonestStandby : \A rv \in observedBy[h] : rv = r
160     /\ fallbackSeals' = fallbackSeals \cup
161         {[root |-> r, obs |-> UNION {observedBy[h] : h \in HonestStandby}]}
162     /\ UNCHANGED <<tick, lastSealTick, policySigned, oldJournal, oldSeals,
163         submitted, observedBy, activation, released>>
164
165 (* Release: the fallback irreversible effect for r, after the wait. *)
166 Release(r) ==
167     /\ activation > 0
168     /\ tick  $\geq$  activation + G
169     /\ \E fs \in fallbackSeals : fs.root = r
170     /\ released' = released \cup {r}
171     /\ UNCHANGED <<tick, lastSealTick, policySigned, oldJournal, oldSeals,
172         submitted, observedBy, activation, fallbackSeals>>
173
174 Next ==
175     \/ Tick
176     \/ \E m \in OldRing, r \in Roots : OldAck(m, r)
177     \/ \E r \in Roots : AssembleOldSeal(r) \/ Submit(r) \/ AssembleFallback(r)
178         \/ Release(r)
179     \/ \E h \in Standby, r \in Roots : Observe(h, r)
180     \/ Activate

```

```

181
182 Spec == Init /\ [] [Next]_vars
183
184 (*****
185 (* Bindingness (§16.5): a rival old seal binds against fallback root r iff *)
186 (* it is in some r-fallback-seal's committed observation set. VoidByPolicy *)
187 (* is its complement -- definitional, by the old ring's own formation *)
188 (* signature. *)
189 (*****
190 Binding(rv, r) ==
191   \E fs \in fallbackSeals : fs.root = r /\ rv \in fs.obs /\ rv # r
192
193 (* T5d-v3: no released fallback effect coexists with a BINDING rival. *)
194 NoBindingRivalReleased ==
195   \A r \in released, rv \in Roots : ~Binding(rv, r)
196
197 (* Activation is never silence-justified: it exists only with the *)
198 (* formation policy and the chain condition met at its tick. *)
199 ActivationJustified ==
200   (activation > 0) => (policySigned /\ activation - lastSealTick >= T_halt)
201
202 Inv ==
203   /\ TypeOK
204   /\ \A fs \in fallbackSeals :
205     \A rv \in fs.obs : rv = fs.root
206   /\ ActivationJustified
207
208 ====

```

A.7.11 SuccessionClockProof.tla

Repository path. \detokenize{internal-docs/architecture/protocols/aft/formal/common_boundary/SuccessionClockProof.tla}

```

1 ---- MODULE SuccessionClockProof ----
2 (*****
3 (* AFT-CB P2.7 -- TLAPS discharge of the §16 v3 kernel. *)
4 (* *)
5 (* T5d-v3's safety is BY REFUSAL: the honest-standby guard means no *)
6 (* fallback certificate ever commits a rival, so no released effect can *)
7 (* face a BINDING rival -- the theorem is the guard's inductive *)
8 (* preservation, under A2(S_v) modeled structurally (the honest share is *)
9 (* necessary, so the guard binds every assembled certificate). A *)
10 (* Byzantine signer padding obs_commit could only ADD rivals -- which *)
11 (* blocks its own fallback (Binding fires) -- the safe direction; the *)
12 (* kernel therefore commits the honest union exactly. *)
13 (* *)
14 (* ActivationJustified: activation exists only with the formation policy *)
15 (* and the objective chain condition at its own tick -- no silence input. *)
16 (*****
17 EXTENDS SuccessionClock, TLAPS
18
19 ASSUME NoRootOutsideRootsS == NoRoot \notin Roots
20
21 LEMMA SInvInit == Init => Inv
22   BY SuccessionConstants DEF Init, Inv, TypeOK, ActivationJustified
23
24 LEMMA SInvNext == Inv /\ [Next]_vars => Inv'
25 <1>1. SUFFICES ASSUME Inv, [Next]_vars PROVE Inv'

```

```

26  OBVIOUS
27  <1>2. CASE UNCHANGED vars
28  BY <1>1, <1>2 DEF Inv, TypeOK, ActivationJustified, vars
29  <1>3. CASE Tick
30  <2>1. tick < MaxTick /\ tick' = tick + 1
31  BY <1>3 DEF Tick
32  <2>2. TypeOK'
33  BY <1>1, <2>1, <1>3, SuccessionConstants DEF Inv, TypeOK, Tick
34  <2>3. (\A fs \in fallbackSeals' : \A rv \in fs.obs : rv = fs.root)
35  BY <1>1, <1>3 DEF Inv, Tick
36  <2>4. ActivationJustified'
37  BY <1>1, <1>3 DEF Inv, ActivationJustified, Tick
38  <2> QED BY <2>2, <2>3, <2>4 DEF Inv
39  <1>4. ASSUME NEW m \in OldRing, NEW r \in Roots, OldAck(m, r)
40  PROVE Inv'
41  BY <1>1, <1>4 DEF Inv, TypeOK, ActivationJustified, OldAck
42  <1>5. ASSUME NEW r \in Roots, AssembleOldSeal(r)
43  PROVE Inv'
44  BY <1>1, <1>5 DEF Inv, TypeOK, ActivationJustified, AssembleOldSeal
45  <1>6. ASSUME NEW r \in Roots, Submit(r)
46  PROVE Inv'
47  BY <1>1, <1>6 DEF Inv, TypeOK, ActivationJustified, Submit
48  <1>7. ASSUME NEW h \in Standby, NEW r \in Roots, Observe(h, r)
49  PROVE Inv'
50  <2>1. h \in HonestStandby /\ r \in submitted
51  BY <1>7 DEF Observe
52  <2>2. TypeOK'
53  BY <1>1, <1>7, <2>1, SuccessionConstants DEF Inv, TypeOK, Observe
54  <2>3. (\A fs \in fallbackSeals' : \A rv \in fs.obs : rv = fs.root)
55  BY <1>1, <1>7 DEF Inv, Observe
56  <2>4. ActivationJustified'
57  BY <1>1, <1>7 DEF Inv, ActivationJustified, Observe
58  <2> QED BY <2>2, <2>3, <2>4 DEF Inv
59  <1>8. CASE Activate
60  <2>1. TypeOK'
61  BY <1>1, <1>8 DEF Inv, TypeOK, Activate
62  <2>2. (\A fs \in fallbackSeals' : \A rv \in fs.obs : rv = fs.root)
63  BY <1>1, <1>8 DEF Inv, Activate
64  <2>3. ActivationJustified'
65  BY <1>1, <1>8 DEF Inv, TypeOK, ActivationJustified, Activate
66  <2> QED BY <2>1, <2>2, <2>3 DEF Inv
67  <1>9. ASSUME NEW r \in Roots, AssembleFallback(r)
68  PROVE Inv'
69  <2>1. TypeOK'
70  <3>1. UNION {observedBy[h] : h \in HonestStandby} \subseteq Roots
71  BY <1>1, SuccessionConstants DEF Inv, TypeOK
72  <3> QED BY <1>1, <1>9, <3>1 DEF Inv, TypeOK, AssembleFallback
73  <2>2. (\A fs \in fallbackSeals' : \A rv \in fs.obs : rv = fs.root)
74  <3>1. SUFFICES ASSUME NEW fs \in fallbackSeals', NEW rv \in fs.obs
75  PROVE rv = fs.root
76  OBVIOUS
77  <3>2. fallbackSeals' = fallbackSeals \cup
78  {[root |-> r, obs |-> UNION {observedBy[h] : h \in HonestStandby}]}
79  BY <1>9 DEF AssembleFallback
80  <3>3. CASE fs \in fallbackSeals
81  BY <1>1, <3>3 DEF Inv
82  <3>4. CASE fs = [root |-> r, obs |-> UNION {observedBy[h] : h \in HonestStandby}]
83  <4>1. \A h \in HonestStandby : \A rv2 \in observedBy[h] : rv2 = r
84  BY <1>9 DEF AssembleFallback
85  <4>2. rv \in UNION {observedBy[h] : h \in HonestStandby}
86  BY <3>1, <3>4
87  <4>3. rv = r

```

```

88     BY <4>1, <4>2
89     <4> QED BY <3>4, <4>3
90     <3> QED BY <3>1, <3>2, <3>3, <3>4
91     <2>3. ActivationJustified'
92     BY <1>1, <1>9 DEF Inv, ActivationJustified, AssembleFallback
93     <2> QED BY <2>1, <2>2, <2>3 DEF Inv
94 <1>10. ASSUME NEW r \in Roots, Release(r)
95     PROVE Inv'
96     BY <1>1, <1>10 DEF Inv, TypeOK, ActivationJustified, Release
97 <1>11. QED
98     BY <1>1, <1>2, <1>3, <1>4, <1>5, <1>6, <1>7, <1>8, <1>9, <1>10
99     DEF Next, vars
100
101 LEMMA InvNoBindingRival == Inv => NoBindingRivalReleased
102 <1>1. SUFFICES ASSUME Inv, NEW r \in released, NEW rv \in Roots,
103     Binding(rv, r)
104     PROVE FALSE
105     BY DEF NoBindingRivalReleased
106 <1>2. PICK fs \in fallbackSeals : fs.root = r /\ rv \in fs.obs /\ rv # r
107     BY <1>1 DEF Binding
108 <1>3. rv = fs.root
109     BY <1>1, <1>2 DEF Inv
110 <1>4. QED
111     BY <1>2, <1>3
112
113 THEOREM SuccessionSafety ==
114     Spec => [] (NoBindingRivalReleased /\ ActivationJustified)
115 <1>1. Init => Inv
116     BY SInvInit
117 <1>2. Inv /\ [Next]_vars => Inv'
118     BY SInvNext
119 <1>3. Spec => [] Inv
120     BY <1>1, <1>2, PTL DEF Spec
121 <1>4. Inv => (NoBindingRivalReleased /\ ActivationJustified)
122     BY InvNoBindingRival DEF Inv
123 <1>5. QED
124     BY <1>3, <1>4, PTL
125
126 ====

```

References

- [1] Maofan Yin, Dahlia Malkhi, Michael K. Reiter, Guy Golan-Gueta, and Ittai Abraham. *HotStuff: BFT Consensus in the Lens of Blockchain*. arXiv:1803.05069. <https://arxiv.org/abs/1803.05069>
- [2] George Danezis, Lefteris Kokoris-Kogias, Alberto Sonnino, and Alexander Spiegelman. *Narwhal and Tusk: A DAG-based Mempool and Efficient BFT Consensus*. arXiv:2105.11827. <https://arxiv.org/abs/2105.11827>
- [3] Ittai Abraham, Dahlia Malkhi, Kartik Nayak, Ling Ren, and Alexander Spiegelman. *Bullshark: DAG BFT Protocols Made Practical*. arXiv:2201.05677. <https://arxiv.org/abs/2201.05677>
- [4] Andrew Miller, Yu Xia, Kyle Croman, Elaine Shi, and Dawn Song. *The Honey Badger of BFT Protocols*. Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security. <https://eprint.iacr.org/2016/199.pdf>

- [5] Bingyong Guo, Zhenliang Lu, Qiang Tang, Jing Xu, and Zhenfeng Zhang. *Dumbo: Faster Asynchronous BFT Protocols*. Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security. <https://dblp.org/rec/conf/ccs/GuoL0XZ20>
- [6] Alistair Stewart and Eleftherios Kokoris-Kogias. *GRANDPA: a Byzantine Finality Gadget*. arXiv:2007.01560. <https://arxiv.org/abs/2007.01560>
- [7] Jelena Dosenovic, Roger Wattenhofer, and others. *Accountable Safety Implies Finality*. arXiv:2308.16902. <https://arxiv.org/abs/2308.16902>
- [8] Chainlink Labs. *Chainlink CCIP's Defense-In-Depth Security and the Risk Management Network*. Chainlink blog, 2023. <https://blog.chain.link/ccip-risk-management-network/>
- [9] John P. Conley. *Proof of Honesty: Coalition-Proof Blockchain Validation without Proof of Work or Stake*. Geeq technical paper, version 2.0, 2019. <https://geeq.io/wp-content/uploads/GEEQ-OFFICIAL-TECHNICAL-PAPER.pdf>
- [10] Sean Bowe, Jack Grigg, and Daira Hopwood. *Halo: Recursive Proof Composition without a Trusted Setup*. IACR ePrint 2019/1021. <https://eprint.iacr.org/2019/1021>
- [11] Benedikt Bünz, Alessandro Chiesa, Pratyush Mishra, and Nicholas Spooner. *Proof-Carrying Data from Accumulation Schemes*. IACR ePrint 2020/499. <https://eprint.iacr.org/2020/499>